Lecture Eleven/Twelve/Thirteen

Creative Thinking for AI and IoT Innovations

# Searching to find intelligent and creative outcome: Simple search to creative experimentation

Parag Kulkarni, PhD, DSc

Lecture Ten

# Searching to find intelligent and creative outcome: Simple search to creative experimentation

Parag Kulkarni, PhD, DSc

# Quick Recap

- Intelligent Systems
- Thinking
- Creativity
- Cognitive Flexibility
- Connectionism
- Multiplying effect of connectionism
- Uncertainty

# Search to Creative Search

- Selecting/searching amongst the available options
- What is meant by options? – could they be represented mathematically
- Then what are available options?
- Classification of available options:
  - Logical obvious options / illogical / non-obvious available options
  - Derived options
  - Indirect options
  - Short sighted – or otherwise (available options)
- Options those are not available:
  - Closed
  - Not opened yet
  - Not known or simply not possible with available knowledge
- Creating new options
- Discovering new options
- Inventing new options
- Search at different points across value chain

# When search could be termed as creative!

- Generates new option
- Discovers new concept, node or route
- Combine two or more nodes
- Creating new sets of actions
- Re-define neighbor

# Dynamic goals

- Changing goal  (dynamic goal states)
- Evolutionary termination criterion
- Multiple demarches with multiple goal states (population evolution) – Genetic algorithm and its variants

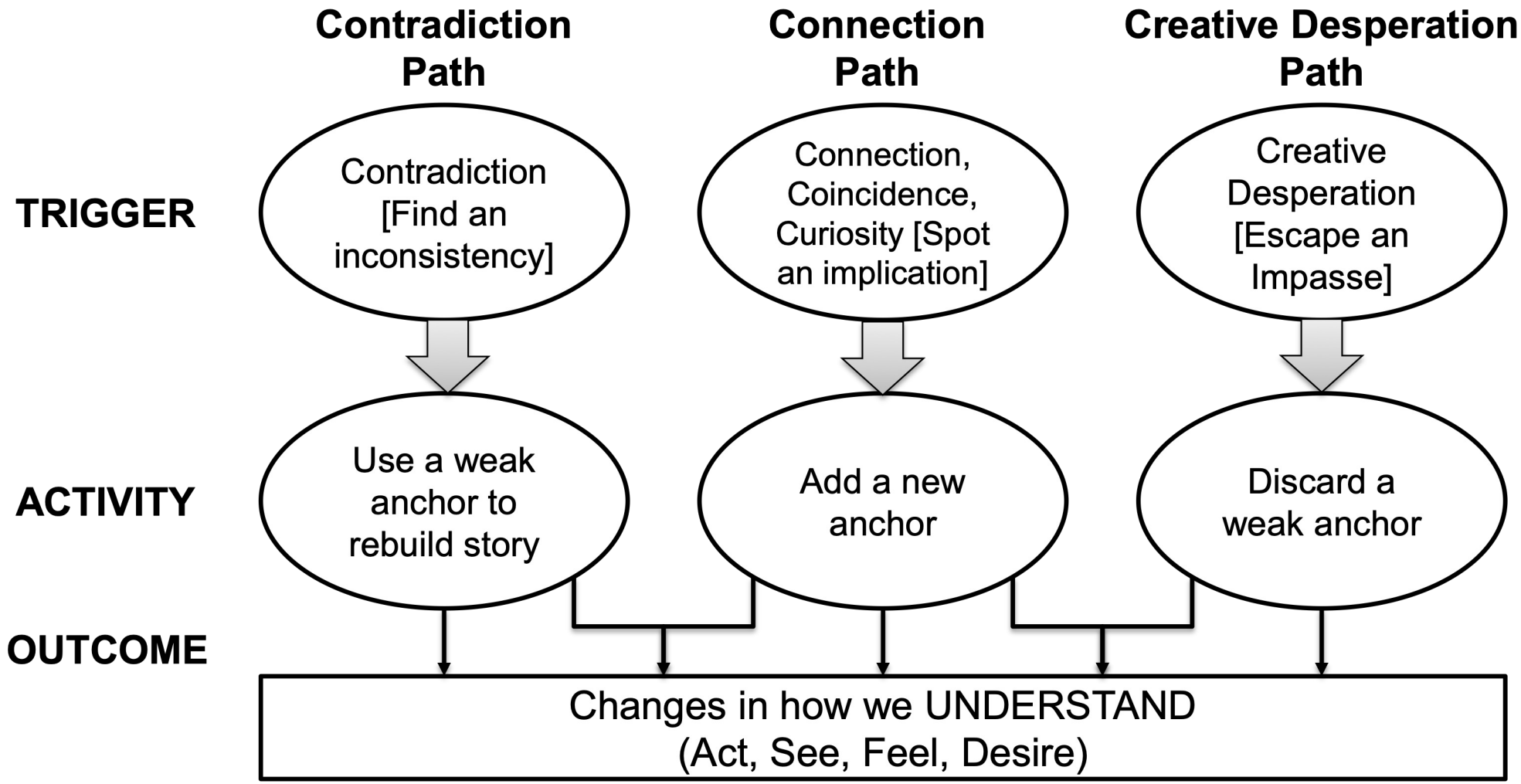
- *Creative heuristics can be derived from functional creativity and invention, and are (at least partly) transferable to other realms.*
- *A creative heuristic can help by overcoming fixation and/or resolving a contradiction.*

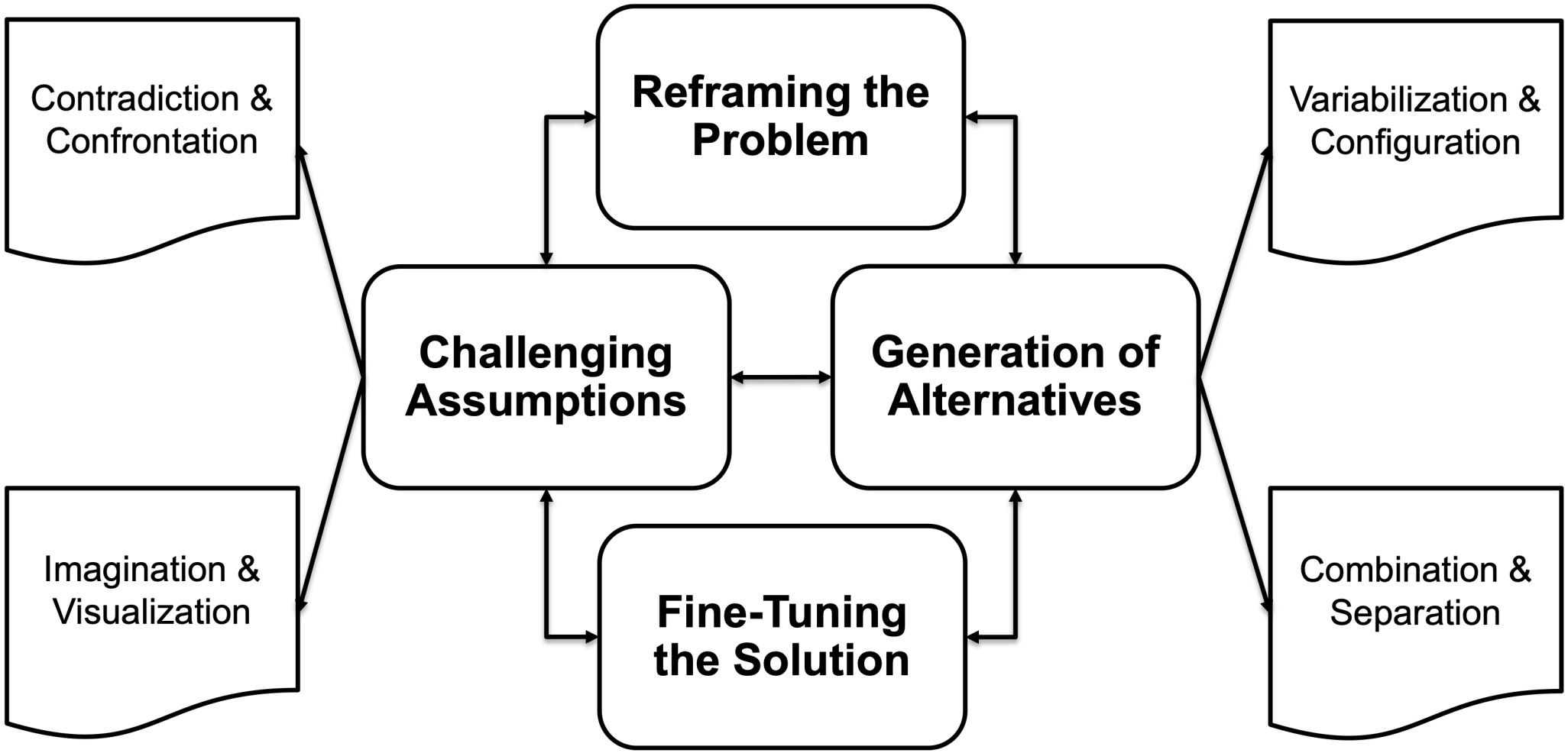
# Problems

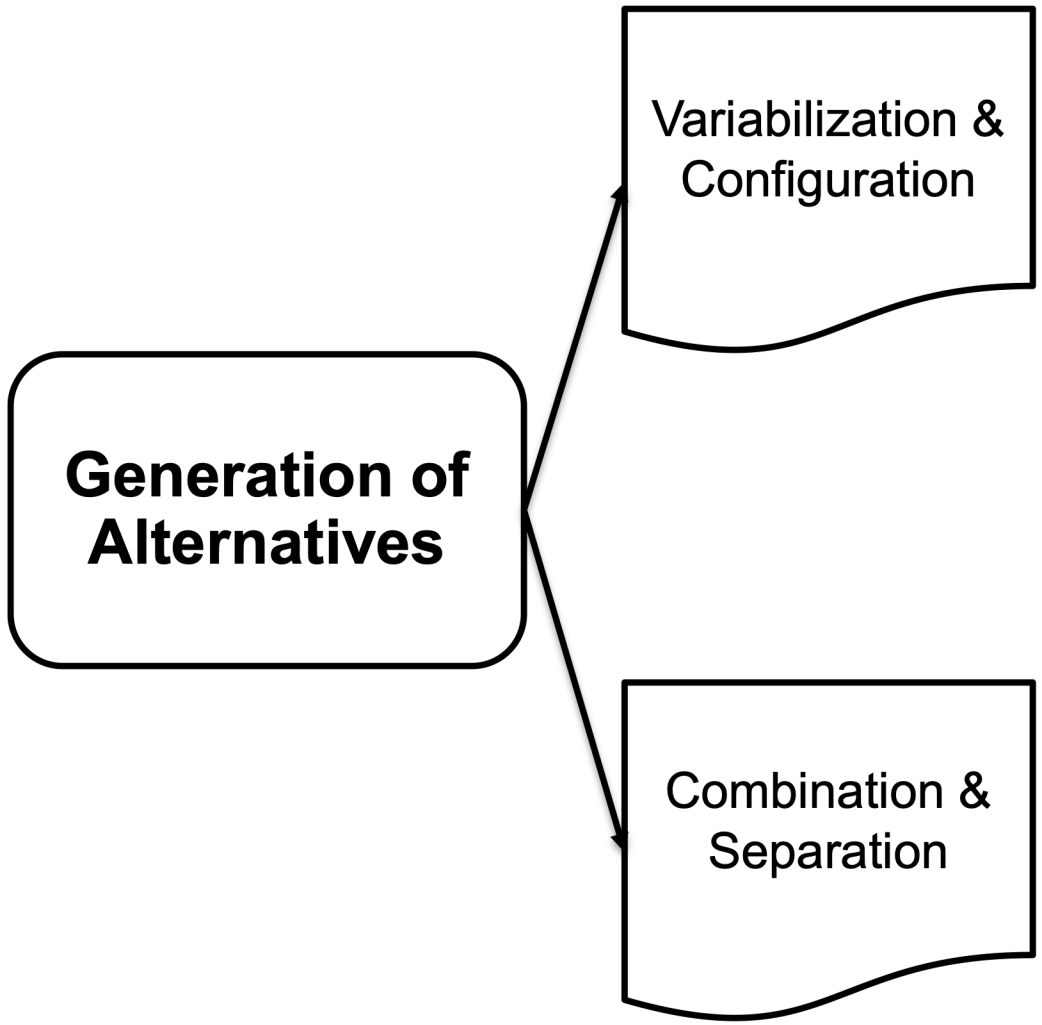
- Rarity
- Payoffs
- Oasis
- Plateau

# Create search space

- Broaden it
- Search space evaluation
- Incremental to radical


Contradiction Path
Connection Path
Creative Desperation Path
TRIGGER
ACTIVITY
OUTCOME
Contradiction [Find an inconsistency]
Connection, Coincidence, Curiosity [Spot an implication]
Creative Desperation [Escape an Impasse]
Use a weak anchor to rebuild story
Add a new anchor
Discard a weak anchor
Changes in how we UNDERSTAND
(Act, See, Feel, Desire)


Contradiction & Confrontation
Reframing the Problem
Variabilization & Configuration
Challenging Assumptions
Generation of Alternatives
Imagination & Visualization
Fine-Tuning the Solution
Combination & Separation




- Vary the Variable
- Use Creativity Checklist (e.g. SCAMPER)
- Find a Trajectory or Evaluation Function
- Add Relevant Features
- Package Relevant Features
- Improve the Human Interface
- Delete Irrelevant Features
- Abstract and Transform (Scale, Dimensionality, Matching)

- Consider the Negative/Inverse
- Combine Inventions with Complementary or Emergent Qualities
- Combine to Eliminate Redundancy
- Interpolate & Extrapolate
- Find a New Purpose
- Separate and Recombine
- Use Separation Principles

- Creativity is a process of knowledge combination, thus internal and external knowledge searching is important for creativity in both fields, particularly in the open innovation age.
- Compared to science research, technology development has clear task goals, which make it easier to identify the related external knowledge and integrate this knowledge and in turn improve employee creativity.
- Thinking searching – creativity in terms of machines

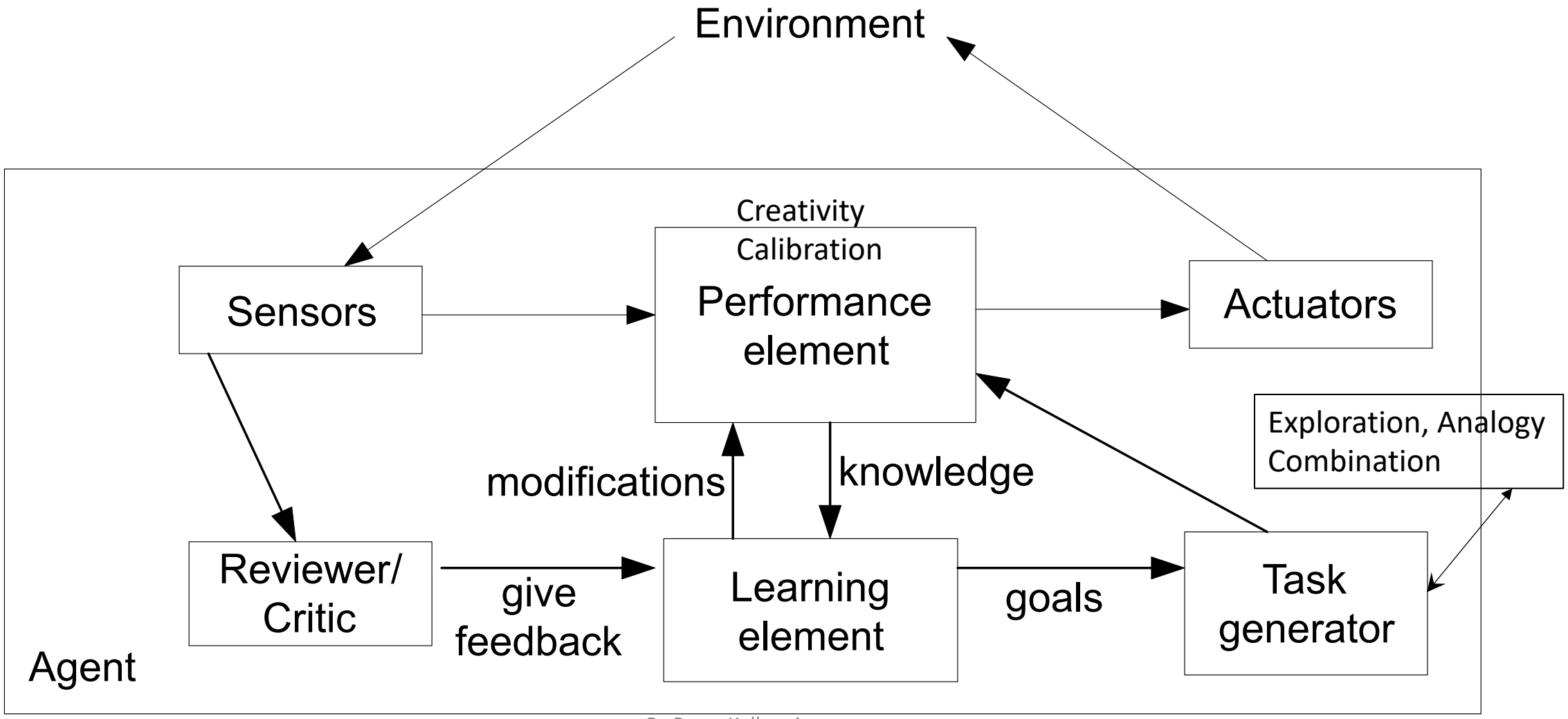
# Rational Agent Should

- Responsive: Respond in timely fashion to perceived environment.
- Pro-active: Should exhibit opportunistic, goal directed behavior and take the initiative wherever necessary.
- Social: Should be able to interact when they deem appropriate with other artificial agents, humans in order to compete problem solving.

# Creative Agent

- Function that can measure extent of creativity
- Has ability to perform obvious tasks and creative task
- Rational but non-obvious
- What could be structure of creative agent
- What type of AI problems creative agent is expected to address
- Classify tasks and outcomes on creativity scale
- Sensing previous performance – sensing competing performance

# Creative Agents



# Searching

- Searching is a basic action
- We search problems, solutions, shops and what not
- Have you search anything recently
- What algorithm did you use
- Is it random/ad hoc/planned/organized
- What are searching applications

# SEARCHING FOR SOLUTIONS

- Problem can be solved by searching through the search space
- Root of search tree is a search node corresponding to initial state
- Checking whether it is a goal state
- Expanding and generating new states

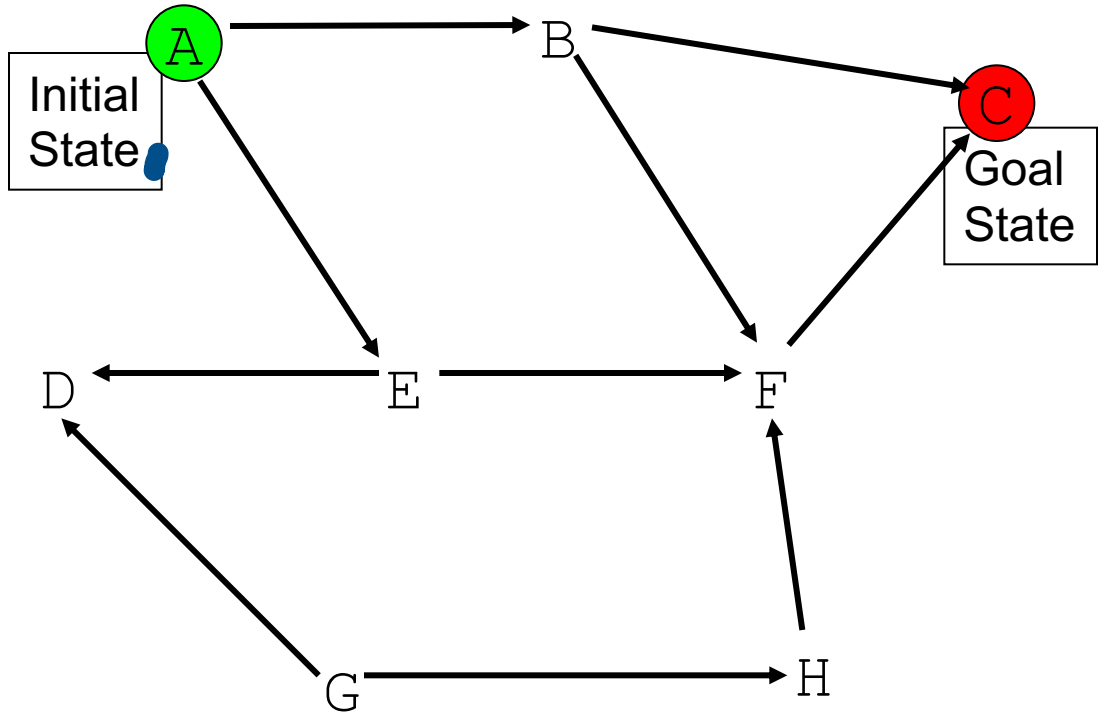
# Search Strategy

- The choice of which state to expand depends on search strategy
- State space and search tree
- In the theory of discrete dynamical systems, a state space is a directed graph where each possible state of a dynamical system is represented by a vertex and there is a directed edge from a to b if and only if ƒ(a) = b where the function f defines the dynamical system.

# AN EXAMPLE PROBLEM: SEARCHING A GRAPH

```
link(g,h).
link(g,d).
link(e,d).
link(h,f).
link(e,f).
link(a,e).
link(a,b).
link(b,f).
link(b,c).
link(f,c).
```

State-Space




```
go(X,X,[X]).
go(X,Y,[X|T]):-
        link(X,Z),
        go(Z,Y,T).
```

Simple search algorithm

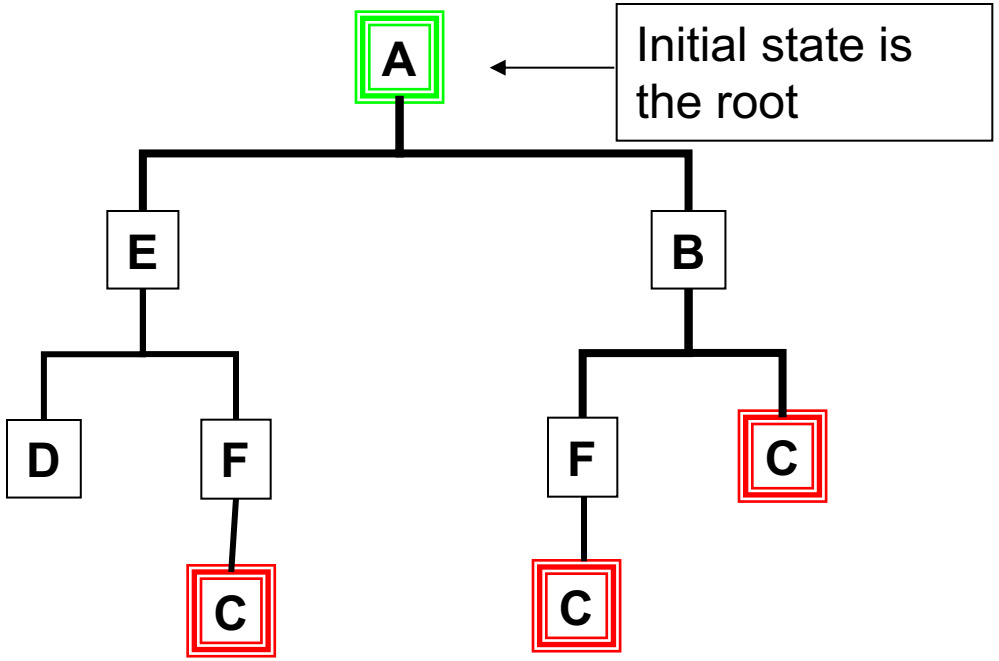
```
| ?- go(a,c,X).
X = [a,e,f,c] ? ;
X = [a,b,f,c] ? ;
X = [a,b,c] ? ;
no
```

Consultation

By Parag Kulkarni

# STATE-SPACE REPRESENTATION

```
link(g,h).
link(g,d).
link(e,d).
link(h,f).
link(e,f).
link(a,e).
link(a,b).
link(b,f).
link(b,c).
link(f,c).
```




- An abstract representation of a state-space is a downwards growing tree. Connected *nodes* represent states in the domain.
- The *branching factor* denotes how many new states you can move to from any state. This problem has an average of 2.
- The *depth* of a node denotes how many moves away from the initial state it is. 'C' has two depths, 2 or 3.

By Parag Kulkarni

# Uninformed Search Strategies

- **Uninformed search strategies**
  - Also known as “blind search,” uninformed search strategies use no information about the likely “direction” of the goal node(s)
  - Uninformed search methods: Breadth-first, depth-first, depth-limited, uniform-cost, depth-first iterative deepening, bidirectional
- **Informed search strategies**
  - Also known as “heuristic search,” informed search strategies use information about the domain to (try to) (usually) head in the general direction of the goal node(s)
  - Informed search methods: Hill climbing, best-first, greedy search, beam search, A, A*

# EVALUATING SEARCH STRATEGIES

- **Completeness**
  - Guarantees finding a solution whenever one exists
- **Time complexity**
  - How long (worst or average case) does it take to find a solution? Usually measured in terms of the number of nodes expanded
- **Space complexity**
  - How much space is used by the algorithm? Usually measured in terms of the maximum size of the “nodes” list during the search
- **Optimality/Admissibility**
  - If a solution is found, is it guaranteed to be an optimal one? That is, is it the one with minimum cost?

# Breadth First Search Algorithm

- Enqueue nodes on nodes in **FIFO** (first-in, first-out) order.
- **Complete**
- **Optimal** (i.e., admissible) if all operators have the same cost. Otherwise, not optimal but finds solution with shortest path length.
- **Exponential time and space complexity**, $O(b^d)$, where d is the depth of the solution and b is the branching factor (i.e., number of children) at each node
- Will take a **long time to find solutions** with a large number of steps because must look at all shorter length possibilities first
  - A complete search tree of depth d where each non-leaf node has b children, has a total of $1 + b + b^2 + ... + b^d = (b^{(d+1)} - 1)/(b-1)$ nodes
  - For a complete search tree of depth 12, where every node at depths 0, ..., 11 has 10 children and every node at depth 12 has 0 children, there are $1 + 10 + 100 + 1000 + ... + 10^{12} = (10^{13} - 1)/9 = O(10^{12})$ nodes in the complete search tree. If BFS expands 1000 nodes/sec and each node uses 100 bytes of storage, then BFS will take 35 years to run in the worst case, and it will use 111 terabytes of memory!

# Breadth First Search

- Root node expanded first, then all successor of root node from left to right and then their successors
- Not always optimal, required more memory, may take more time

# Depth First Search

- Enqueue nodes on nodes in **LIFO** (last-in, first-out) order. That is, nodes used as a stack data structure to order nodes.
- **May not terminate** without a “depth bound,” i.e., cutting off search below a fixed depth D ( “depth-limited search”)
- **Not complete** (with or without cycle detection, and with or without a cutoff depth)
- **Exponential time**, $O(b^d)$, but only **linear space**, O(bd)
- Can find **long solutions quickly** if lucky (and **short solutions slowly** if unlucky!)
- When search hits a dead end, can only back up one level at a time even if the “problem” occurs because of a bad operator choice near the top of the tree. Hence, only does “chronological backtracking”

# DEPTH FIRST SEARCH

- A variant is backtracking search
- Only one successor generated at a time rather than all successors
- Each partially expanded node remembers which successor to generate next
- Depth first search may make a wrong choice and may go down a very long path

# DEPTH-FIRST VS. BREADTH-FIRST

Advantages of depth-first:

- Simple to implement;
- Needs relatively small memory for storing the state-space.

Advantages of breadth-first:

- Guaranteed to find a solution (if one exists);
- Depending on the problem, can be guaranteed to find an *optimal* solution.

Disadvantages of depth-first:

- Can sometimes fail to find a solution;
- Not guaranteed to find an *optimal* solution;
- Can take a lot longer to find a solution.

Disadvantages of breadth-first:

- More complex to implement;
- Needs a lot of memory for storing the state space if the search space has a high branching factor.

# UNIFORM-COST (UCS)

- Enqueue nodes by **path cost**. That is, let g(n) = cost of the path from the start node to the current node n. Sort nodes by increasing value of g.
- Called *"Dijkstra's Algorithm"* in the algorithms literature and similar to *"Branch and Bound Algorithm"* in operations research literature
- **Complete (*)**
- **Optimal/Admissible** (*)
- Admissibility depends on the goal test being applied *when a node is removed from the nodes list*, not when its parent node is expanded and the node is first generated
- **Exponential time and space complexity**, $O(b^d)$

# UNIFORM-COST (UCS)

- BFS Always expand shallowest unexpanded node
- While UCS expands the node n with lowest path cost
- The completeness is guaranteed if the cost of every step is greater than zero- otherwise it may get stuck in infinite loop if it expands the node that has zero-cost action leading back to same state.

# Depth Limited Search

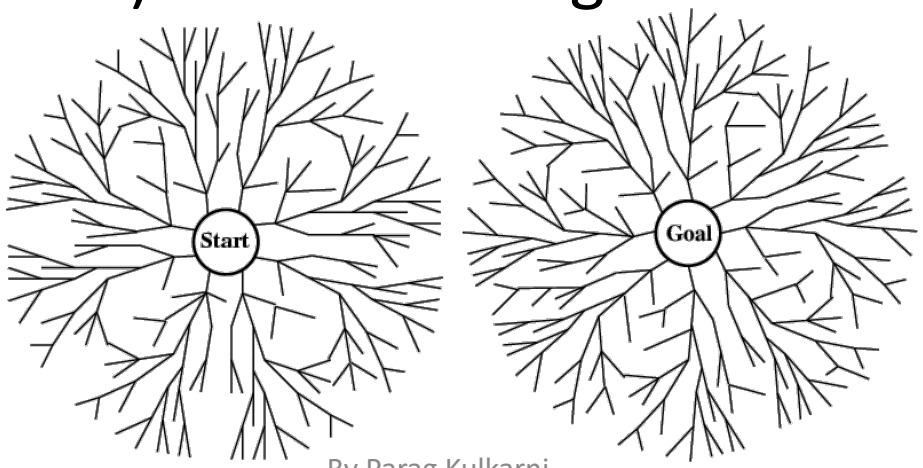
- The typical problems encountered in depth first search due to unbounded trees can be alleviated by supplying depth first search with a predetermined depth limit.
- This solves infinite path problem
- In this case shallowest goal beyond the depth limit is missed.
- Depth first search can be viewed as a special case of depth limited search with limit infinity

# Iterative deepening depth first search

- If the optimal solution is the shortest path from the initial state to the goal state depth-first search will usually not find this.
- We need to vary the depth at which we look for a solution; increasing the depth every time we have exhausted all nodes at a particular depth.
- This finds the best depth limit
- It does that by gradually increasing the limit until the goal is found
- This happens when the depth reaches d – i.e. the depth of the shallowest goal
- This technique combines advantages of breadth first search and depth first search

# Bidirectional Search

- Alternate searching from the start state toward the goal and from the goal state toward the start.
- Stop when the frontiers intersect.
- Works well only when there are unique start and goal states.
- Requires the ability to generate “predecessor” states.
- Can (sometimes) lead to finding a solution more quickly.



# Bidirectional Search

- Bidirectional search is implemented by having one or both of the searches check each node before it is expanded to see if it is in the fringe of the other search tree – if it is so then solution is found
- In this case it is required that at least one of the search tree has to be kept in the memory
- This space requirement is major weakness of bidirectional search
- The time complexity makes it attractive but faces issues in backward searching

# Avoiding Repeated States

- Repeated states can make solvable problem unsolvable if algorithm fails to detect them
- Detection usually means comparing the node about to be expanded to those that have already been expanded – if it match then algorithm has discovered two paths to same state and can discard one of them
- Can help in detecting looping paths and discarding them
- It does not avoid exponential proliferation of non looping paths

# Avoiding repeated states

- Tree search algorithm can be modified to include data structure called closed list - which stores every expanded node
- If the current node matches the node on the closed list – it is discarded instead being of expanded
- It can result in some disadvantages with reference to detecting optimal path in case of graph

# Searching with Partial Information

- The incomplete knowledge of states and actions can lead to three distinct problem types:

❑Sensorless problems: – No sensor – Then it could be in any of the several possible initial states – each action can lead to one of the several successors

❑Contingency problems: - Percept provides new information after each action. Partially observable environment – each possible percept define a contingency that must be planned for -

❑Exploration problems: When the states and actions of the environment are unknown – agent must act to discover them

- **Uninformed search strategies**
  - Also known as “blind search,” uninformed search strategies use no information about the likely “direction” of the goal node(s)
  - Uninformed search methods: Breadth-first, depth-first, depth-limited, uniform-cost, depth-first iterative deepening, bidirectional
- **Informed search strategies**
  - Also known as “heuristic search,” informed search strategies use information about the domain to (try to) (usually) head in the general direction of the goal node(s)
  - Informed search methods: Hill climbing, best-first, greedy search, beam search, A, A*



# Lecture Twelve

## Demystifying Creativity from evolution perspective:

Heuristic functions and searches
Associative and Evolutionary aspects
Evolution
Evolutionary Algorithm (Genetic Algo)

Parag Kulkarni, PhD, DSc

# Mid semester exam

- Date 22 Oct
- Based on topics covered till 19 Oct
- Format
  - In class: Online Close Book (submit pdf)
- Question bank will be made available 12/18 HRS in advance
- Question to be answered by each candidate will be given at the beginning of class on 22 Oct

# Quick Recap

- Creativity
- Historical and Psychological Creativity
- Agent
- Intelligent Agent
- Types of agent
- Searching
- Heuristic and non-heuristic searches

# Pointers with Objectives

CONCEPTS

- What is AI?
- Human Learning,
- Creativity
- Thinking
- Conceptual Spaces
- Experience and Examples
- Predictions
- Intelligence
- Learning models and Paradigms
- Building blocks of Intelligent systems
- Algorithms
- Applications
- Case-studies

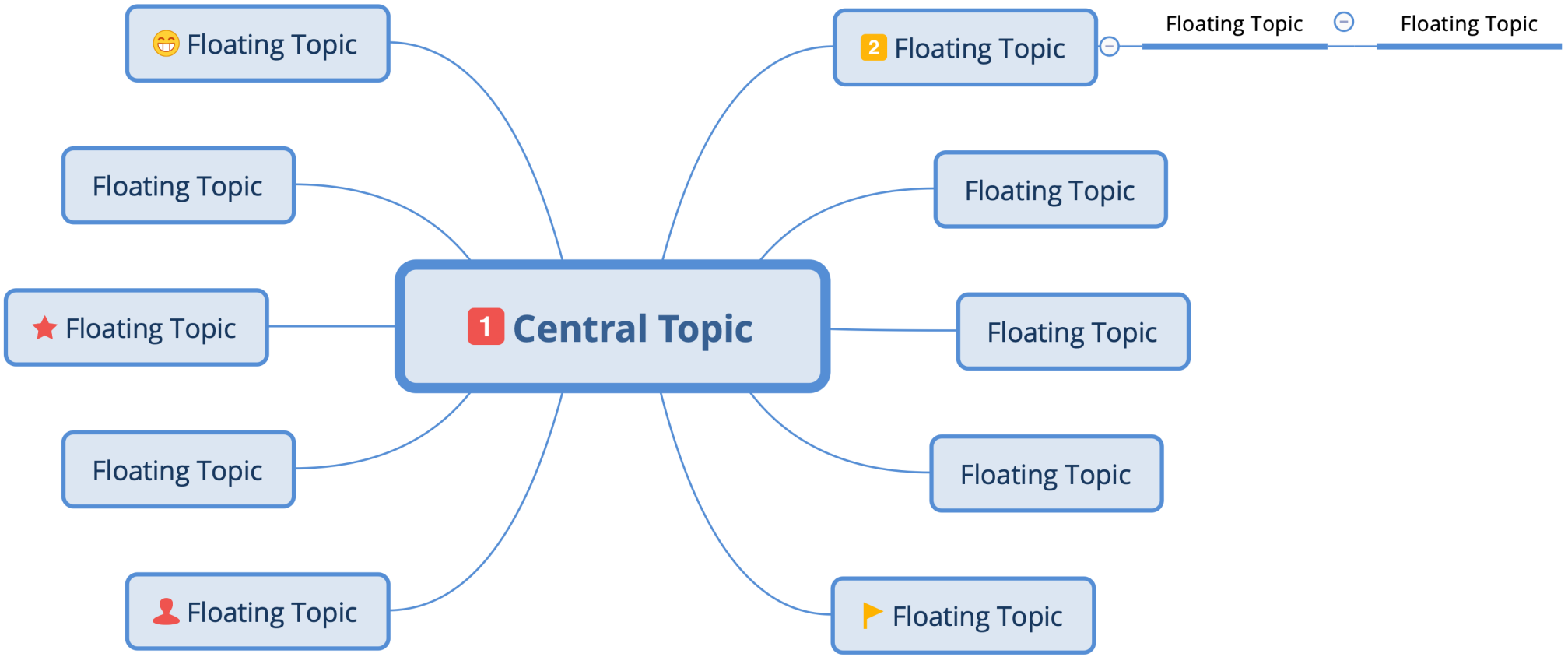
PRODUCTS and SOLUTIONS

APPROACHES

- Statistical
- Associative
- Behaviorism
- Evaluation
- Evolution
- Exploration
- Genetic
- Symmetry
- Analogy
- Combination
- Time-Series Predictive
- Immune System "Group Behavior"
- Discriminative
- Generative

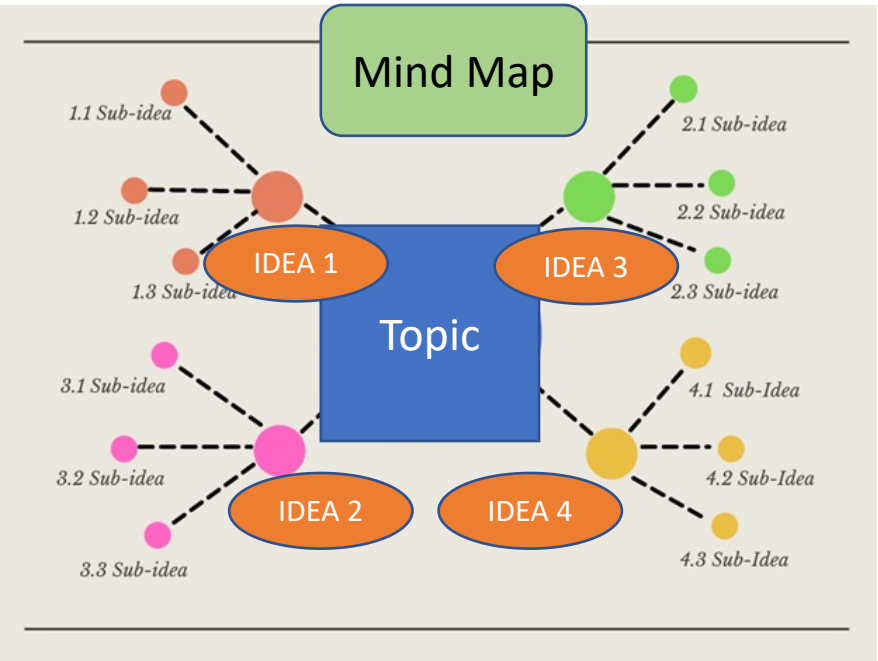
# Connections and Creativity

- “Creative people tend to be open to experience, to seeing things in new ways.” - Is that about connecting?
- Facilitation to make connections
- Seeing the connection between things others could not see
- Map of connectivity and connectivity patterns
- Map of Mind: exploring connectivity to realize untapped potential
- Exploration through Possible connections and relationships
- Exploration of invisible relationships
- Creating new relationships


Floating Topic
Floating Topic
Floating Topic
Floating Topic
Floating Topic
Floating Topic
1 Central Topic
2 Floating Topic
Floating Topic
Floating Topic
Floating Topic
Floating Topic
Floating Topic
Floating Topic
Floating Topic

# Example mind-maps

- Career
- Job selection
- Finding a route
- Product development
- Requirement gathering
- Design

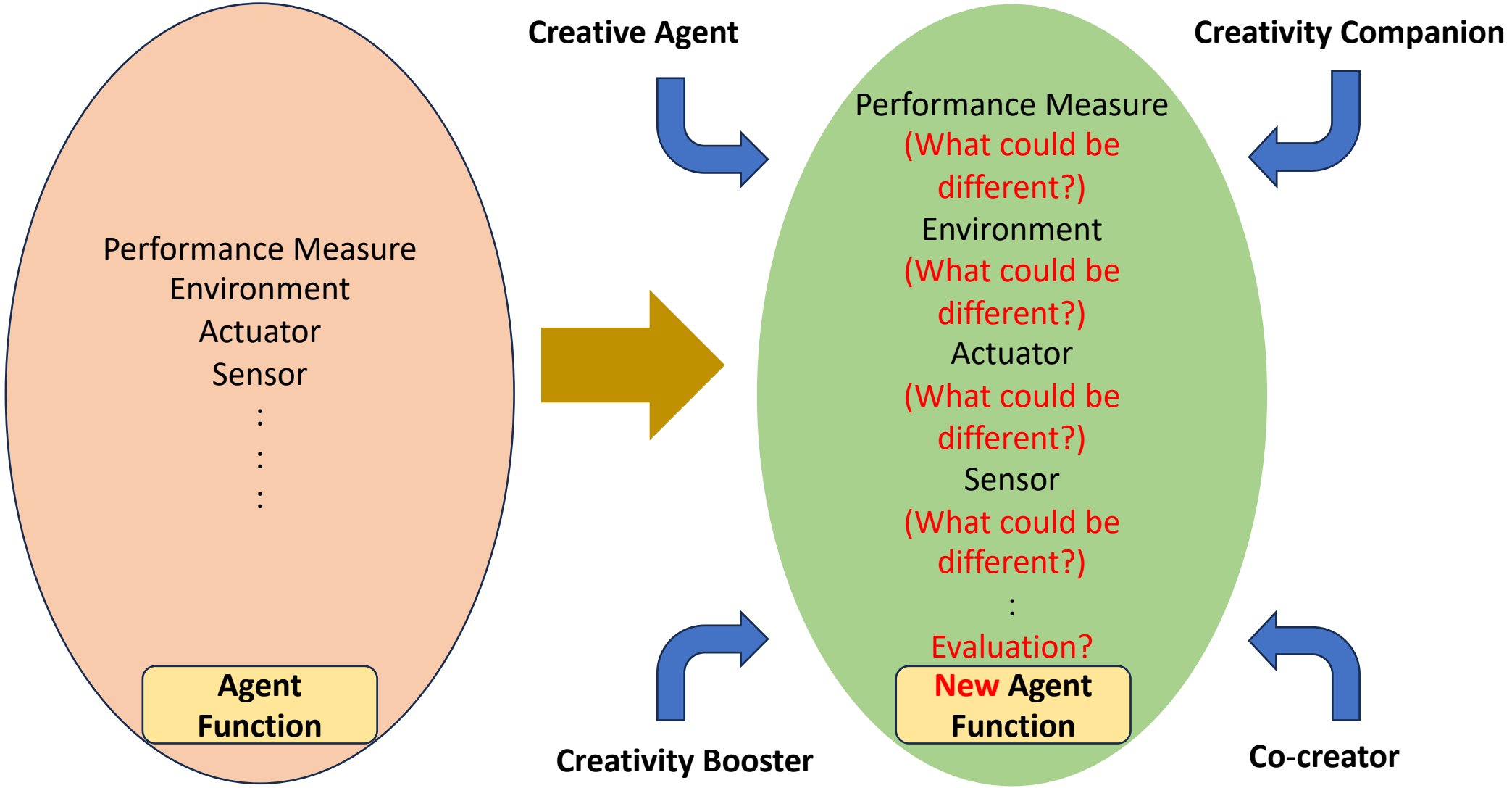


# Connections

- Connections as vectors
- Context based connection
- Stability not in terms of value but band
- Connection selection
- Connection based co-operation
- Evolving connections

# Intelligent Agent

# Creative Agent



# Choose!

**Inertia diluent** – (Respect inertia – do not work against it – just reduce the impact of it): Make choices easy to access. Default is better

**Less is more:** Less choices more returns. Keep the choices to limited but attractive number.

**Story and relevance:** There should be a path or story that takes user from one choice to another. Stories are about engagement.

**Surprise at critical point**: surprises just in vicinity of choice points. Happy moment?

**Slow to heat and gradual cooling:** Best choice should be introduced gradually and it should be kept warm after introduction.

**Neighbours matter:** Careful selection of choice clusters and choices near key choice point is crucial.

© Parag Kulkarni

# Laws of choosing based on Gestalt's laws

- *The law of rationality*: filter all the rational choices those are reachable from current state.
- *The law of next choice*: A suite of objects in close proximity are viewed together as a collective group under an assumed association. The next choices should be placed carefully to improve chances of target product.
- *The law of similarity*: Among a collection of objects, a grouping is done on the premises of some sort of resemblance to each other. The similar choices are viewed as a group. Associating target choices with selective choices is the key.

© Parag Kulkarni

- *The law of unit formation*: Sub-choices contributing to major choices are brought together to form a single choice. Such formation may impact choice path.
- *The law of default and simple choice*: The complex choices are quickly mapped to simple and default object. This mapping may undervalue or overvalue the complex choices. It makes it necessary to simplify target choices and paths leading to target choices.
- *The law of symmetry*: Bringing together symmetrical target choices can be looked at possible option to enhance the chances.
- *The law of storytelling*: Implies the grouping together of objects that have the same trend of motion and associated with each other tells the story leading to logical conclusion regarding utility, value creation or differentiation. Story is about the choice path.

© Parag Kulkarni

# System one – Hight recall continuous responder

- **Automatic —** generates impressions and feelings without any sense of deliberate effort.
- **Always on —** works continuously, even when we are not consciously aware of it doing so.
- **Associative —** draws on memories, patterns, and associations built up over a lifetime of experience.
- **Fast by design —** produces judgments that often feel like simple perception rather than active reasoning.
- **Cannot be switched off —** it keeps running in the background no matter what we intend to do.

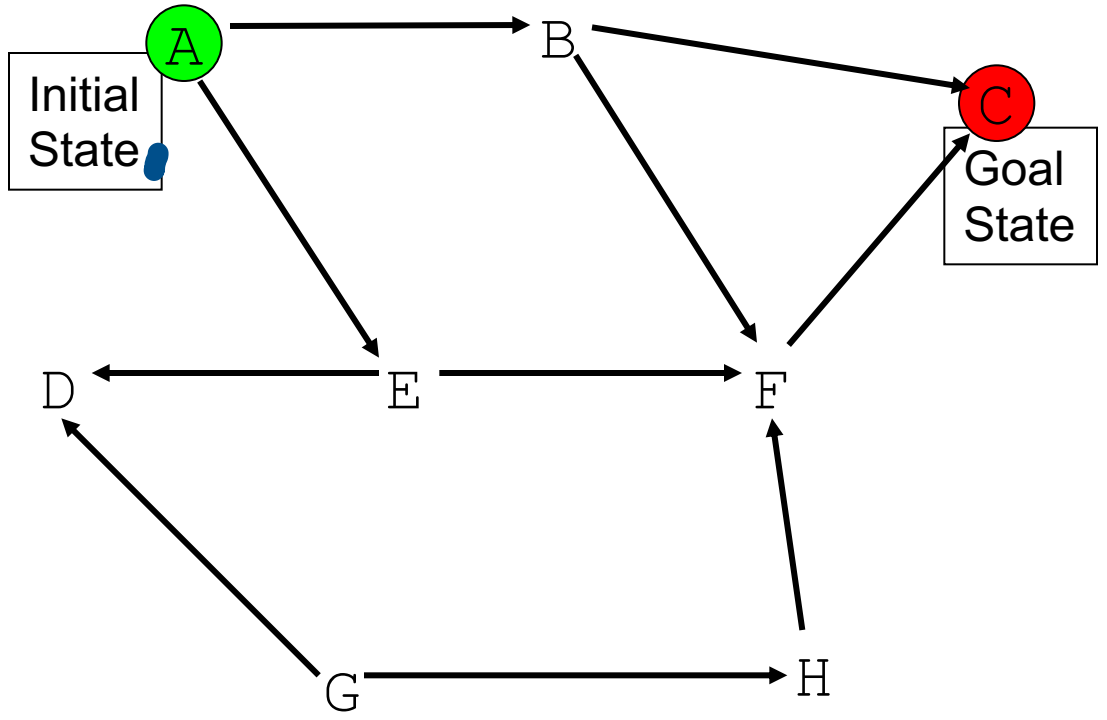
# Decoding System 2

- **Effortful —** engages in mental activities that require focused, sustained attention. The activities that has some learning part too…
- **Experienced as agency —** associated with the subjective feeling of choosing, concentrating, and reasoning. Tacit knowledge?
- **Deliberately activated —** unlike System 1, it does not run automatically and must be switched on.
- **Rule-based —** handles complex computation, logical analysis, and step-by-step procedures.
- **Limited in capacity —** can only hold a small number of things in focus at any one time.

# AN EXAMPLE PROBLEM: SEARCHING A GRAPH

```
link(g,h).
link(g,d).
link(e,d).
link(h,f).
link(e,f).
link(a,e).
link(a,b).
link(b,f).
link(b,c).
link(f,c).
```

State-Space




```
go(X,X,[X]).
go(X,Y,[X|T]):-
        link(X,Z),
        go(Z,Y,T).
```

Simple search algorithm

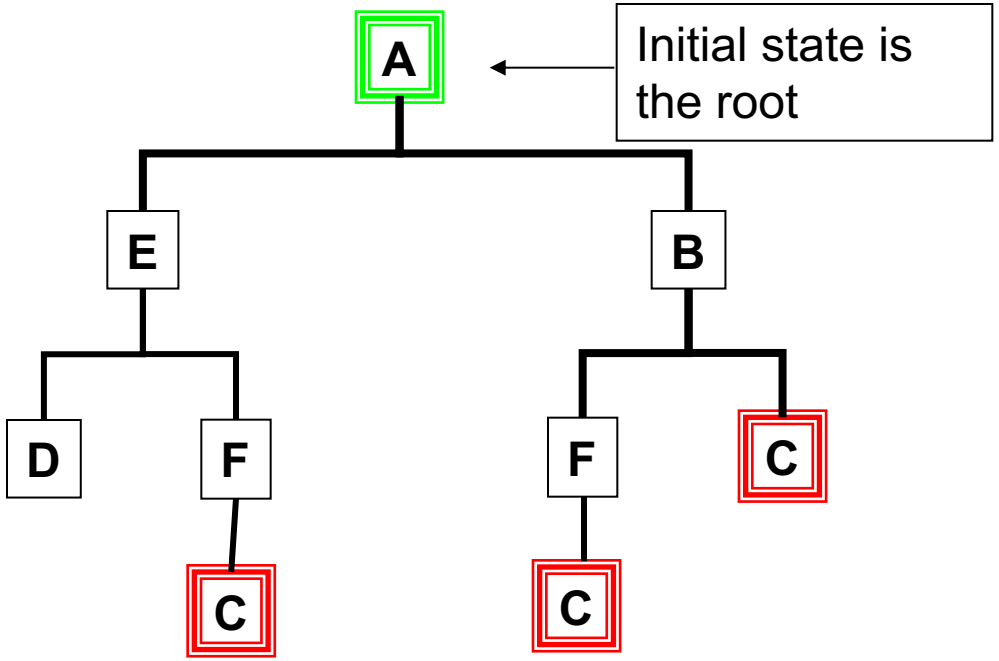
```
| ?- go(a,c,X).
X = [a,e,f,c] ? ;
X = [a,b,f,c] ? ;
X = [a,b,c] ? ;
no
```

Consultation

By Parag Kulkarni

# STATE-SPACE REPRESENTATION

```
link(g,h).
link(g,d).
link(e,d).
link(h,f).
link(e,f).
link(a,e).
link(a,b).
link(b,f).
link(b,c).
link(f,c).
```




- An abstract representation of a state-space is a downwards growing tree. Connected *nodes* represent states in the domain.
- The *branching factor* denotes how many new states you can move to from any state. This problem has an average of 2.
- The *depth* of a node denotes how many moves away from the initial state it is. 'C' has two depths, 2 or 3.

By Parag Kulkarni

# Uninformed Search Strategies

- **Uninformed search strategies**
  - Also known as “blind search,” uninformed search strategies use no information about the likely “direction” of the goal node(s)
  - Uninformed search methods: Breadth-first, depth-first, depth-limited, uniform-cost, depth-first iterative deepening, bidirectional
- **Informed search strategies**
  - Also known as “heuristic search,” informed search strategies use information about the domain to (try to) (usually) head in the general direction of the goal node(s)
  - Informed search methods: Hill climbing, best-first, greedy search, beam search, A, A*

# Availability Heuristic

- **What is it? —** people judge the frequency or likelihood of an event by how easily relevant examples come to mind. (Top of mind rules the thoughts)

- **Ease as evidence —** easily recalled instances are treated as more common, regardless of their actual statistical frequency.

- **What boosts recall —** recency, vividness, and emotional intensity all make an example easier to recall — and feel more frequent.

- **Fast and associative —** relies on quick memory retrieval rather than careful statistical reasoning.

- **Usually helpful —** recent, memorable events are often genuinely relevant, but this shortcut misleads when memory is skewed.

# Representativeness Heuristic

- **What is it? —** likelihood is judged by how closely something resembles a familiar category or prototype.
- **Resemblance as evidence —** the more an example "looks like" a typical case, the more probable it's assumed to be.
- **Fast pattern-matching —** operates through rapid matching rather than deliberate statistical calculation.
- **Overrides base rates —** often outweighs more reliable cues, such as known base rates or sample size.
- **Efficient but risky —** great for quick categorization, but prone to error when surface similarity misleads.

# Injecting Heuristic into Agents

- **Optimization's blind spot —** pure search and optimization often produce efficient but predictable, uncreative solutions.
- **A spark of novelty —** human-style heuristics introduce shortcuts that can surface non-obvious, creative options.
- **Fast prioritization —** mimicking availability and representativeness lets agents favor promising ideas without exhaustive evaluation.
- **A creative pre-filter —** these heuristics can narrow a vast possibility space before deeper evaluation begins.
- **A deliberate trade-off —** some bias is accepted in exchange for speed, flexibility, and creative range.

# Availability Heuristic

- making judgment regarding the likelihood of an event based on information that quickly comes into mind.
- making decisions, people typically rely on the past knowledge or experience of an event.
- judge a situation based on the examples of similar situations that come to mind.

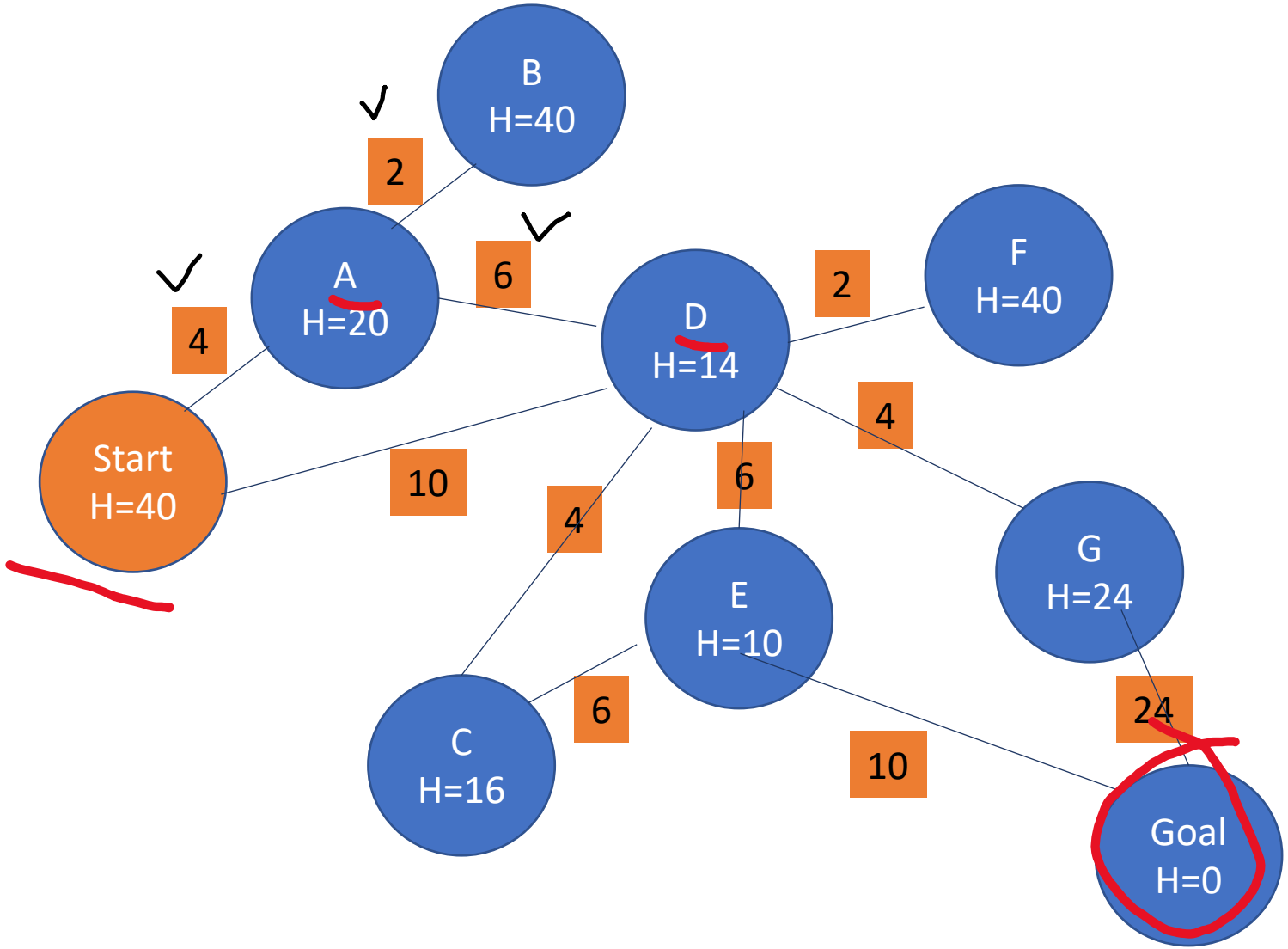
# Representativeness Heuristic

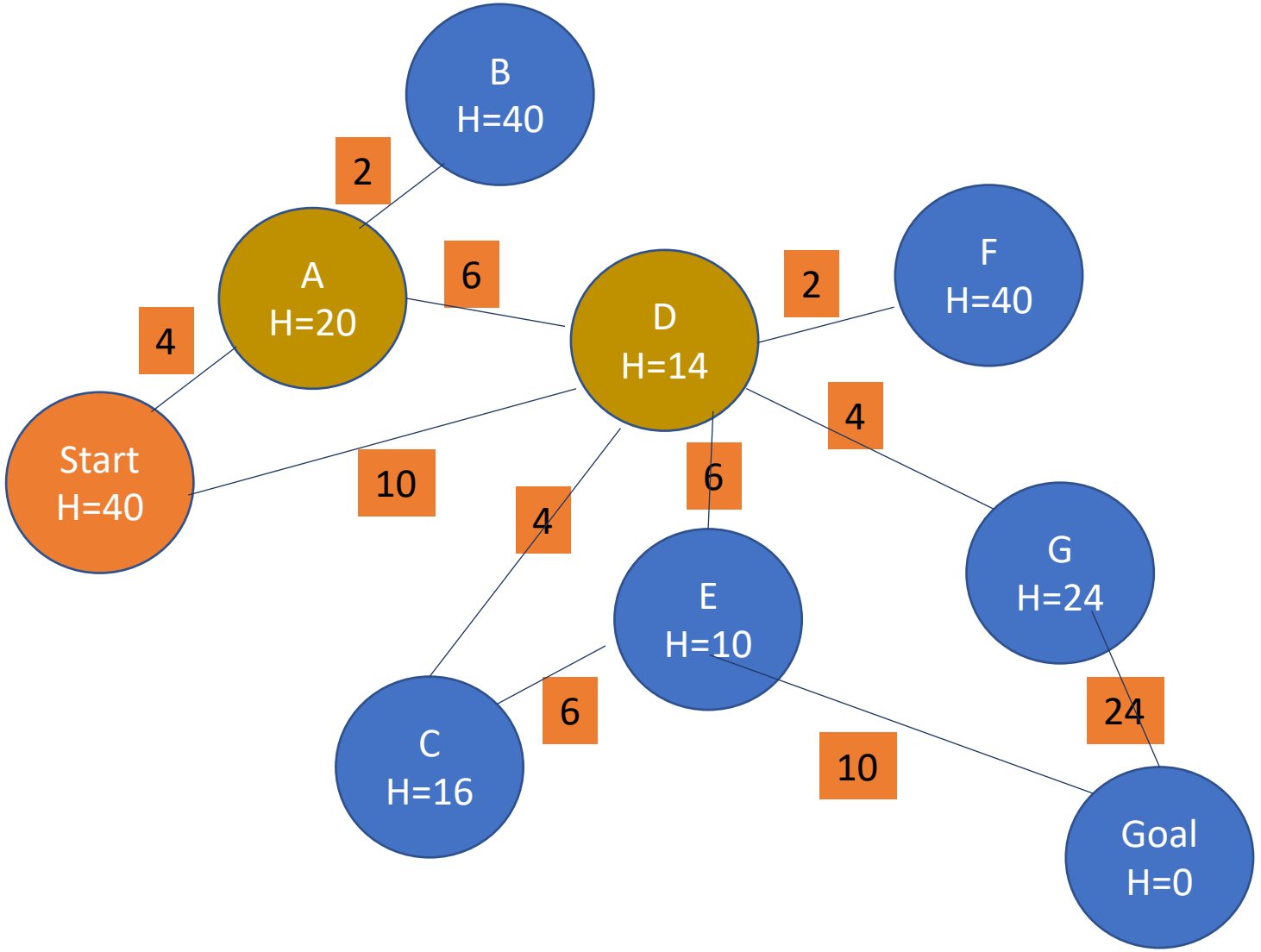
- evaluate an event's probability on the basis of its similarity with another event.
- It could be associating properties and outcome of event to the neighbours, packaging, …

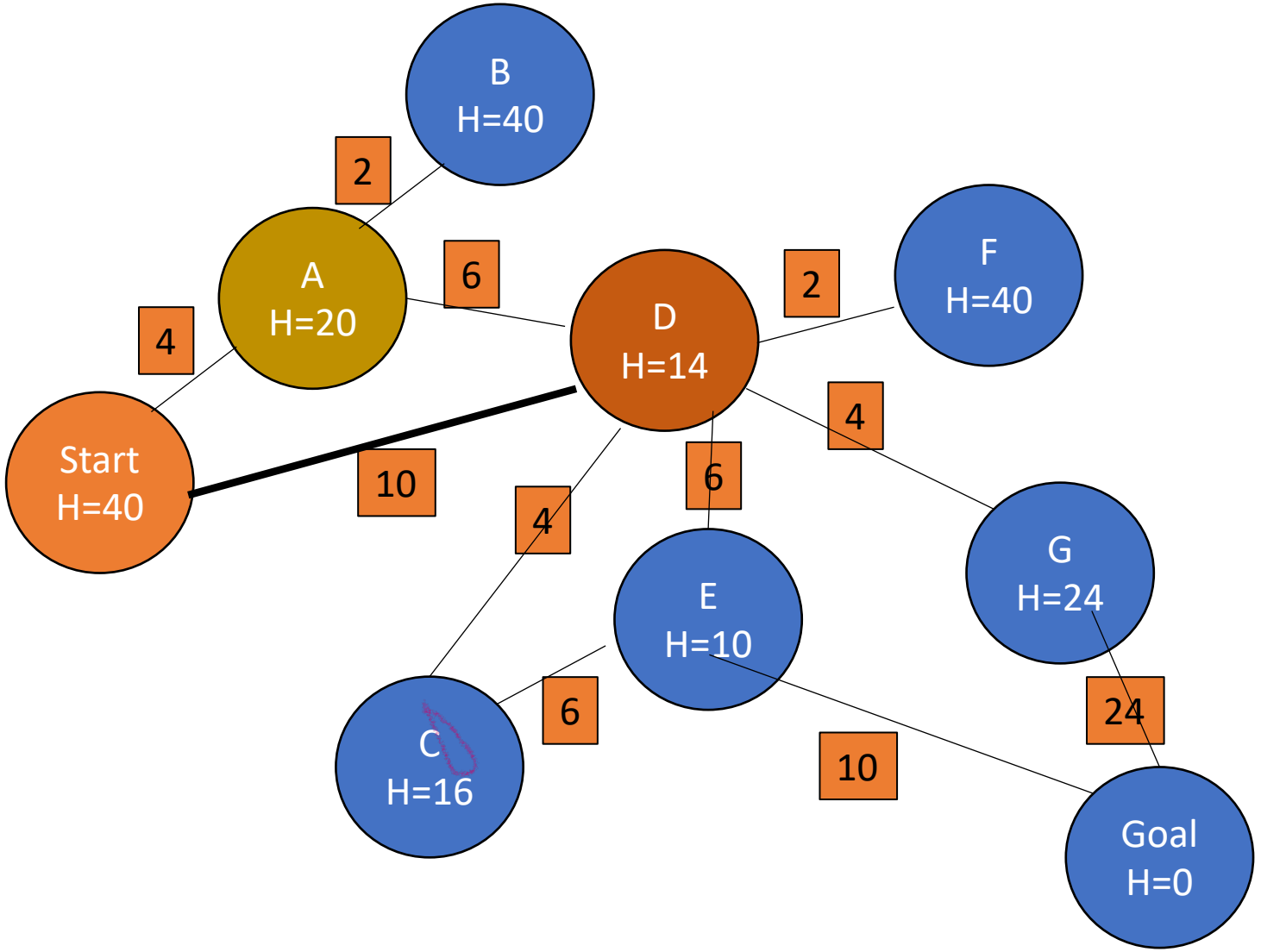
# Affect Heuristic

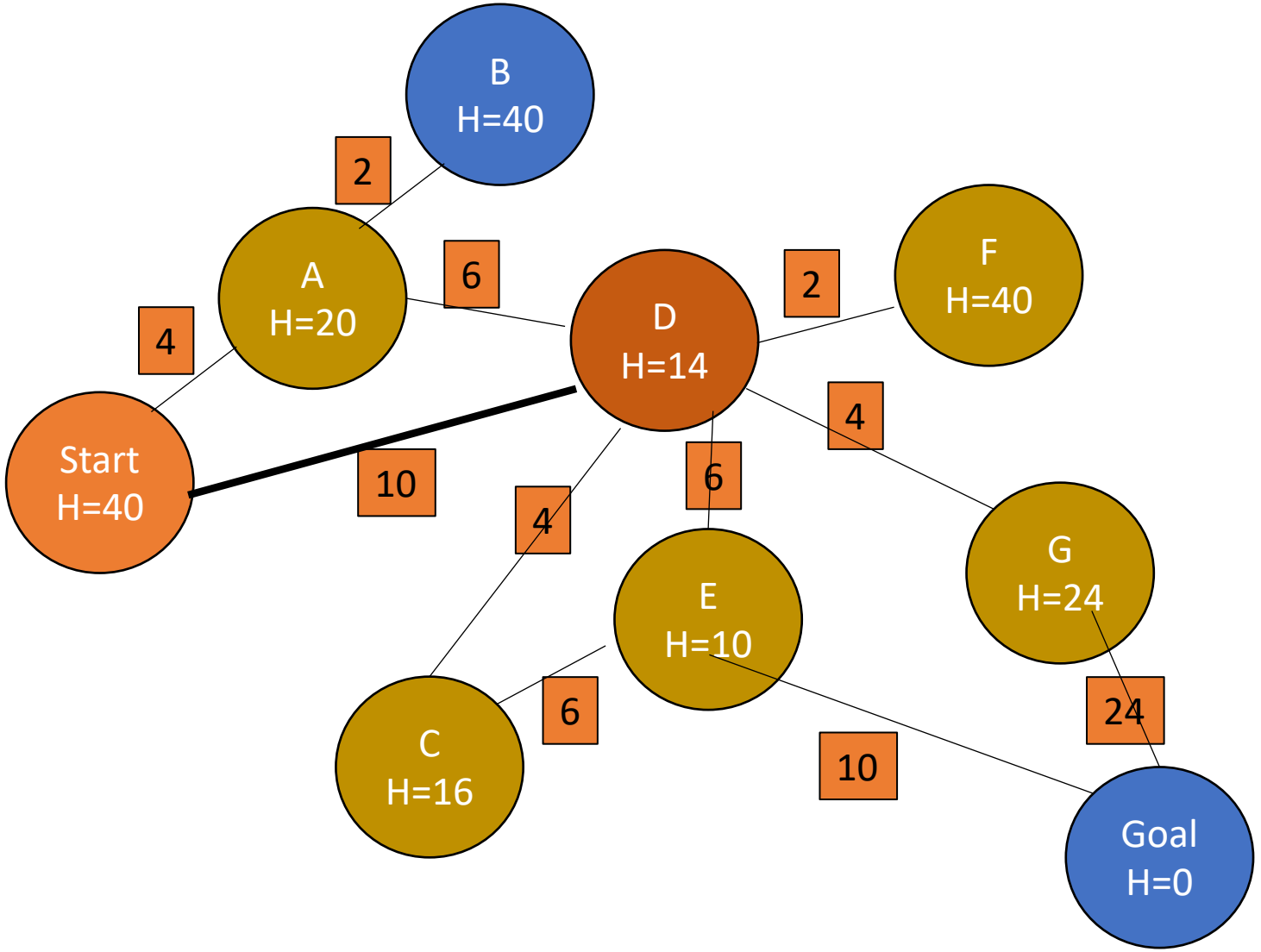
- negative and positive feelings that are linked with a certain stimulus.
- quick feelings that are based on past beliefs.
- emotional response to a stimulus that can affect the decisions
- Immediate emotional response based on associated stimulus

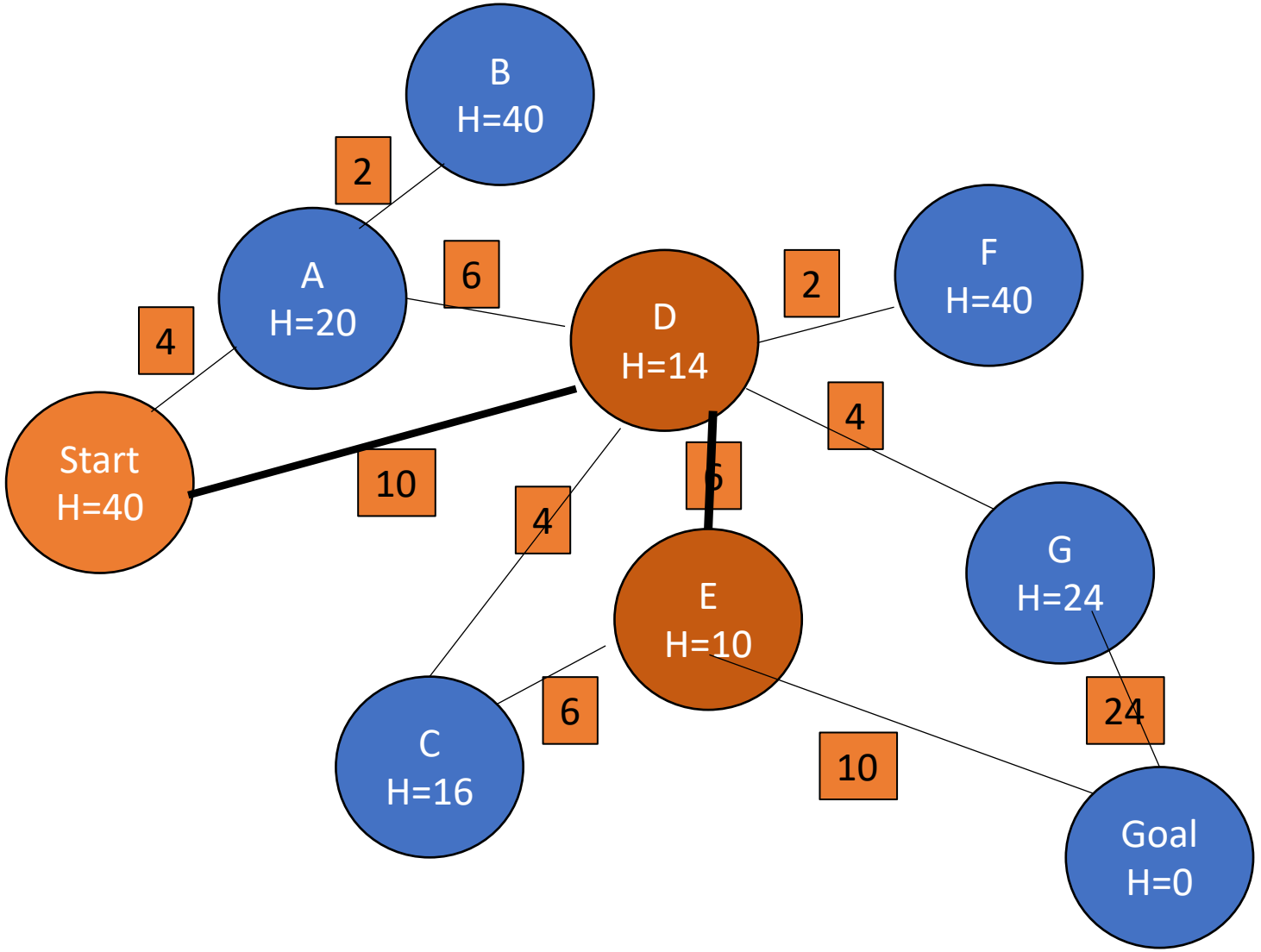
# Greedy Best First Search

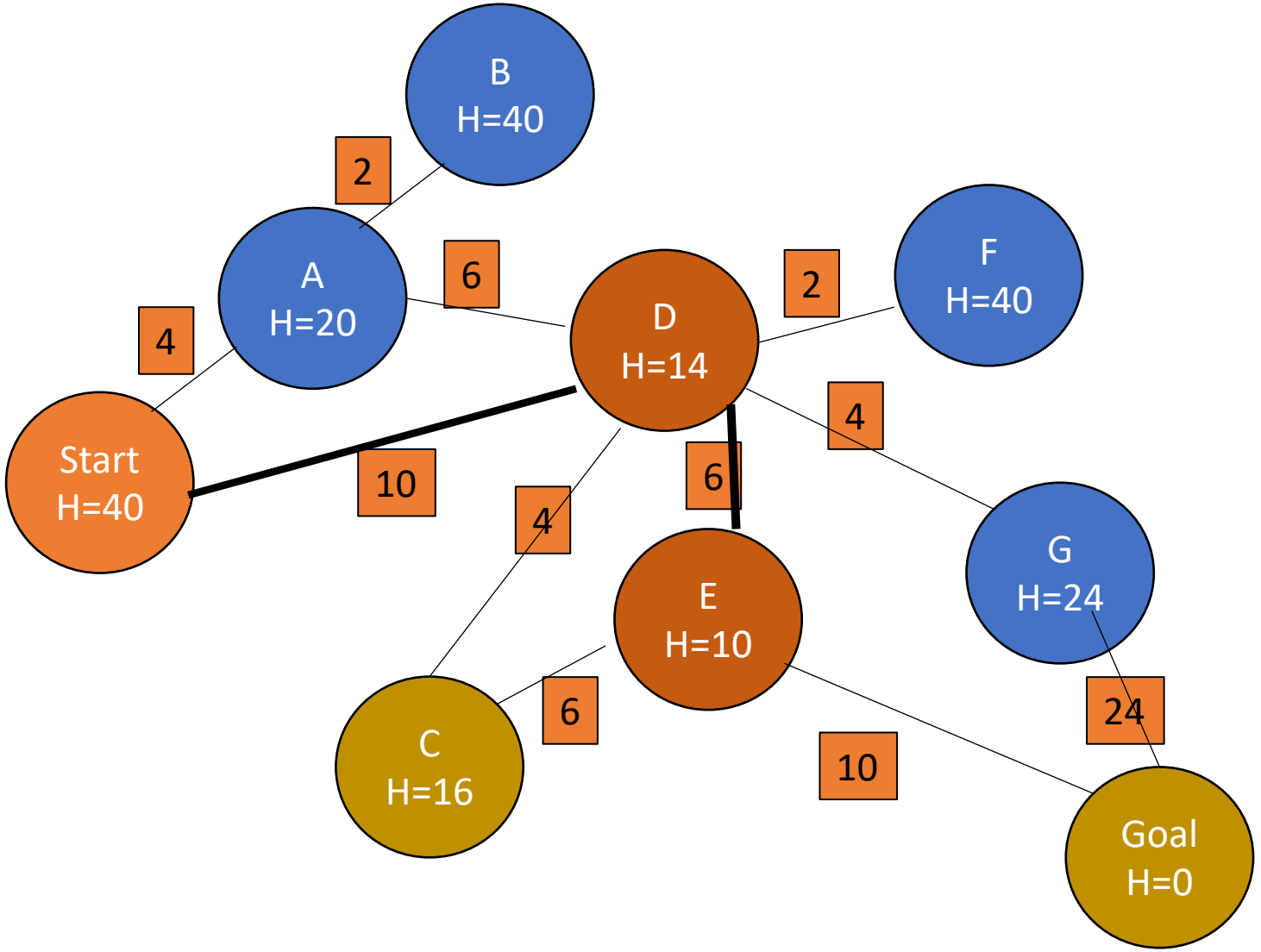


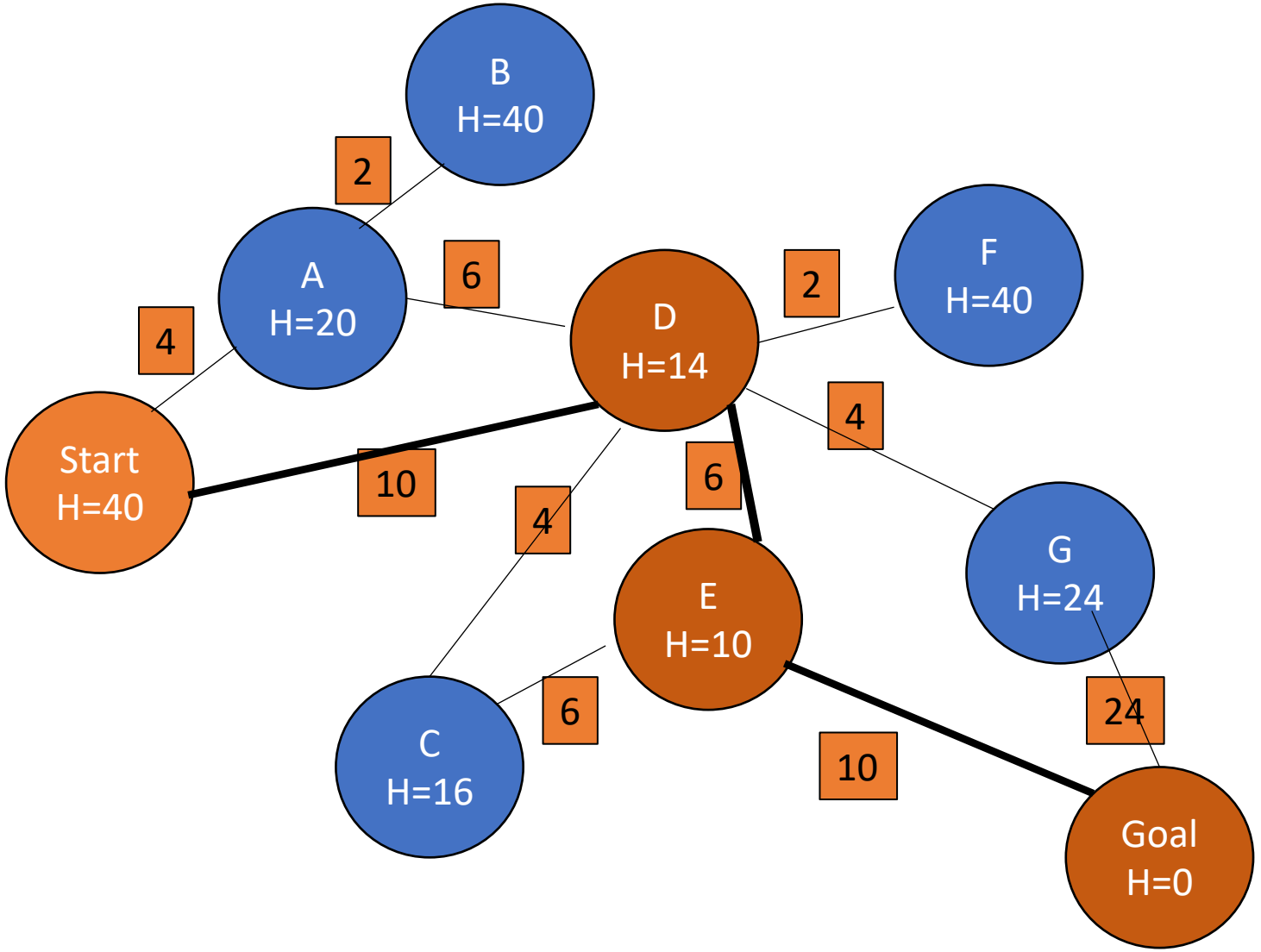

B
H=40
2
A
H=20
6
D
H=14
2
F
H=40
4
Start
H=40
10
4
6
4
G
H=24
E
H=10
C
H=16
6
10
24
Goal
H=0


B
H=40
2
A
H=20
6
D
H=14
2
F
H=40
4
Start
H=40
10
4
6
4
G
H=24
E
H=10
C
H=16
6
24
10
Goal
H=0


B
H=40
2
A
H=20
6
D
H=14
2
F
H=40
4
Start
H=40
10
4
6
4
G
H=24
E
H=10
C
H=16
6
24
10
Goal
H=0

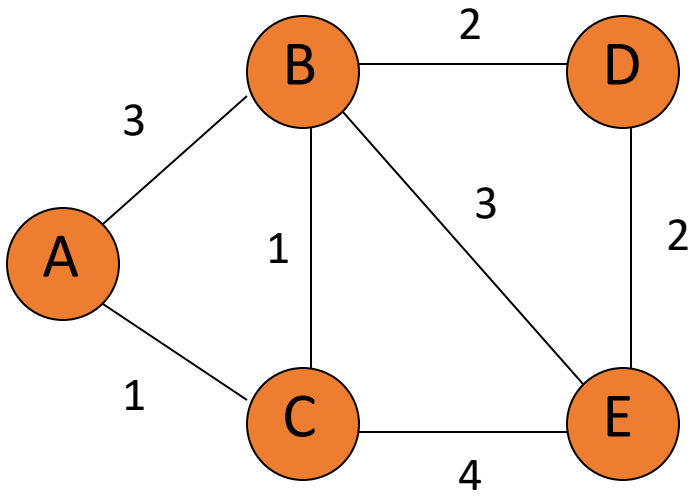

B
H=40
2
A
H=20
6
D
H=14
2
F
H=40
4
Start
H=40
10
4
6
4
G
H=24
E
H=10
C
H=16
6
24
10
Goal
H=0


B
H=40
2
A
H=20
6
D
H=14
2
F
H=40
4
Start
H=40
10
4
6
4
G
H=24
E
H=10
C
H=16
6
10
24
Goal
H=0


B
H=40
2
A
H=20
6
D
H=14
2
F
H=40
4
4
Start
H=40
10
6
4
G
H=24
E
H=10
C
H=16
6
24
10
Goal
H=0

# Shortest paths

- We can use BFS

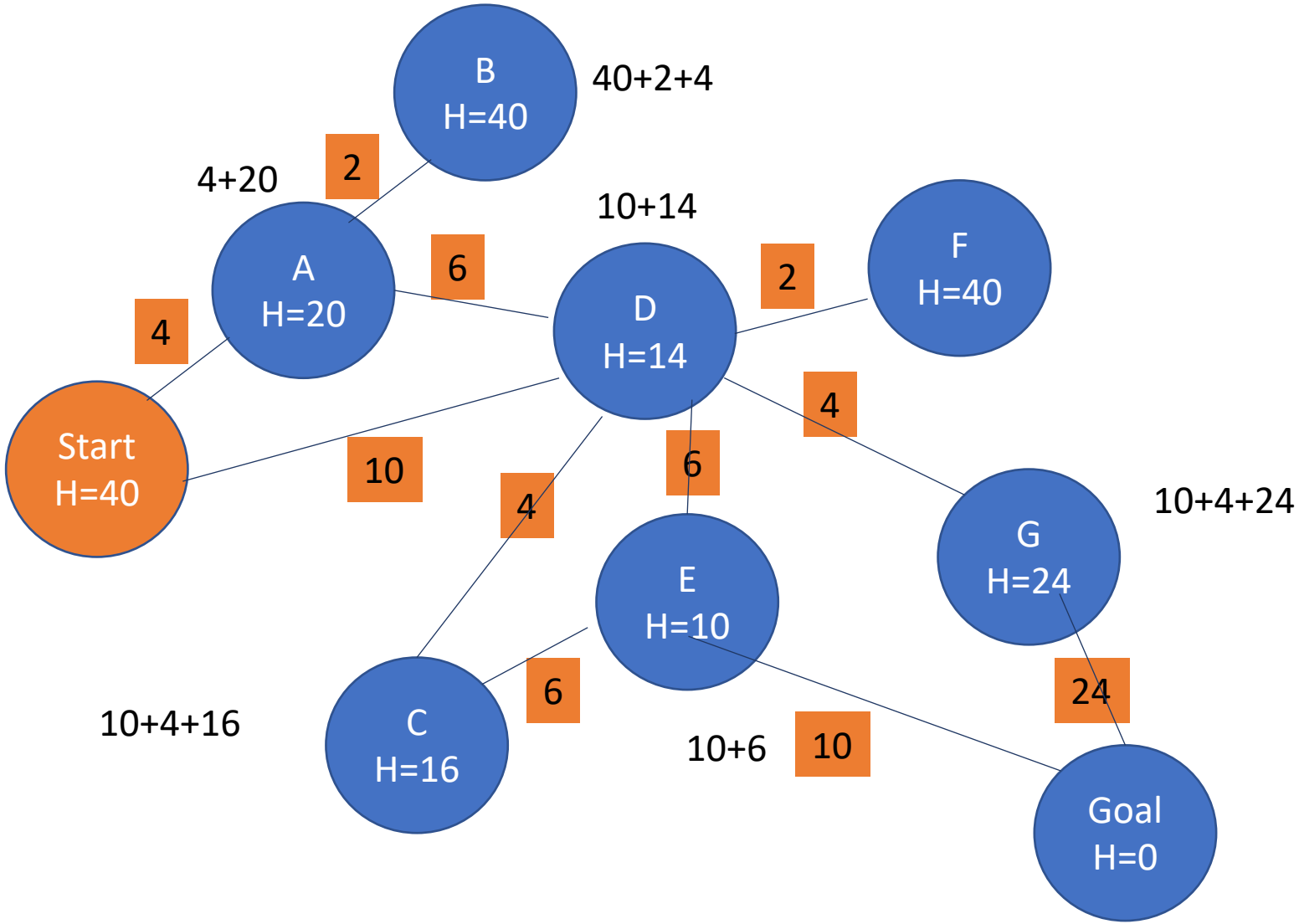


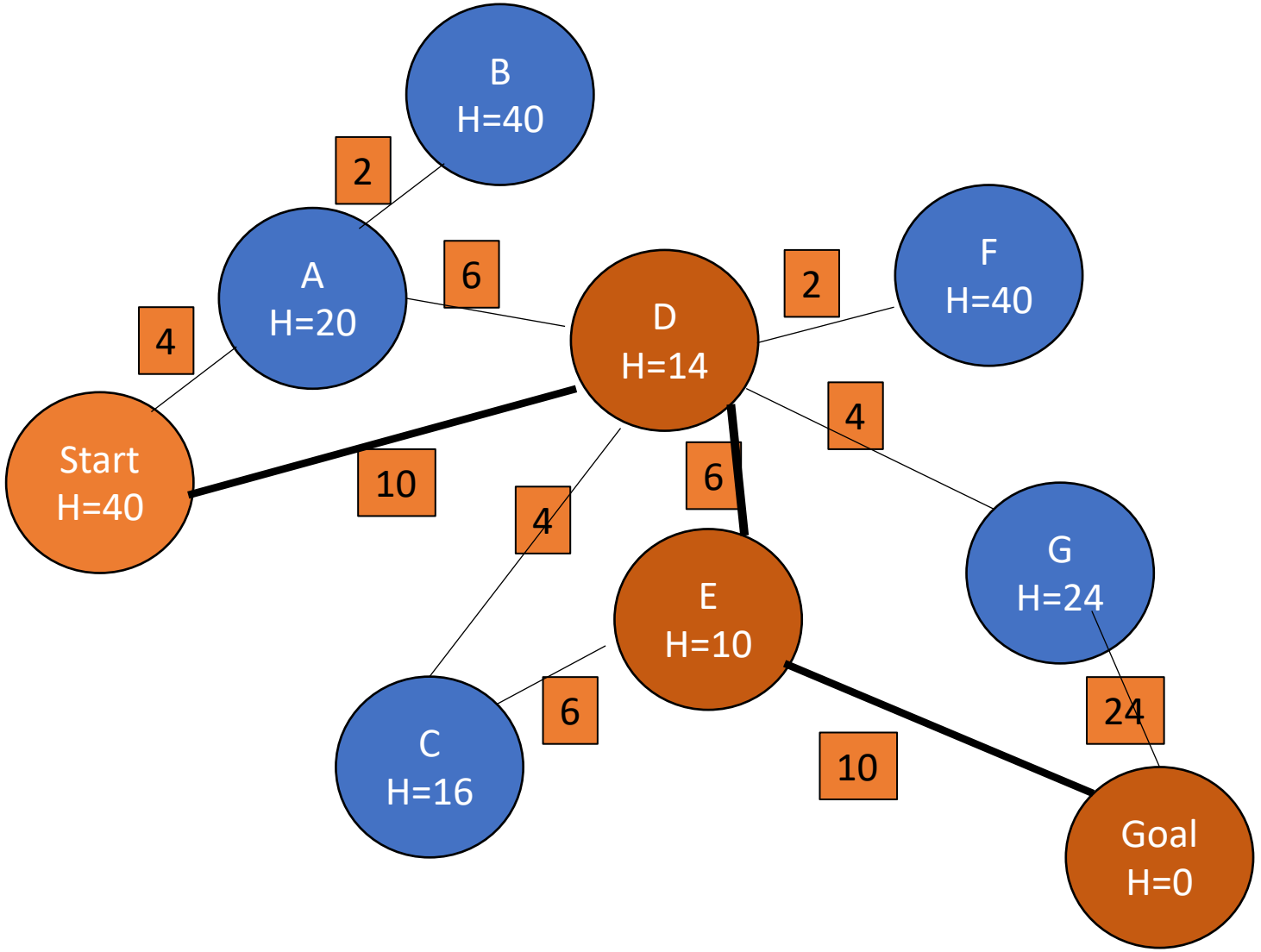
# A* SEARCH

- Idea: avoid expanding paths that are already expensive – combine cost to reach the node and cost estimated to get from the node to goal
- Evaluation function $f(n) = g(n) + h(n)$
- $g(n)$ = cost so far to reach $n$
- $h(n)$ = estimated cost from $n$ to goal
- $f(n)$ = estimated total cost of path through $n$ to goal



# How Can we Apply A* here



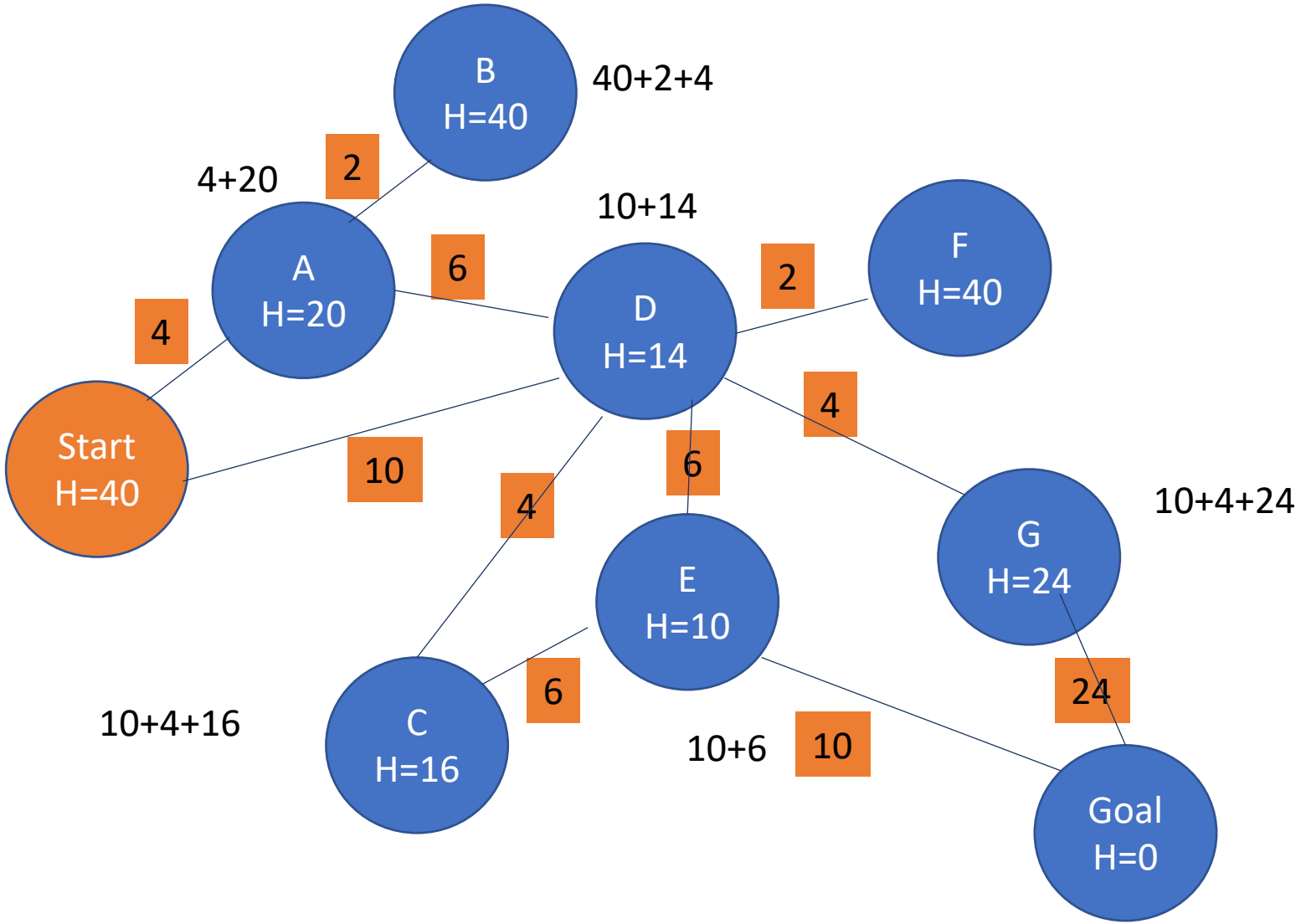

B
H=40
2
A
H=20
6
D
H=14
2
F
H=40
4
4
Start
H=40
10
6
4
E
H=10
G
H=24
C
H=16
6
24
10
Goal
H=0

# A* SEARCH and IDA*

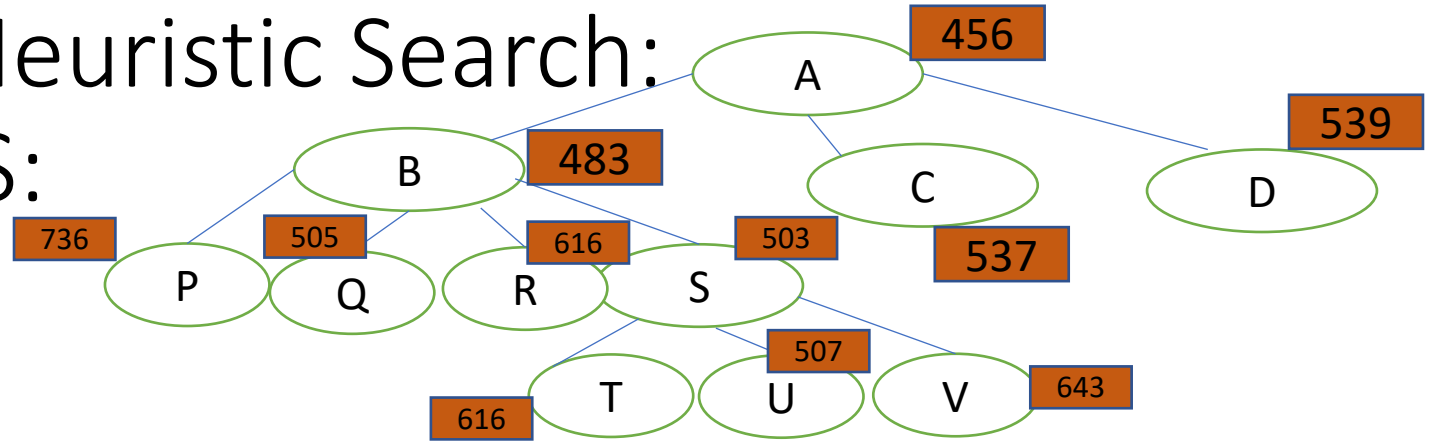
- A*: Idea: avoid expanding paths that are already expensive – combine cost to reach the node and cost estimated to get from the node to goal
  - Evaluation function *f(n) = g(n) + h(n)*
  - *g(n)* = cost so far to reach *n*
  - *h(n)* = estimated cost from *n* to goal
  - *f(n)* = estimated total cost of path through *n* to goal
- Iterative deepening A* - IDA* - In standard iterative deepening cutoff used is depth at each iteration while in IDA* f – cost (g+h)



# How Can we Apply A* here



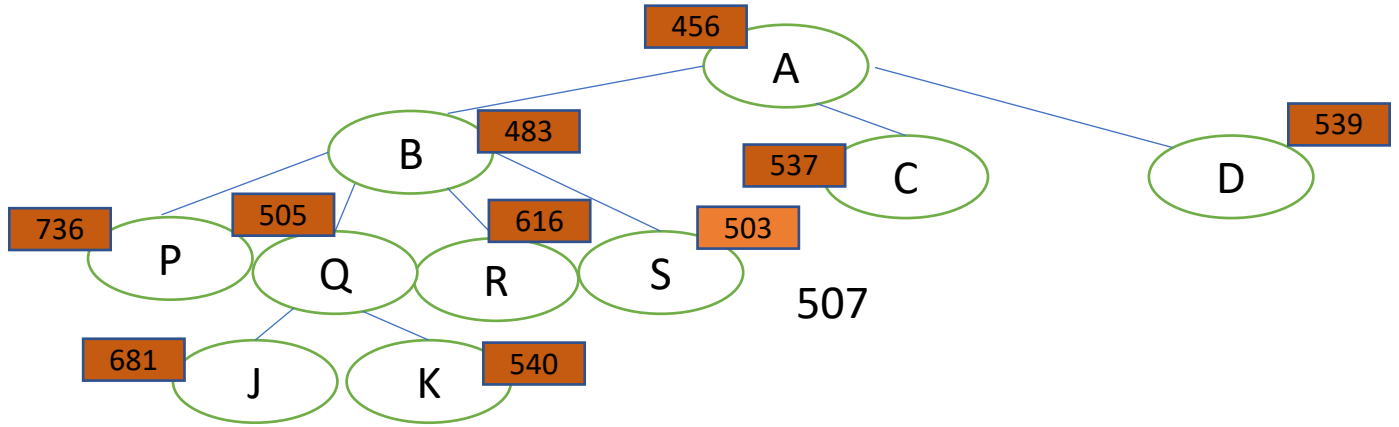
# Memory Bounded Heuristic Search: Recursive BFS - RBFS:

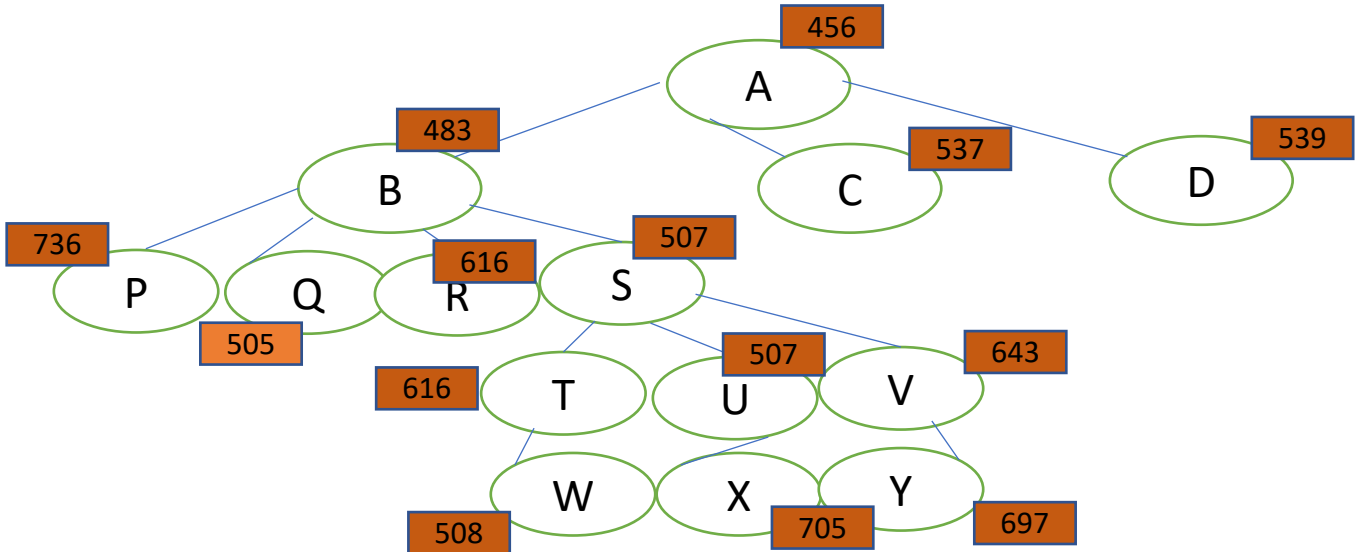



RBFS changes its mind very often in practice.

This is because the f=g+h become more accurate (less optimistic) as we approach the goal. Hence, higher level nodes have smaller f-values and will be explored first.

Problem: We should keep in memory whatever we can.



# Simple Memory Bounded A*

- This is like A*, but when memory is full, we delete the worst node (largest f-value).
- Like RBFS, we remember the best descendent in the branch we delete.
- If there is a tie (equal f-values) we first delete the oldest nodes first.
- simple-MBA* finds the optimal *reachable* solution given the memory constraint.
- Time can still be exponential.



# Heuristic Function

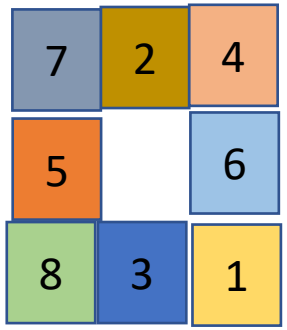
- The exhaustive search might be expensive, it might be brought under control by getting rid of repetitive states.
- Even that might be impractical in absence of good heuristic function that never over estimates
- Why?
- How one should define heuristic function?



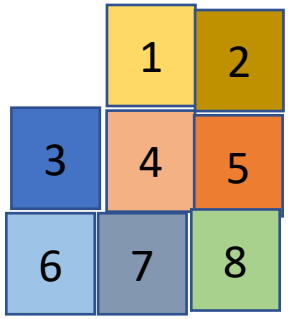
# ADMISSIBLE HEURISTICS

- *H1(n)* = number of misplaced tiles
- *H2(n)* = total Manhattan distance (city block distance - )

(i.e., no. of squares from desired location of each tile)




Start




Goal

- H1(s) = 8
- H2(s)3+1+2+2+2+3+3+2 = 18

# Some Hill-Climbing Algo's

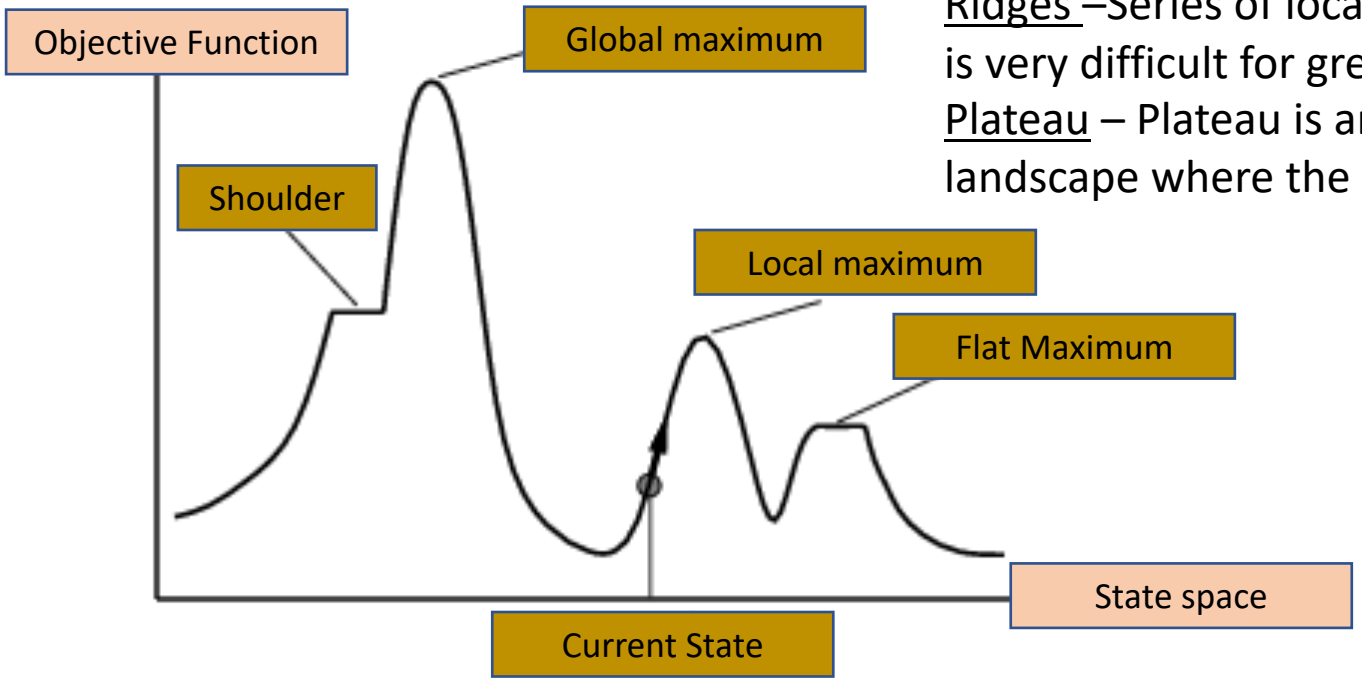
- Start = random state or special state.
- Until (no improvement)
  - Steepest Ascent: find best successor
  - OR (greedy): select first improving successor
  - Go to that successor
- Repeat the above process some number of times (Restarts).
- Can be done with partial solutions or full solutions.

# Beam

- Mix of hill-climbing and best first
- Storage is a cache of best K states
- Solves storage problem, but…
- Not optimal, not complete

# Local search: hill climbing

- In addition to finding goals local search algorithms are useful for solving pure optimization problems i.e. finding the best state according to objective function
- State space landscape:




**Issues**:
Local Maxima
Ridges –Series of local maxima that is very difficult for greedy algorithm to navigate
Plateau – Plateau is an area where of the state space landscape where the evaluation function is flat

# Hill climbing variants

- Stochastic Hill Climbing: Chooses at random from among the uphill moves (converges bit slowly but in some state landscapes can lead to better solutions)
- First choice hill climbing generates successors randomly until one is generated that is better than current state
- Random restart hill climbing: Conduct a series of hill climbing searches from randomly generated initial states – stop when goal is found
- Simulated Annealing: A hill climbing algorithm is incomplete as it can get stuck at local maximum- while purely random walk is complete but extremely inefficient. Combining hill climbing and random walk in some way can yield both efficiency and completeness



# Surprise Test (30 Min)

- Model Course Selection using heuristic search algorithm
  - Discuss local maxima and global maxima in your case
- Suggest interesting modification in hill climbing search algorithm that could lead to improved or interesting outcomes
- Think of embedding humanized aspects in hill-climbing search algo

# APPLICATIONS

- Open systems: dynamically changing systems
- Complex systems: Abstraction
- Process control
- Manufacturing
- Traffic control
- Information management
- E-commerce
- Business process management
- Medical domain – monitoring, etc
- Games

# Lecture Thirteen/Fourteen

## Demystifying Creativity from evolution perspective:

Heuristic functions and searches
Associative and Evolutionary aspects
Evolution
Evolutionary Algorithm (Genetic Algo)

Parag Kulkarni, PhD, DSc

# Pointers with Objectives

CONCEPTS

- What is AI?
- Human Learning,
- Creativity
- Thinking
- Conceptual Spaces
- Experience and Examples
- Predictions
- Intelligence
- Learning models and Paradigms
- Building blocks of Intelligent systems
- Algorithms
- Applications
- Case-studies

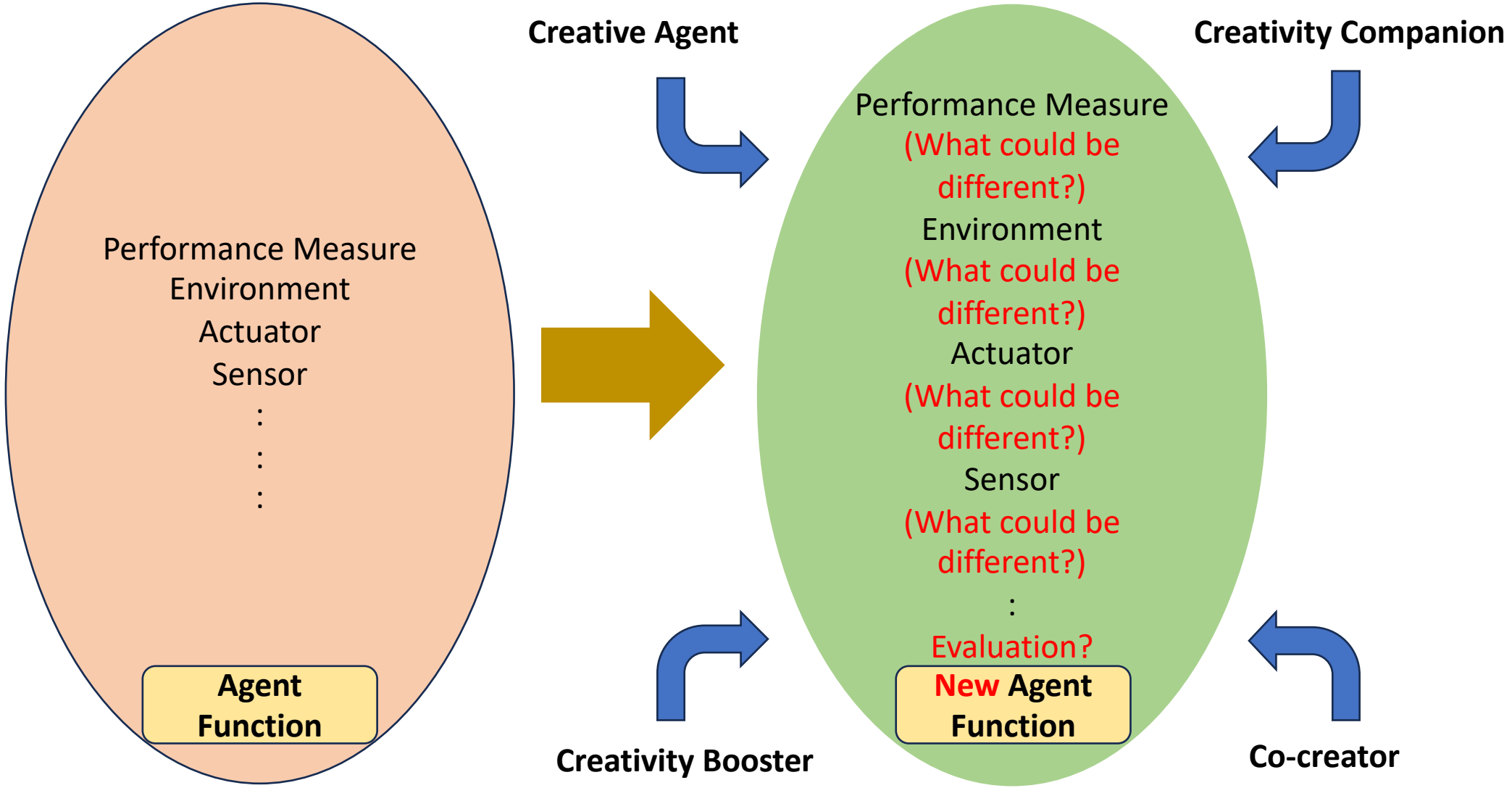
PRODUCTS and SOLUTIONS

APPROACHES

- Statistical
- Associative
- Behaviorism
- Evaluation
- Evolution
- Exploration
- Genetic
- Symmetry
- Analogy
- Combination
- Time-Series Predictive
- Immune System "Group Behavior"
- Discriminative
- Generative

# Intelligent Agent

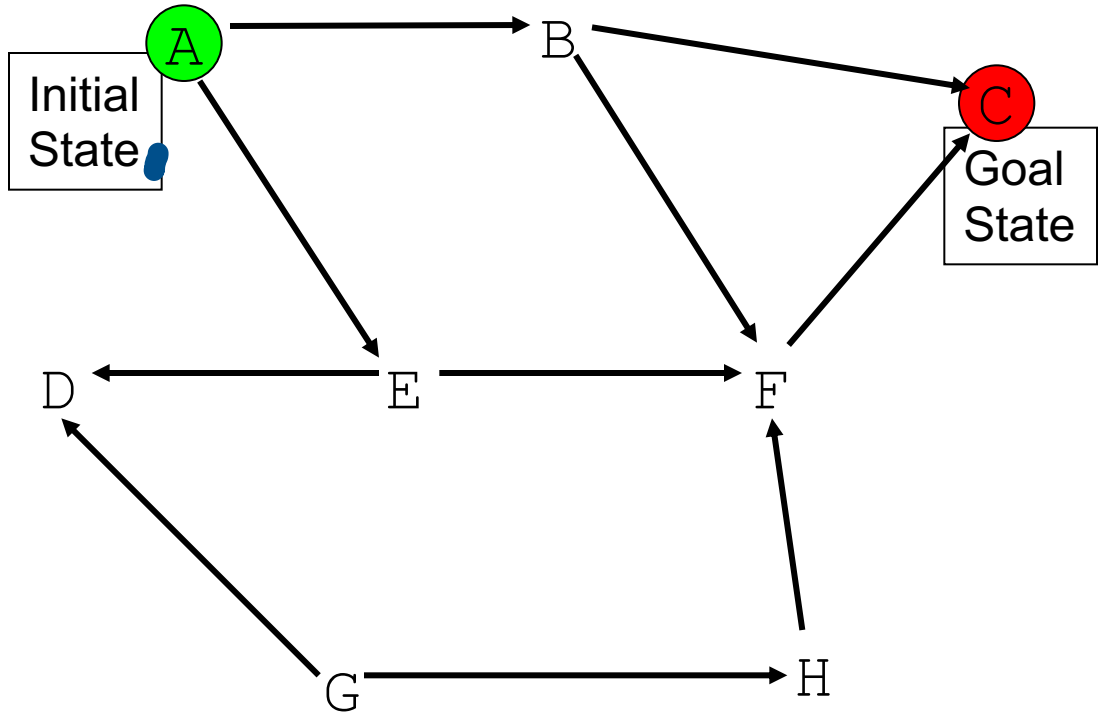
# Creative Agent



# AN EXAMPLE PROBLEM: SEARCHING A GRAPH

```
link(g,h).
link(g,d).
link(e,d).
link(h,f).
link(e,f).
link(a,e).
link(a,b).
link(b,f).
link(b,c).
link(f,c).
```

State-Space




```
go(X,X,[X]).
go(X,Y,[X|T]):-
        link(X,Z),
        go(Z,Y,T).
```

Simple search algorithm

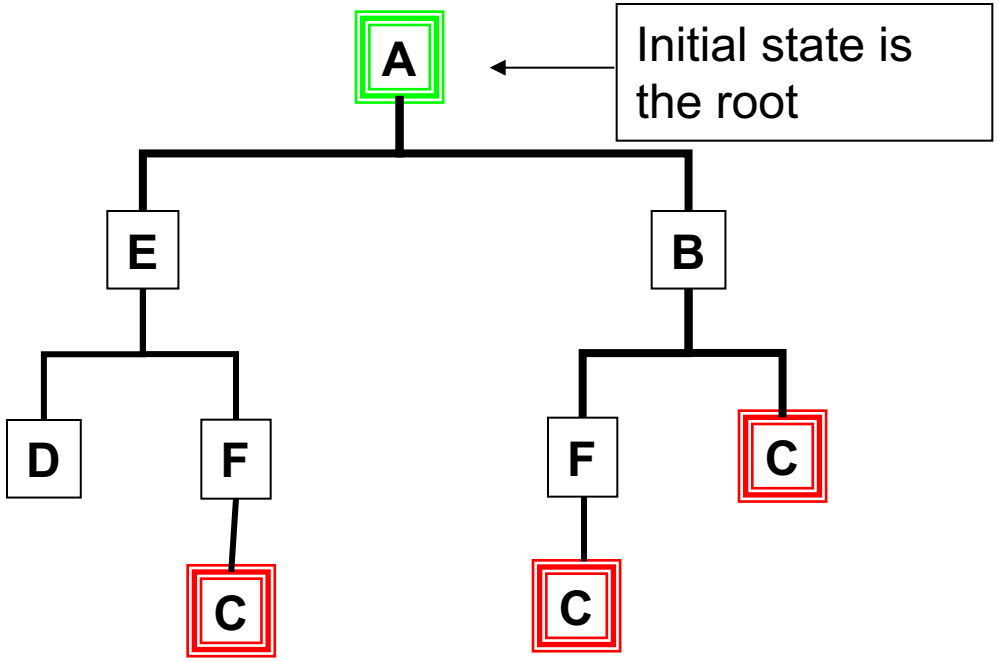
```
| ?- go(a,c,X).
X = [a,e,f,c] ? ;
X = [a,b,f,c] ? ;
X = [a,b,c] ? ;
no
```

Consultation

By Parag Kulkarni

# STATE-SPACE REPRESENTATION

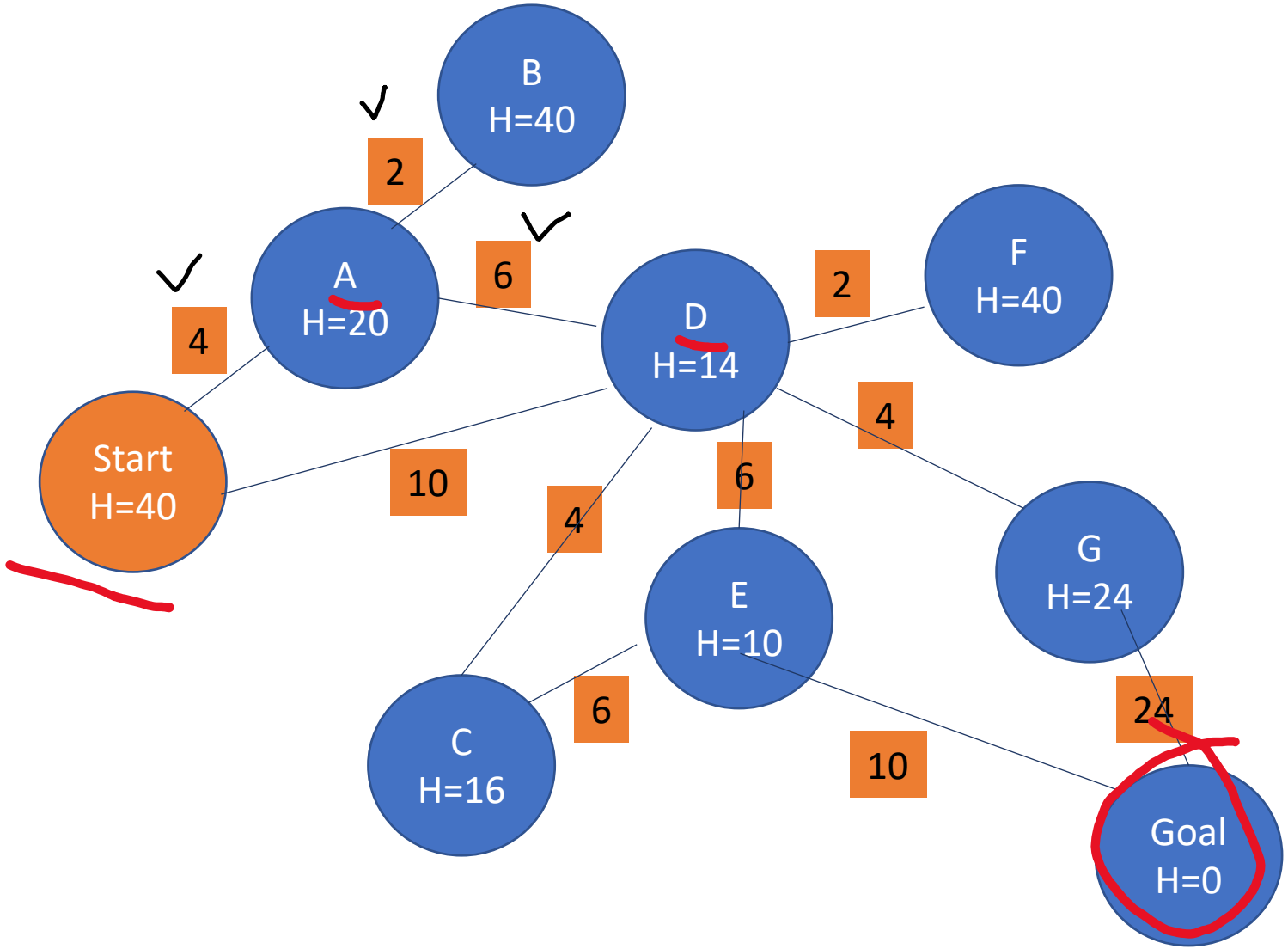
```
link(g,h).
link(g,d).
link(e,d).
link(h,f).
link(e,f).
link(a,e).
link(a,b).
link(b,f).
link(b,c).
link(f,c).
```

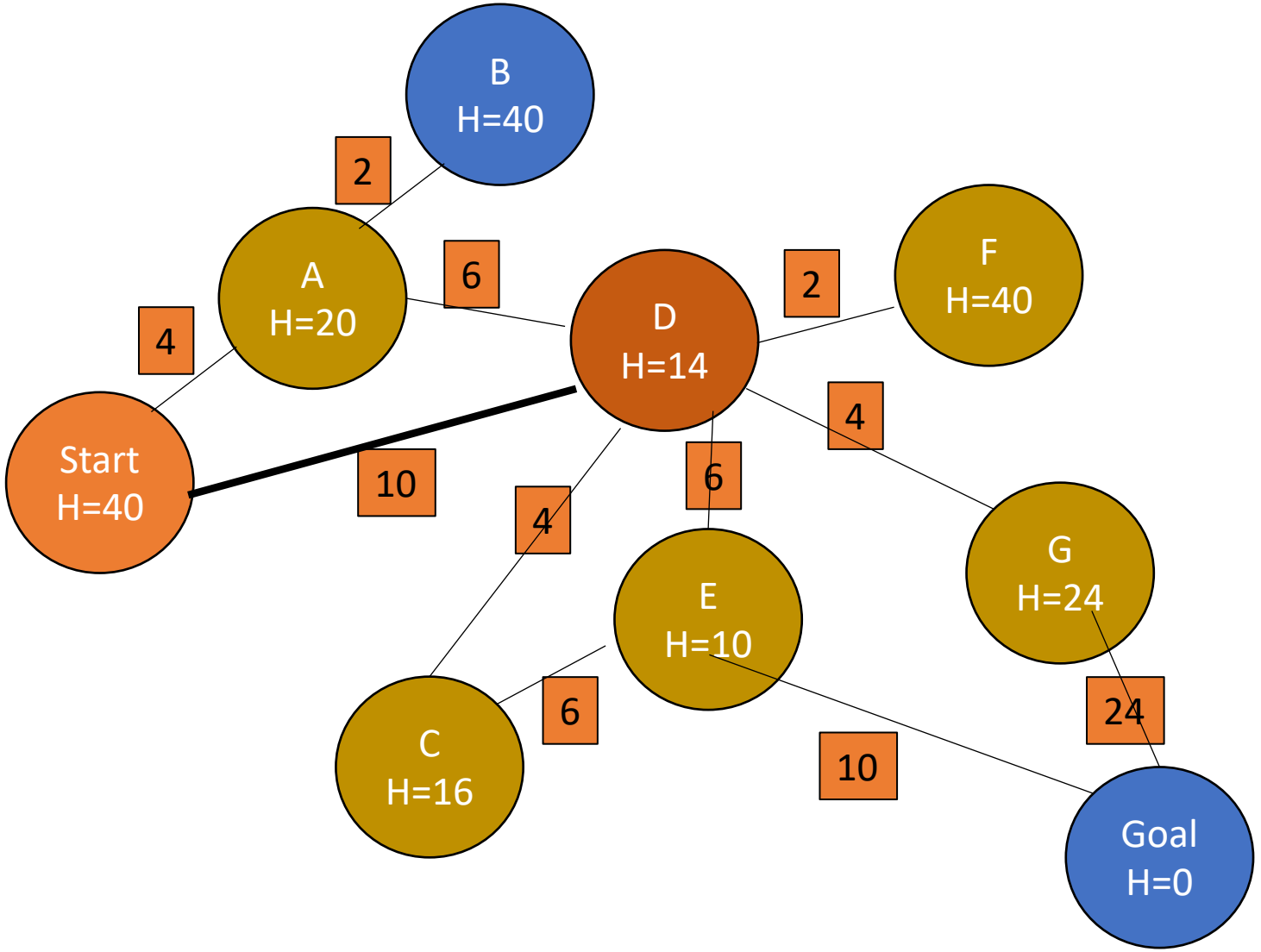


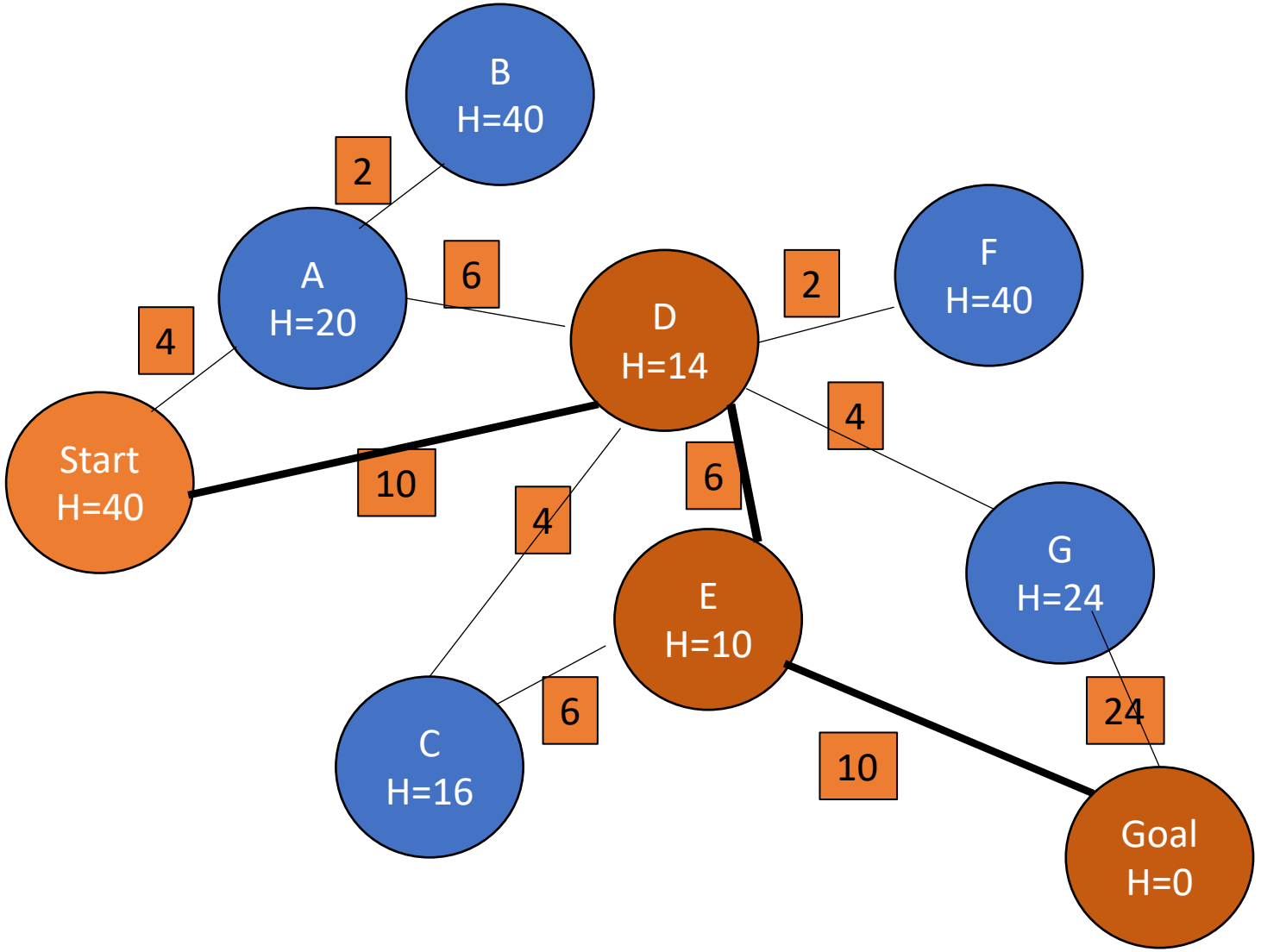

- An abstract representation of a state-space is a downwards growing tree. Connected *nodes* represent states in the domain.
- The *branching factor* denotes how many new states you can move to from any state. This problem has an average of 2.
- The *depth* of a node denotes how many moves away from the initial state it is. 'C' has two depths, 2 or 3.

By Parag Kulkarni

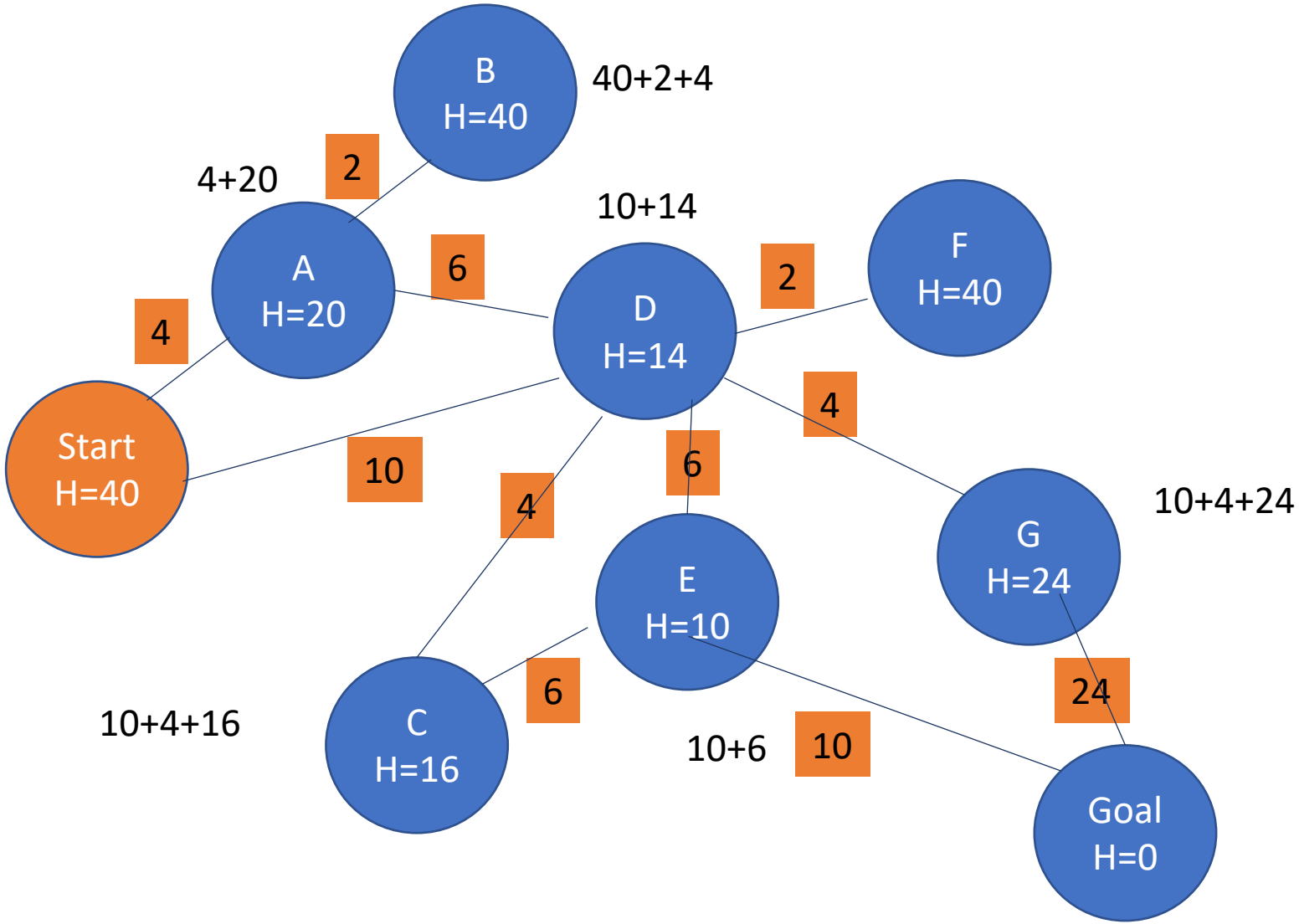
# Greedy Best First Search

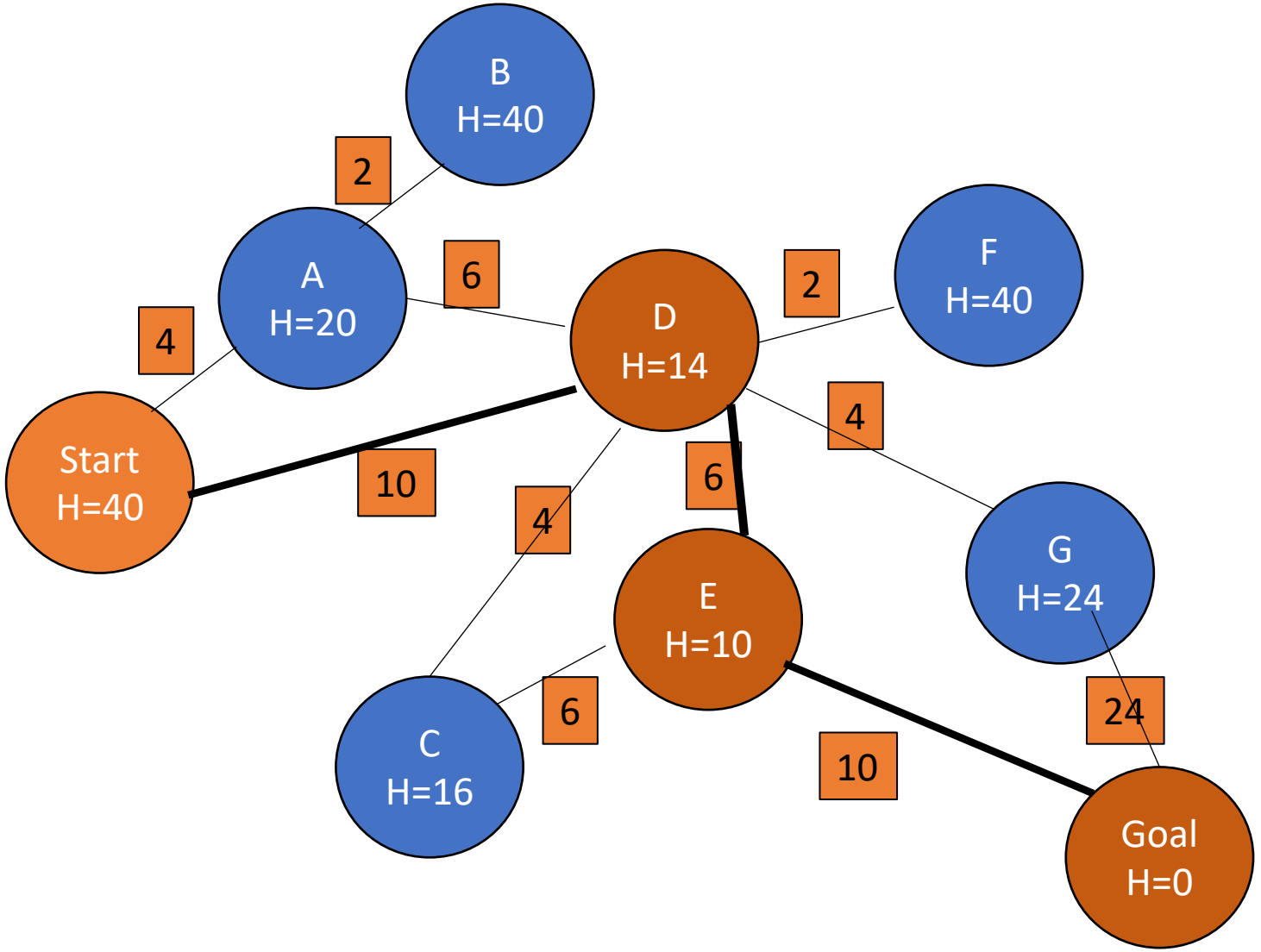



B
H=40
2
A
H=20
6
D
H=14
2
F
H=40
4
Start
H=40
10
4
6
4
G
H=24
E
H=10
C
H=16
6
24
10
Goal
H=0


B
H=40
2
A
H=20
6
D
H=14
2
F
H=40
4
4
Start
H=40
10
6
4
E
H=10
G
H=24
C
H=16
6
24
10
Goal
H=0

# How Can we Apply A* here




B
H=40
2
A
H=20
6
D
H=14
2
F
H=40
4
4
Start
H=40
10
6
4
G
H=24
E
H=10
C
H=16
6
24
10
Goal
H=0

# A* SEARCH and IDA*

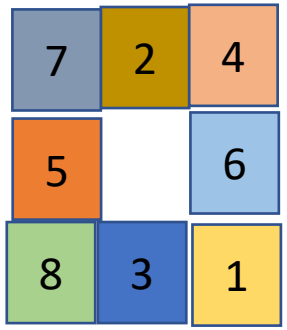
- A*: Idea: avoid expanding paths that are already expensive – combine cost to reach the node and cost estimated to get from the node to goal
  - Evaluation function *f(n) = g(n) + h(n)*
  - *g(n)* = cost so far to reach *n*
  - *h(n)* = estimated cost from *n* to goal
  - *f(n)* = estimated total cost of path through *n* to goal
- Iterative deepening A* - IDA* - In standard iterative deepening cutoff used is depth at each iteration while in IDA* f – cost (g+h)



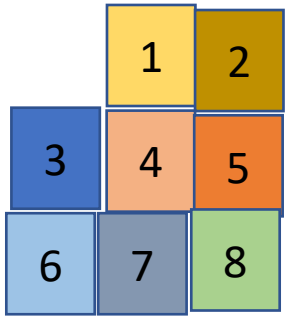
# ADMISSIBLE HEURISTICS

- *H1(n)* = number of misplaced tiles
- *H2(n)* = total Manhattan distance (city block distance - )

(i.e., no. of squares from desired location of each tile)




Start

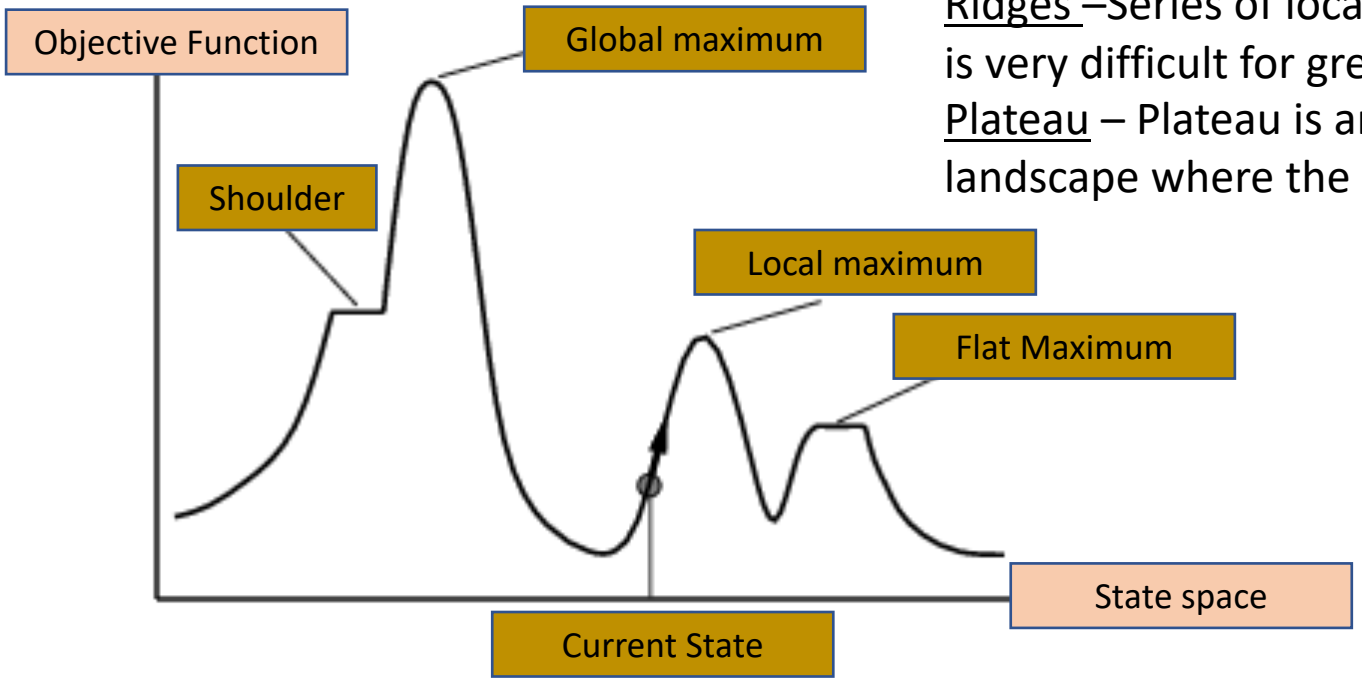



Goal

- H1(s) = 8
- H2(s)3+1+2+2+2+3+3+2 = 18

# Local search: hill climbing

- In addition to finding goals local search algorithms are useful for solving pure optimization problems i.e. finding the best state according to objective function
- State space landscape:




**Issues**:
Local Maxima
Ridges –Series of local maxima that is very difficult for greedy algorithm to navigate
Plateau – Plateau is an area where of the state space landscape where the evaluation function is flat

# Hill climbing variants

- Stochastic Hill Climbing: Chooses at random from among the uphill moves (converges bit slowly but in some state landscapes can lead to better solutions)
- First choice hill climbing generates successors randomly until one is generated that is better than current state
- Random restart hill climbing: Conduct a series of hill climbing searches from randomly generated initial states – stop when goal is found
- Simulated Annealing: A hill climbing algorithm is incomplete as it can get stuck at local maximum- while purely random walk is complete but extremely inefficient. Combining hill climbing and random walk in some way can yield both efficiency and completeness



# Local Search Algorithms

- In many optimization problems, the path to the goal is irrelevant; the goal state itself is the solution – we are worried about the goal state only
- State space = set of "complete" configurations
- Find configuration satisfying constraints, e.g., n-queens
- In such cases, we can use local search algorithms
- keep a single "current" state (rather than multiple paths), try to improve it.
- Very memory efficient (only remember current state)
- Can find solutions in large or infinite state space



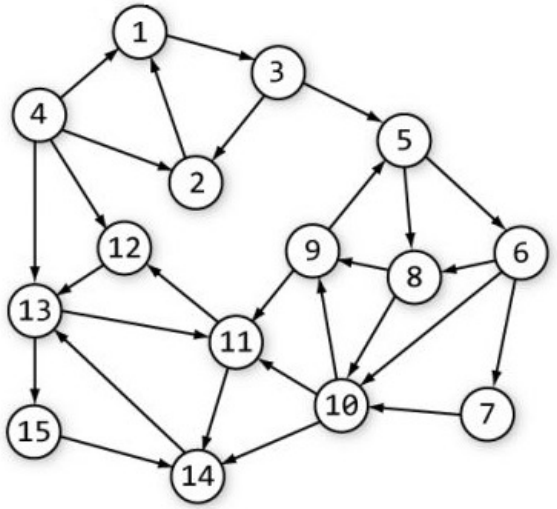
# Local search

- Moves between configurations by performing local moves
- Works with complete assignments of the variables
- Optimization problems:
  - Start from a suboptimal configuration
  - Move towards better solutions
- Satisfaction problems:
  - Start from an infeasible configuration
  - Move towards feasibility
- No guarantees
- Can work great in practice!



# Neighborhood

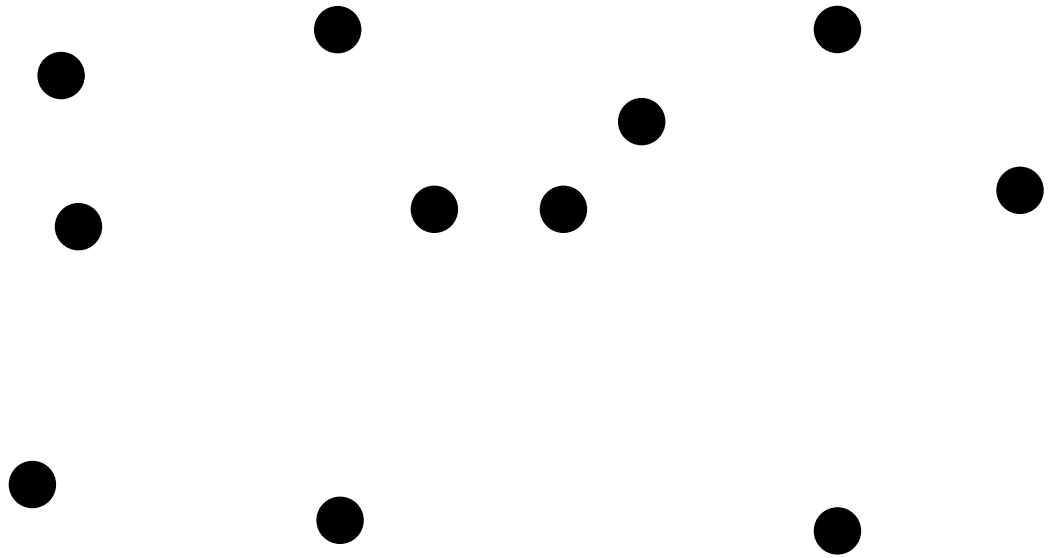
- Optimizing a function f
- Local moves define a neighborhood : Local moves are rational moves?
  - Configurations that are close
  - N: C -> $2^C$
- Local search is a graph exploration



# Travelling salesman problem (TSP)

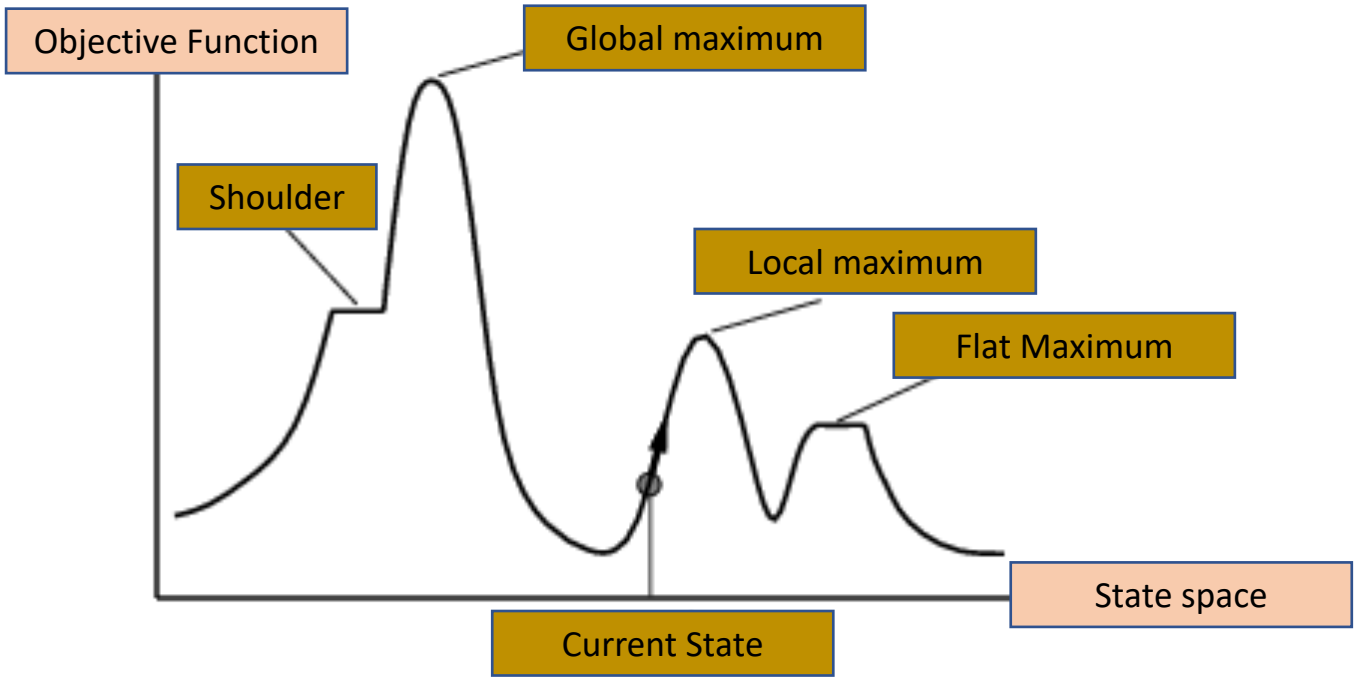
Task: find the shortest path to visit all cities exactly once

- Hamiltonian cycle in a graph
- simplification: cities are points in 2D

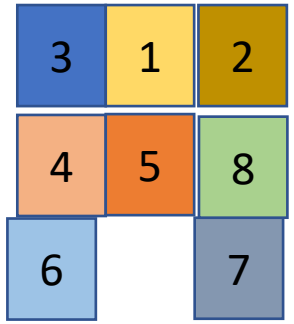


# Local search: hill climbing

- In addition to finding goals local search algorithms are useful for solving pure optimization problems i.e. finding the best state according to objective function
- State space landscape:

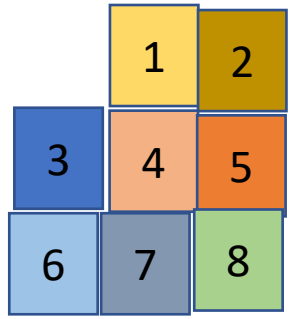


Start



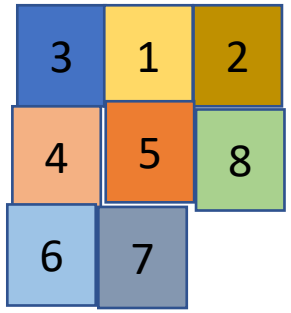

H = 5

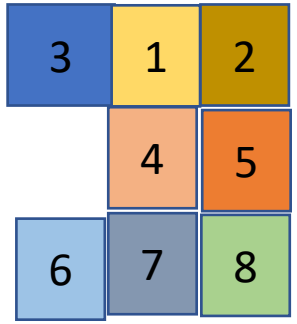
Goal




H = 0

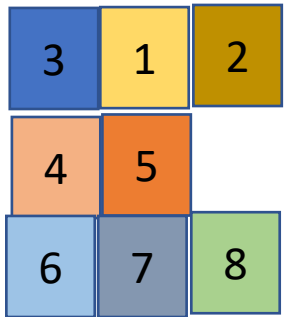
H = 4



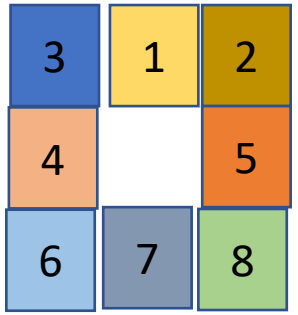




H = 1

H = 3




H = 2



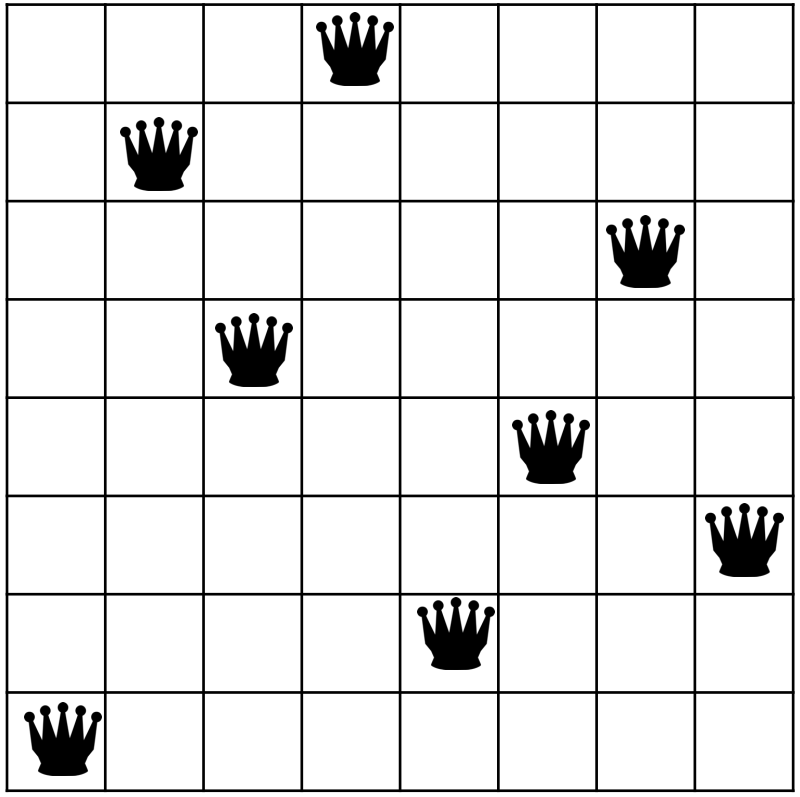

By Parag Kulkarni



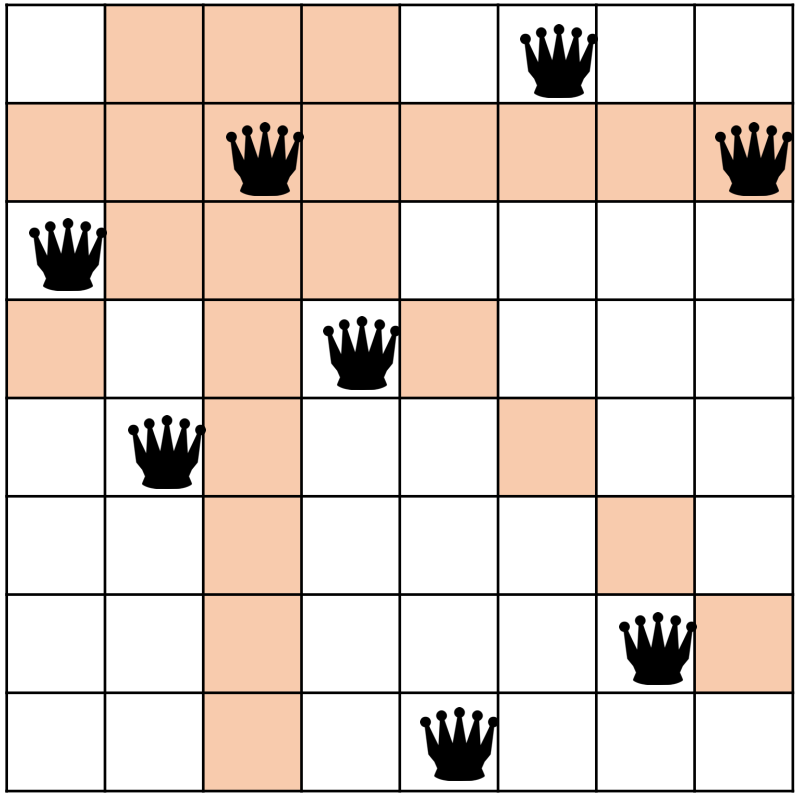
# Example: satisfaction problem

Task: place 8 queens on the chess board such that they do not attack each other

Goal



Position





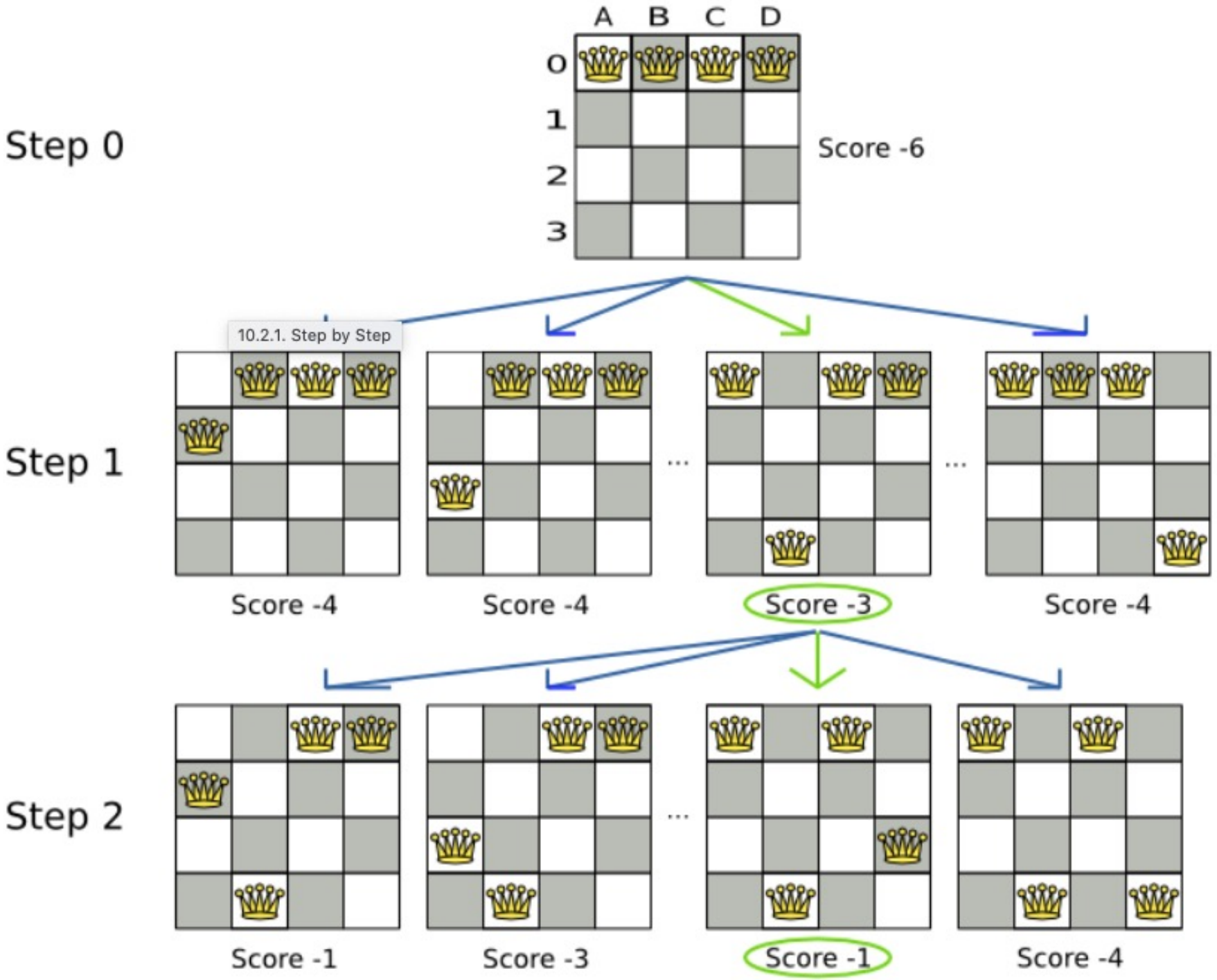
# Example: satisfaction problem

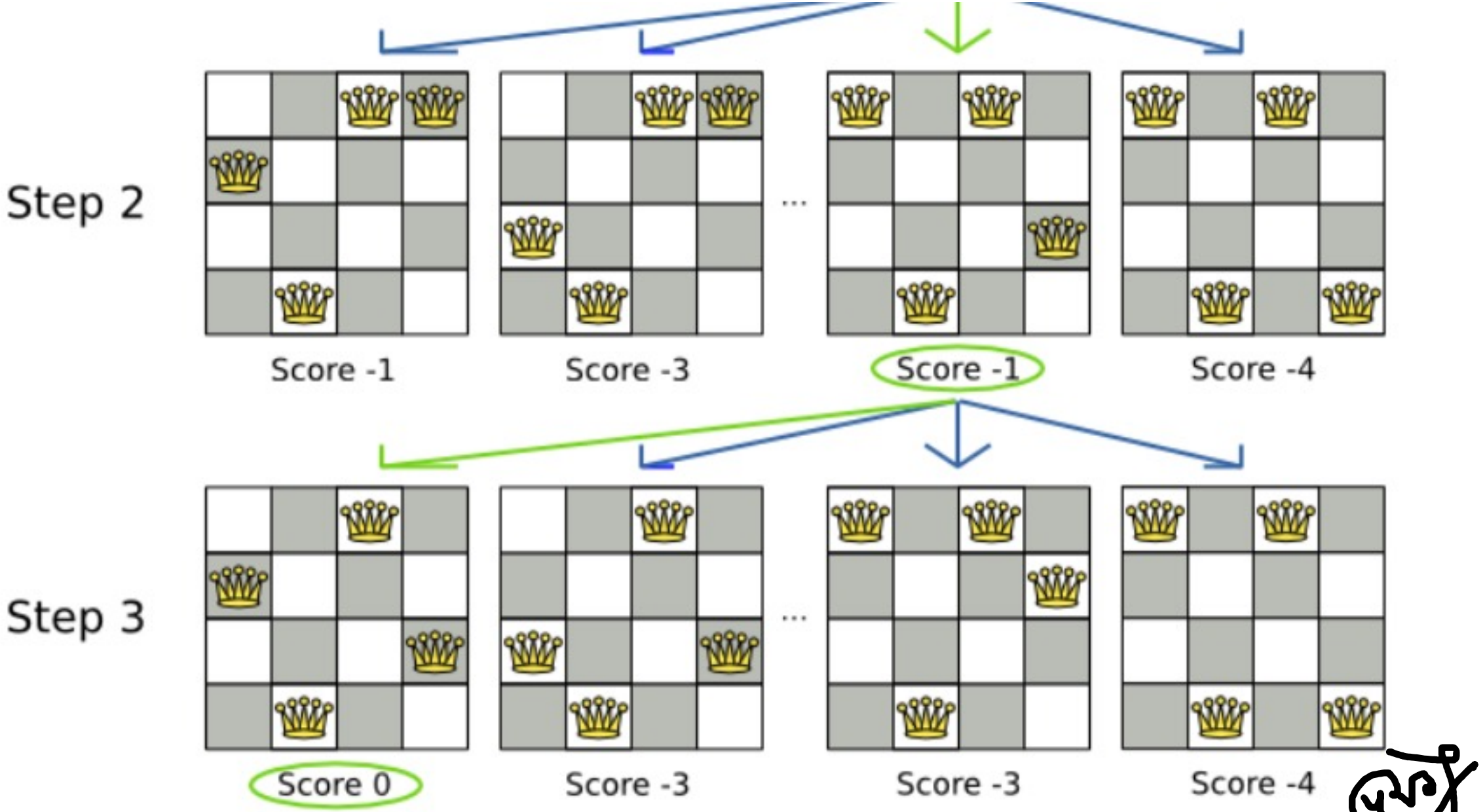
Task: place 4 or 8 queens on the chess board (or 4X4 board) such that they do not attack each other

Representation: the row of the queen in each column

Local steps: move one of the queens in its column




Step 0
A B C D
0
1
2
3
Score -6
10.2.1. Step by Step
Step 1
Score -4
Score -4
Score -3
Score -4
Step 2
Score -1
Score -3
Score -1
Score -4


Step 2
Score -1
Score -3
Score -1
Score -4
Step 3
Score 0
Score -3
Score -3
Score -4

# Surprise Test (30 Min)

- Model Product Recommendation using heuristic search algorithm
  - Define heuristic function
  - Define Start State
  - Define Goal State
  - Define actions and neighboring states
  - Traverse search path for a specific example using hill climbing search (Assume 10 Product set to work out example)
  - Discuss local maxima and global maxima in your case
- How you can handle multiple goal scenario in this case
- Provide a vector transition with correction using A* Algorithm
- Suggest interesting modification in hill climbing search algorithm that could lead to improved or interesting outcomes

# Example: satisfaction problem

Task: place 8 queens on the chess board such that they do not attack each other

Representation: the row of the queen in each column

Local steps: move one of the queens in its column

Local search:

- start with infeasible configuration
- move towards feasibility



# 8 queens

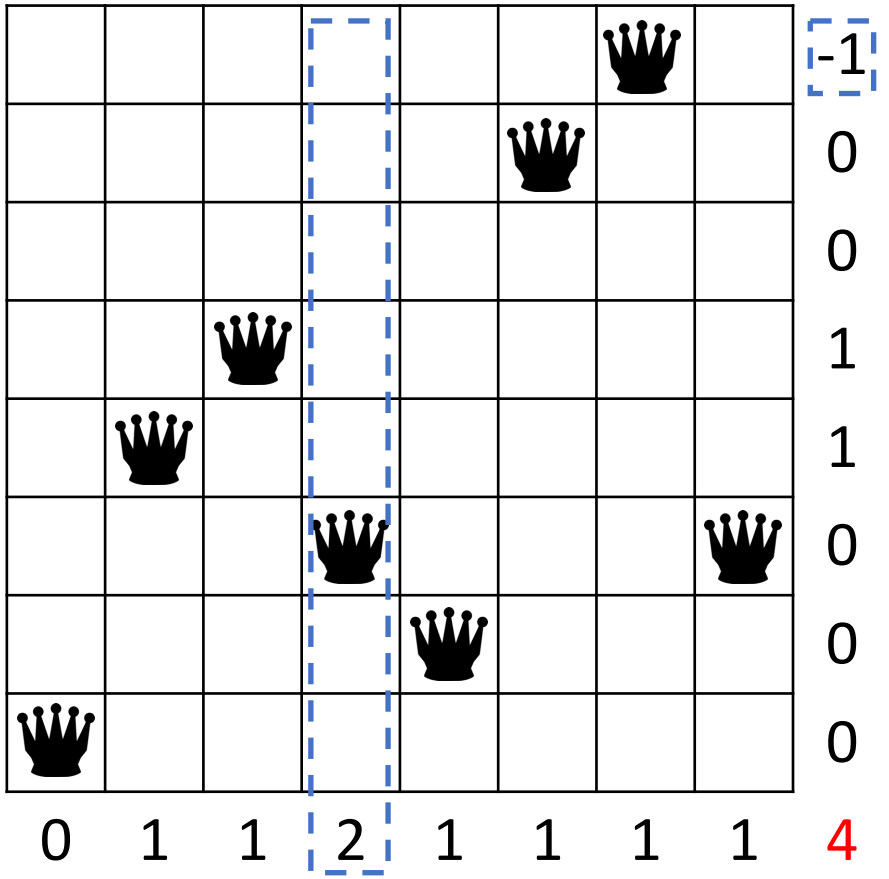
Task: place 8 queens on the chess board such that they do not attack each other

Count constraint violations

**Queen with most violations**

Possible moves

Find the best move



# Local search: no guarantees

- Local minima:
- no step improves the objective
- No guarantees for global optimality
- Escaping local optima is a critical issue in local search



# Hill Climbing Search

- Problem: depending on initial state, can get stuck in local maxima
- Continually moves in the direction of increasing value
- Terminates when reaches to peak
- Does not look beyond immediate neighbors
- Also called as greedy local search – makes rapid progress



# Hill climbing local maxima problem

- Local Maxima
- Ridges –Series of local maxima that is very difficult for greedy algorithm to navigate
- Plateau – Plateau is an area where of the state space landscape where the evaluation function is flat



# Hill climbing variants

- Stochastic Hill Climbing: Chooses at random from among the uphill moves (converges bit slowly but in some state landscapes can lead to better solutions)
- First choice hill climbing generates successors randomly until one is generated that is better than current state
- Random restart hill climbing: Conduct a series of hill climbing searches from randomly generated initial states – stop when goal is found



# Hill Climbing

- Success of Hill Climbing depends on shape of state-space landscape



# Simulated Annealing

- A hill climbing algorithm is incomplete as it can get stuck at local maximum- while purely random walk is complete but extremely inefficient
- Combining hill climbing and random walk in some way can yield both efficiency and completeness
- Simulated annealing is quite similar to annealing in metallurgy – The energy or motion of the particle is increased so that it will come out of the local minima – slowly this energy is reduced to get the best position to start hill climbing



# Local beam search

- Only one node in memory may pose limitations – local beam search keep track of k states rather than just one
- Local beam search is like running k random restart in parallel instead of in sequence
- In random search each process runs independently while in local beam search useful information is passed among the k parallel search threads

# Stochastic beam search – more than one successor

- Local beam search may quickly become concentrated in small region – instead of random successor – stochastic beam successor chooses k successors at random, with probability of choosing a given successor being an increasing function of its value (fitness).

# Tabu Search

- What is Tabu list?
- Maintaining list of already visited nodes.
- Next move can be

# Tabu search

- Key idea
- maintain the sequence of nodes already visited tabu lists and tabu nodes

- Select the best configurations that is not tabu,

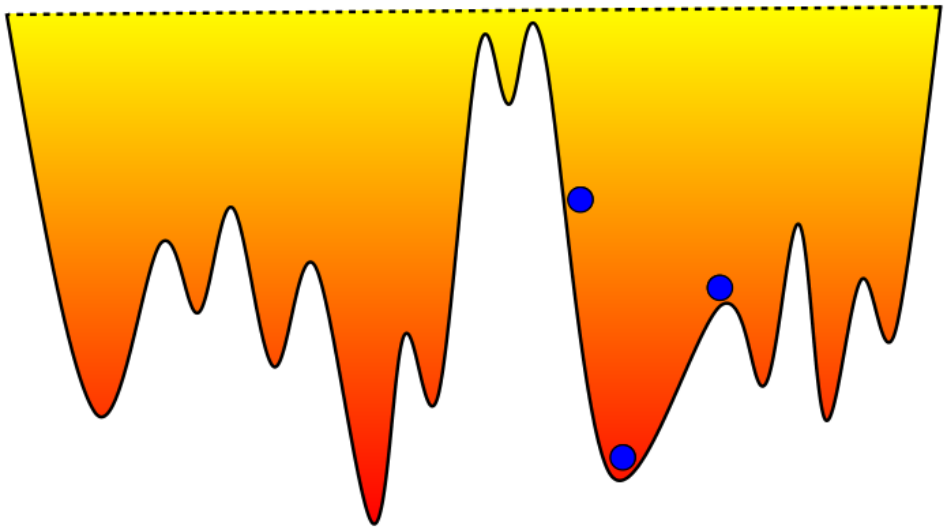
i.e., has not been visited before



# Tabu search

- Key idea
  - maintain the sequence of nodes already visited

    tabu lists and tabu nodes

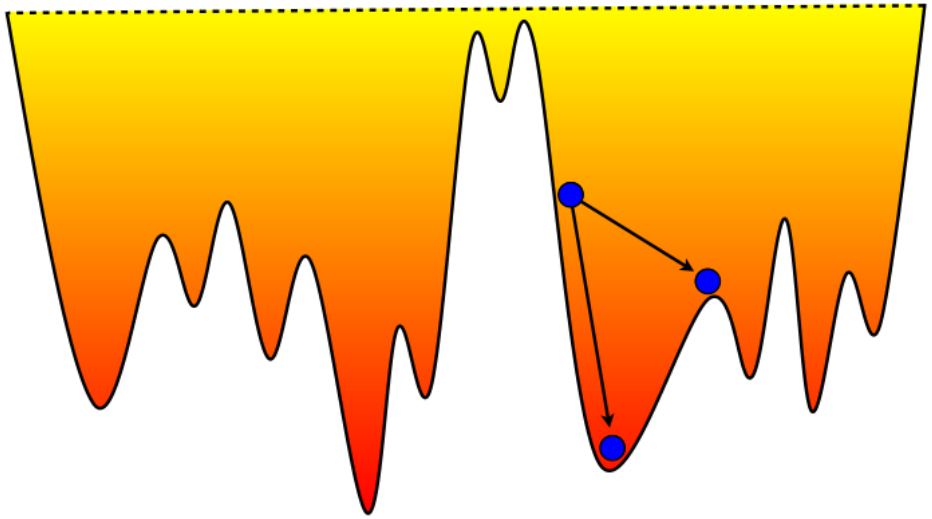


Tabu nodes ●: nodes already visited

# Tabu search

- Key idea
  - maintain the sequence of nodes already visited

    tabu lists and tabu nodes

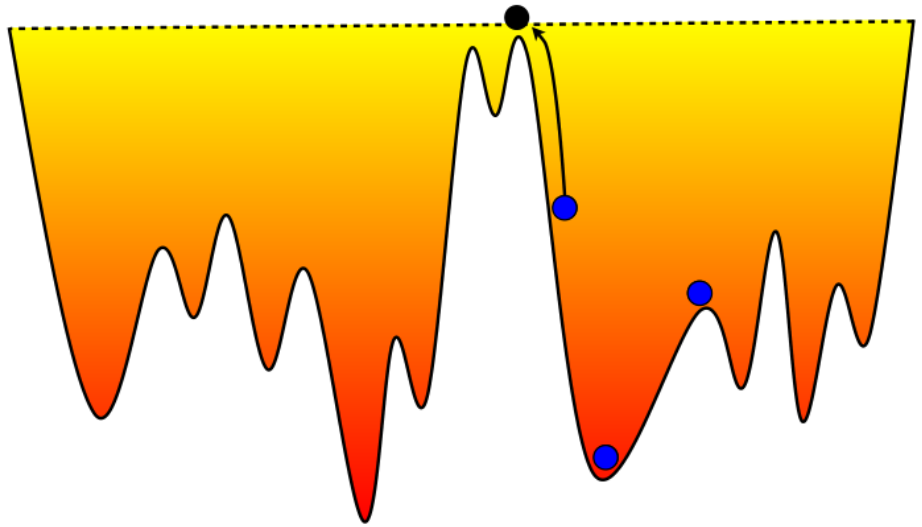


Tabu nodes ●: nodes already visited

# Tabu search

- Key idea
  - maintain the sequence of nodes already visited

    tabu lists and tabu nodes



Tabu nodes : nodes already visited

# Searching based problem solving

- Defining goal state

# Examples

- Team member selection
- Summarization
- Product recommendation



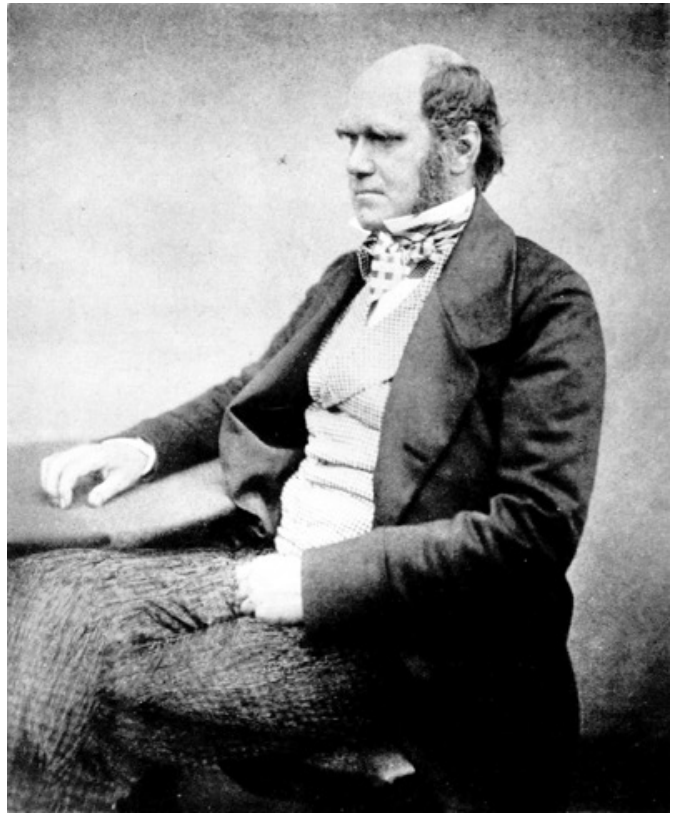
# Problem solving techniques

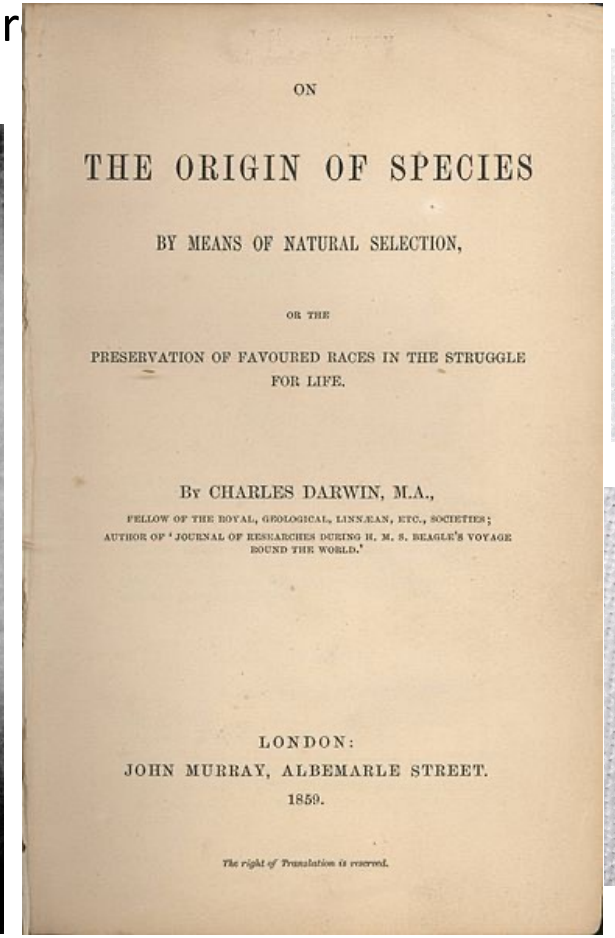
- Search space
- State evaluation (Function in practical applications)
- Greedy approaches
- Manhattan distance based searches
- Abstracting from real world details
- Avoiding repeated visits…



# Evolution/Survival of the Fittest

The theory of evolution is the statement that all species on Earth have arisen in this way by evolution from one or more very simple self-reproducing molecules in the primeval soup. I.e. we have evolved via the accumulation of countless advantageous (in context) mutations over countless generations, and species have diversified to occupy environmental niches, as a result of different environments favouring differ





ON

THE ORIGIN OF SPECIES

BY MEANS OF NATURAL SELECTION,

OR THE

PRESERVATION OF FAVOURED RACES IN THE STRUGGLE FOR LIFE.

By CHARLES DARWIN, M.A.,

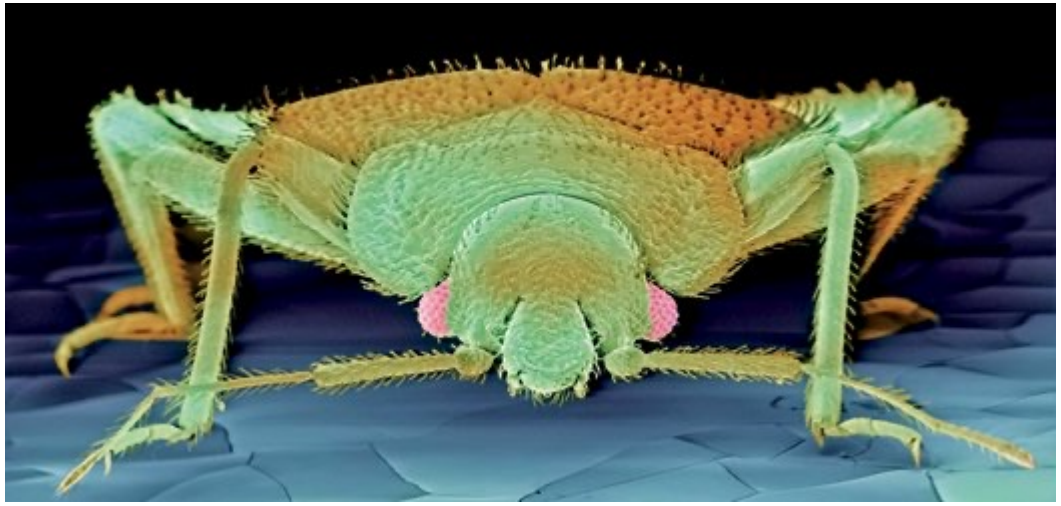
FELLOW OF THE ROYAL, GEOLOGICAL, LINNÆAN, ETC., SOCIETIES;
AUTHOR OF 'JOURNAL OF RESEARCHES DURING H. M. S. BEAGLE'S VOYAGE ROUND THE WORLD.'

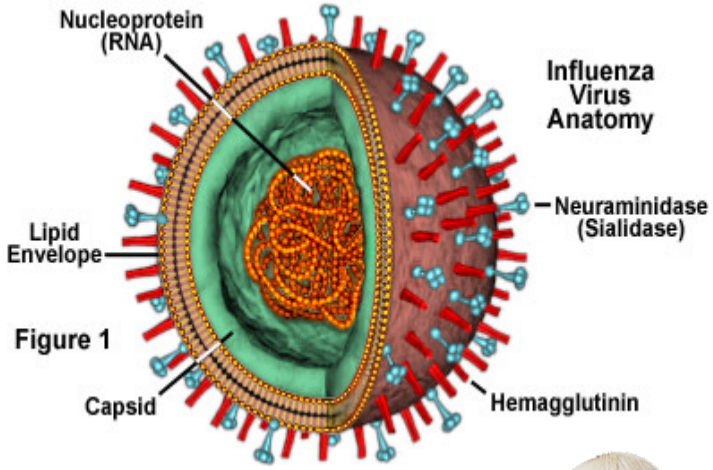
LONDON:
JOHN MURRAY, ALBEMARLE STREET.
1859.

*The right of Translation is reserved.*








Nucleoprotein
(RNA)
Influenza
Virus
Anatomy
Neuraminidase
(Sialidase)
Lipid
Envelope
Capsid
Hemagglutinin
Figure 1





# Pillars of Evolution Theory

- Selection (part of population capable of reproducing and transmitting its characteristics)
- Progress
- Rank based selection

# Introduction

- Evolutionary computation uses the computational model of evolutionary processes as key elements in the design and implementation of computer-based systems and problem solving applications.

- There are a variety of evolutionary computational models that have been proposed and studied which we will refer to as evolutionary algorithms.

- They share a common conceptual base of simulating the evolution of individual structures via processes of selection and reproduction.

- They depend on the performance (fitness) of the individual structures.

# Evolutionary algorithms (EA)

- More precisely, evolutionary algorithms maintain a population of structures that evolve according to rules of selection and other operators, such as recombination and mutation.

- Each individual in the population receives a measure of its fitness in the environment.

- Selection focuses attention on high fitness individuals, thus exploiting the available fitness information.

# Evolutionary algorithms (EA)

- procedure EA
  {
    t = 0;
    initialize population P(t);
    evaluate P(t);
    until (done) {
      t = t + 1;
      parent_selection P(t);
      recombine P(t);
      mutate P(t);
      evaluate P(t);
      survive P(t);
    }
  }

# Evolutionary algorithms (EA)

- An evolutionary algorithm typically initializes its population randomly, although domain specific knowledge can also be used to bias the search.
- Evaluation measures the fitness of each individual according to its worth in some environment.
- Evaluation may be as simple as computing a fitness function or as complex as running an elaborate simulation.

# Types of evolutionary algorithms

- Choice of genetic representation and operators
- Genetic algorithms (cross over and mutation)
- Evolutionary programming
- Island models (Maintains set of diverse population that evolve parallel and exchange genetic strigs every generation)
- Steady state evolution (individuals that obtained the worst fitness values are replaced by offspring of individuals that obtained best fitness values)
- Simulated annealing
- Population based incremental learning: operates on a single genetic string that represents an entire population
- Adaptive PBIL

# Evolutionary algorithms

- A population of individuals exists in an  environment (there are limited resources)
- **Individuals Compete** to acquire there resources  - causes selection of those **fitter** individuals that are better adapted to the environment
- These fitter individuals act as seeds for the generation of new individuals through different processes like recombination and mutation
- The new individuals have their fitness evaluated and compete (possibly also with parents) for survival.
- Over time **Natural selection** causes a rise in the fitness of the population
- This results in fitter population

- EAs fall into the category of “generate and test” algorithms
- They are stochastic, population-based algorithms
- Variation operators (recombination and mutation) create the necessary diversity and thereby facilitate novelty
- Selection reduces diversity and acts as a force pushing quality

# What is Heredity?

- Heredity is mechanism indicating the transmission of individual characteristics to their offspring through reproduction

# Then what is selection?

- Selection indicates that only part of the population is capable of reproducing and transmitting its characteristics to future generations.
- As per Darwin and Wallace – individuals tend to make several offspring and that not all of them may reproduce
- Selection of individuals that can reproduce is not completely random, but regulated by environment constraints..
- What will decide chances of survival and reproduction?

# Fittest Vs Best

- Fittest may not be the best and
- Best may not be the fittest
- Is it about individual score?
- What is the meaning of progress? (Is it new individuals are better than previous ones? - but natural selection has no comparative memory. Individuals are selected against environment and/or their peers at specific point in the space and time. Thus any change in environment over the time generates different selection conditions…. In other meaning of progress – individual tend to become better in future.

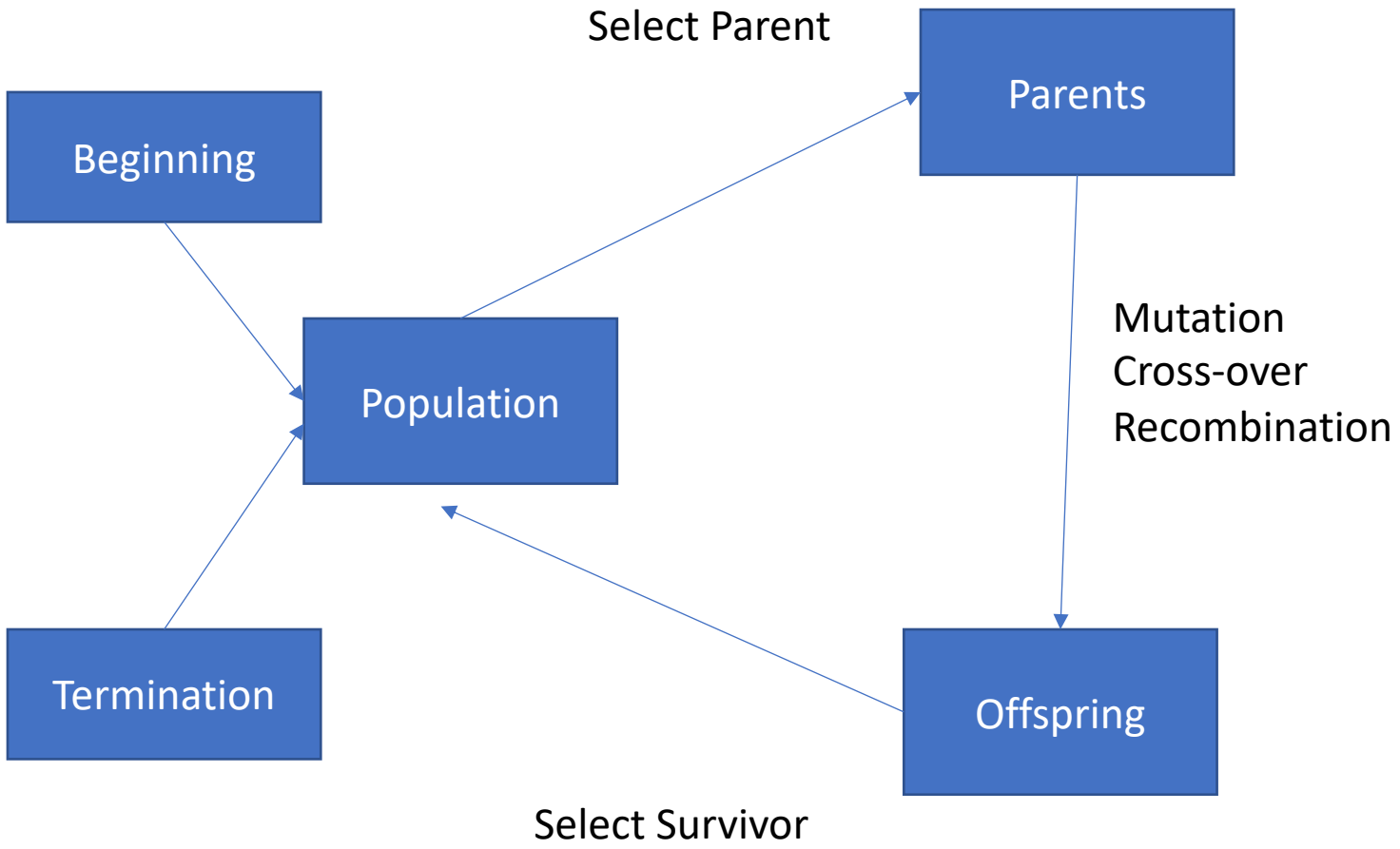
# Can evolutionary Learning…

- Termed give organism better ability to survive and cope up with the environment with respect to its peers and therefore to reproduce, have a probability of being transmitted to future generations.
- The generation of diversity provides adaptation power to evolving populations.
- Natural evolution makes sure that characteristics that affect the organism negatively have less probability of being transmitted to the next generations.

# Genotype

- The genetic material of an individual is known as the genotype whereas its manifestation as an organism is known as phenotype
- Natural selection solely operates on phenotype, but genotype is the ultimate vehicle of inheritance
- Cell contain a class of molecules, known as proteins, whose shape, concentration and behavior determine the properties of the cell. E.g. hair cells and muscle cells are different because they are composed of different proteins.
- Definition of specific protein depends on another molecule known as DNA which intern relies on proteins to become operative and on the mediation of third type of molecule known as RNA.
- DNA is gentic material that is transmitted over generations.

# EA


Select Parent
Parents
Beginning
Population
Mutation
Cross-over
Recombination
Termination
Offspring
Select Survivor

# Genetic Algorithm

- Mutation

# How it works?

```
BEGIN
  INITIALISE population with random candidate solutions;
  EVALUATE each candidate;
  REPEAT UNTIL ( TERMINATION CONDITION is satisfied ) DO
    1 SELECT parents;
    2 RECOMBINE pairs of parents;
    3 MUTATE the resulting offspring;
    4 EVALUATE new candidates;
    5 SELECT individuals for the next generation;
  OD
END
```

# Competition

- Adapting to environment
- Objective function
- Basis or criteria for selection
  - Variety
- Want to improve fitness

# Population

- Holds (representations of) possible solutions
- Usually has a fixed size and is a *multiset* of genotypes
- Some sophisticated EAs also assert a spatial structure on the population e.g., a grid.
- Selection operators usually take whole population into account i.e., reproductive probabilities are *relative* to *current* generation
- Diversity of a population refers to the number of different fitness values / phenotypes / genotypes present

- Single point crossover
- Two point crossover

# Genetic Algorithms:
# Genetic Procreation Operators

- However, there are a number of crossover operators that have been used on binary and real-coded GAs:
  - Single-point Crossover,
  - Two-point Crossover,
  - Uniform Crossover

# Genetic Algorithms: Single-Point Crossover

- Given two parents, single-point crossover will generate a cut-point and recombines the first part of first parent with the second part of the second parent to create one offspring.
- Single-point crossover then recombines the second part of the first parent with the first part of the second parent to create a second offspring.

# Genetic Algorithms: Single-Point Crossover

- Example:
  - Parent 1: X X | X X X X X
  - Parent 2: Y Y | Y Y Y Y Y
  - Offspring 1: X X Y Y Y Y Y
  - Offspring 2: Y Y X X X X X

# Genetic Algorithms: Two-Point Crossover

- Two-Point crossover is very similar to single-point crossover except that two cut-points are generated instead of one.

# Genetic Algorithms: Two-Point Crossover

- Example:
  - Parent 1: **X X** | X X X | **X X**
  - Parent 2: Y Y | **Y Y Y** | Y Y
  - Offspring 1: **X X Y Y Y X X**
  - Offspring 2: Y Y X X X Y Y

# Genetic Algorithms: Uniform Crossover

- In Uniform Crossover, a value of the first parent's gene is assigned to the first offspring and the value of the second parent's gene is to the second offspring with probability 0.5.
- With probability 0.5 the value of the first parent's gene is assigned to the second offspring and the value of the second parent's gene is assigned to the first offspring.

# Genetic Algorithms: Uniform Crossover

- Example:
  - Parent 1: **X** X **X** X X **X** X
  - Parent 2: Y **Y** Y **Y** **Y** Y **Y**
  - Offspring 1: **X Y X Y Y X Y**
  - Offspring 2: Y X Y X X Y X

# Genetic Algorithms: Real-Coded Crossover Operators

- For Real-Coded representations there exist a number of other crossover operators:
  - Mid-Point Crossover,
  - Flat Crossover (BLX-0.0),
  - BLX-0.5

# Genetic Algorithms: Mid-Point Crossover

- Given two parents where X and Y represent a floating point number:
  - Parent 1: X
  - Parent 2: Y
  - Offspring: (X+Y)/2
- If a chromosome contains more than one gene, then this operator can be applied to each gene with a probability of $P_{mp}$.

# Genetic Algorithms: Flat Crossover

- Flat crossover was developed by Radcliffe (1991)
- Given two parents where X and Y represent a floating point number:
  - Parent 1: X
  - Parent 2: Y
  - Offspring: rnd(X,Y)
- Of course, if a chromosome contains more than one gene then this operator can be applied to each gene.
- It uses the random real number to create offspring from two parents. It selects two parents as X and Y and generate the an offspring such that it selects random real number which is either min or max from genes in both parents and then assign this real number in child gene

# GENETIC ALGORITHMS

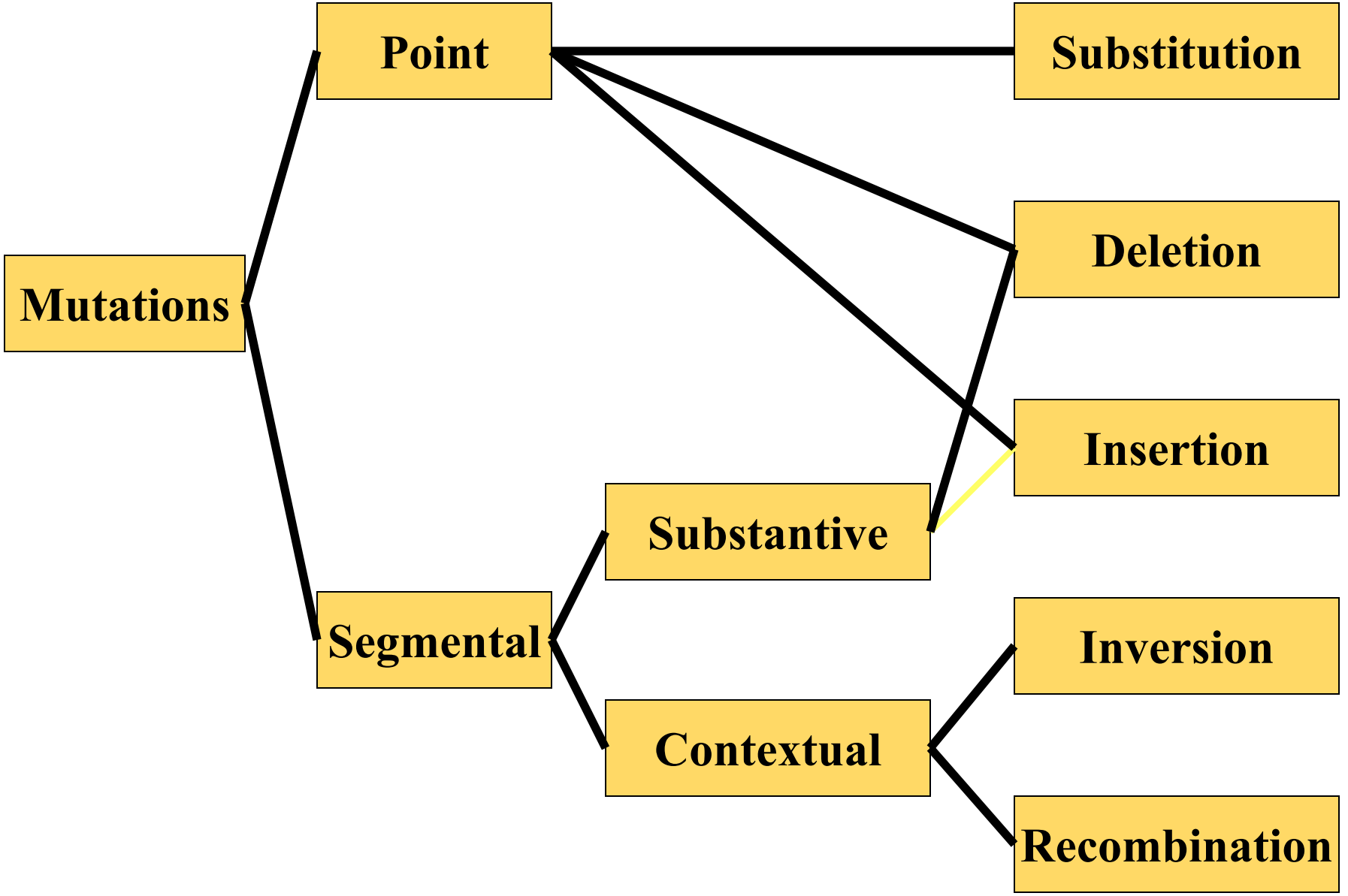
- Genetic Algorithm is variant of local beam search where successor states are generated by combining two parent states, rather than by modifying a single state.
- K randomly generated states – population
- Each state is rated by evaluation function or fitness function
- Fitness function returns higher value for the better state

- Mutation: In genetic algorithms of computing, mutation is a genetic operator used to maintain genetic diversity from one generation of a population of algorithm chromosomes to the next. It is analogous to biological mutation. The classic example of a mutation operator involves a probability that an arbitrary bit in a genetic sequence will be changed from its original state. A common method of implementing the mutation operator involves generating a random variable for each bit in a sequence. This random variable tells whether or not a particular bit will be modified. This mutation procedure, based on the biological point mutation, is called single point mutation. Other types are inversion and floating point mutation. When the gene encoding is restrictive as in permutation problems, mutations are swaps, inversions and scrambles.

- The purpose of mutation in GAs is preserving and introducing diversity. Mutation should allow the algorithm to avoid local minima by preventing the population of chromosomes from becoming too similar to each other, thus slowing or even stopping evolution.
- Cross Over: In genetic algorithms, crossover is a genetic operator used to vary the programming of a chromosome or chromosomes from one generation to the next. It is analogous to reproduction and biological crossover, upon which genetic algorithms are based.

# Genetic Mutation

- DNA molecules can change by means of mutation, or errors, that occur during replication.
- Substitution mutation
- Inversion
- Recombination
- Insertion and deletion


Mutations
Point
Substitution
Deletion
Insertion
Segmental
Substantive
Inversion
Contextual
Recombination

# General Algorithm for GA

- **Initialization**
  - Initially many individual solutions are randomly generated to form an initial population. The population size depends on the nature of the problem, but typically contains several hundreds or thousands of possible solutions.
  - Traditionally, the population is generated randomly, covering the entire range of possible solutions (the *search space*).
  - Occasionally, the solutions may be "seeded" in areas where optimal solutions are likely to be found.

# General Algorithm for GA

**Selection**

- During each successive generation, a proportion of the existing population is selected to breed a new generation.
- Individual solutions are selected through a *fitness-based* process, where fitter solutions (as measured by a fitness function) are typically more likely to be selected.
- Certain selection methods rate the fitness of each solution and preferentially select the best solutions. Other methods rate only a random sample of the population, as this process may be very time-consuming.
- Most functions are stochastic and designed so that a small proportion of less fit solutions are selected. This helps keep the diversity of the population large, preventing premature convergence on poor solutions. Popular and well-studied selection methods include roulette wheel selection and tournament selection.

# General Algorithm for GA

**Reproduction**

- The next step is to generate a second generation population of solutions from those selected through genetic operators:

  crossover (also called recombination), and/or mutation.
- For each new solution to be produced, a pair of "parent" solutions is selected for breeding from the pool selected previously.
- By producing a "child" solution using the above methods of crossover and mutation, a new solution is created which typically shares many of the characteristics of its "parents". New parents are selected for each child, and the process continues until a new population of solutions of appropriate size is generated.

# General Algorithm for GA

- These processes ultimately result in the next generation population of chromosomes that is different from the initial generation.

- Generally the average fitness will have increased by this procedure for the population, since only the best organisms from the first generation are selected for breeding, along with a small proportion of less fit solutions, for reasons already mentioned above.

# Crossover

- the most common type is single point crossover. In single point crossover, you choose a locus at which you swap the remaining alleles from on parent to the other. This is complex and is best understood visually.
- As you can see, the children take one section of the chromosome from each parent.
- The point at which the chromosome is broken depends on the randomly selected crossover point.
- This particular method is called single point crossover because only one crossover point exists. Sometimes only child 1 or child 2 is created, but oftentimes both offspring are created and put into the new population.
- Crossover does not always occur, however. Sometimes, based on a set probability, no crossover occurs and the parents are copied directly to the new population. The probability of crossover occurring is usually 60% to 70%.

# Mutation

- After selection and crossover, you now have a new population full of individuals.
- Some are directly copied, and others are produced by crossover.
- In order to ensure that the individuals are not all exactly the same, you allow for a small chance of mutation.
- You loop through all the alleles of all the individuals, and if that allele is selected for mutation, you can either change it by a small amount or replace it with a new value. The probability of mutation is usually between 1 and 2 tenths of a percent.
- Mutation is fairly simple. You just change the selected alleles based on what you feel is necessary and move on. Mutation is, however, vital to ensuring genetic diversity within the population.

# General Algorithm for GA

**Termination**

- This generational process is repeated until a termination condition has been reached.
- Common terminating conditions are:
  - A solution is found that satisfies minimum criteria
  - Fixed number of generations reached
  - Allocated budget (computation time/money) reached
  - The highest ranking solution's fitness is reaching or has reached a plateau such that successive iterations no longer produce better results
  - Manual inspection
  - Any Combinations of the above

# Evolving Neural Networks

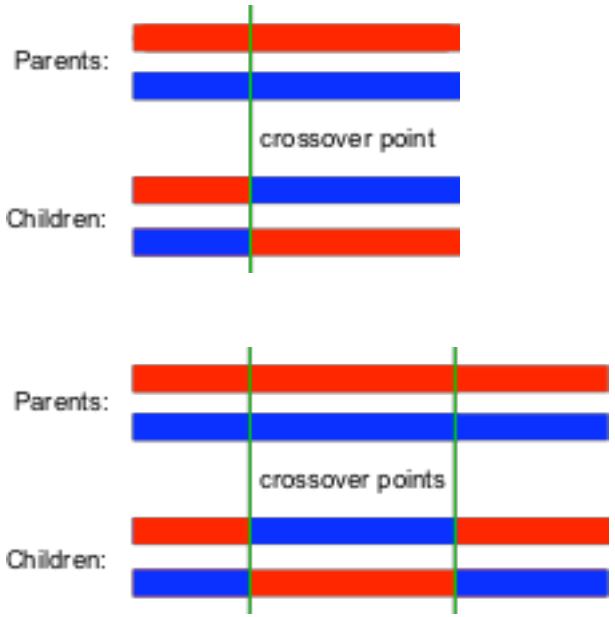
- Evolving the architecture of neural network is slightly more complicated, and there have been several ways of doing it. For small nets, a simple matrix represents which neuron connects which, and then this matrix is, in turn, converted into the necessary 'genes', and various combinations of these are evolved.

# Evolving Neural Networks

- Many would think that a learning function could be evolved via genetic programming. Unfortunately, genetic programming combined with neural networks could be *incredibly* slow, thus impractical.
- As with many problems, you have to constrain what you are attempting to create.
- For example, in 1990, David Chalmers attempted to evolve a function as good as the delta rule.
- He did this by creating a general equation based upon the delta rule with 8 unknowns, which the genetic algorithm then evolved.

# CROSS OVER

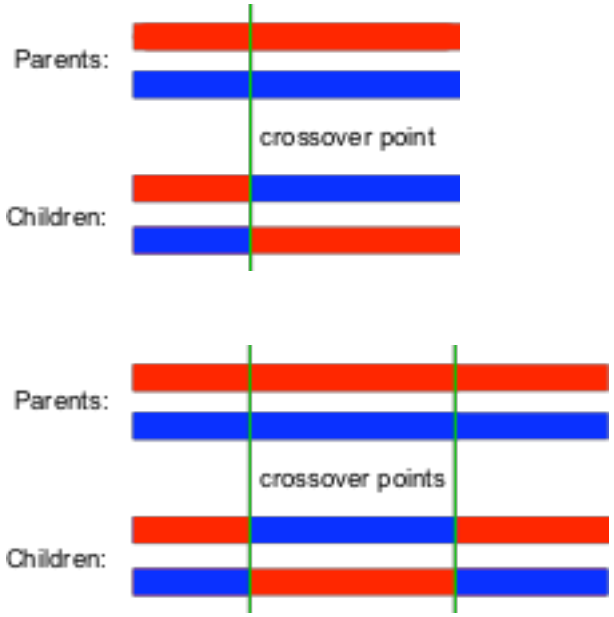
- Many crossover techniques exist for organisms which use different data structures to store themselves.



# Other Areas

- Genetic Algorithms can be applied to virtually any problem that has a large search space.
- Al Biles uses genetic algorithms to filter out 'good' and 'bad' riffs for jazz improvisation.
- The military uses GAs to evolve equations to differentiate between different radar returns.
- Stock companies use GA-powered programs to predict the stock market.

# Genetic

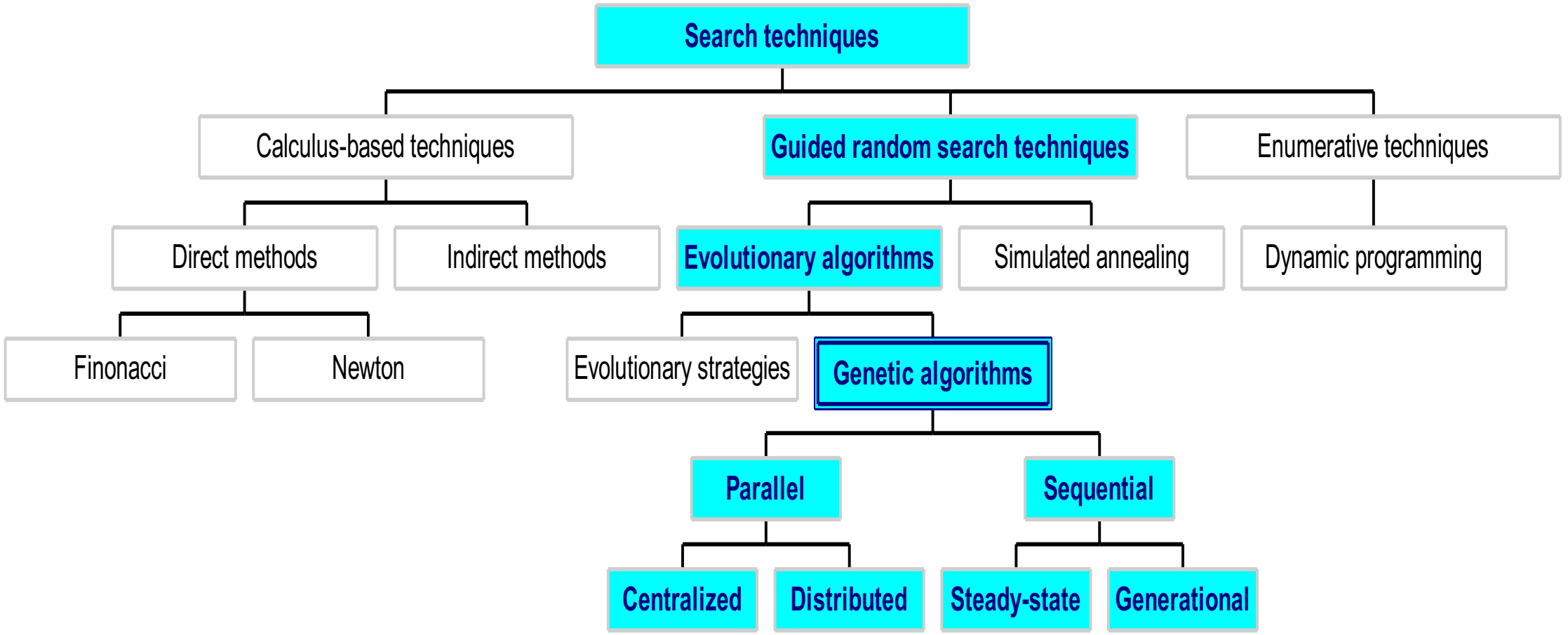



*"Genetic Algorithms are good at taking large, potentially huge search spaces and navigating them, looking for optimal combinations of things, solutions you might not otherwise find in a lifetime."*

- Salvatore Mangano

*Computer Design*, May 1995

# Guided Searching – Heuristic and Evolution



# Genetic Algorithm – An evolutionary horizon?

A problem to solve, and ...

- Encoding technique (*gene, chromosome*)
- Initialization procedure *(creation)*
- Evaluation function *(environment)*
- Selection of parents *(reproduction)*
- Genetic operators *(mutation, recombination)*
- Parameter settings *(practice and art)*

# Simple Genetic Algorithm → What complexity you may introduce?

```
{
    initialize population;
    evaluate population;
    while Exit Critria
    {
        select parents for reproduction;
        perform recombination and mutation;
        evaluate population;
    }
}
```

# Genetic algorithms

- Twist on Local Search: successor is generated by combining two parent states
- A state is represented as a string over a finite alphabet (e.g. binary)
  - 8-queens
    - State = position of 8 queens each in a column
- Start with *k* randomly generated states (population)
- Evaluation function (fitness function):
  - Higher values for better states.
  - Opposite to heuristic function, e.g., # non-attacking pairs in 8-queens
- Produce the next generation of states by “simulated evolution”
  - Random selection
  - Crossover
  - Random mutation

# Genetic algorithms



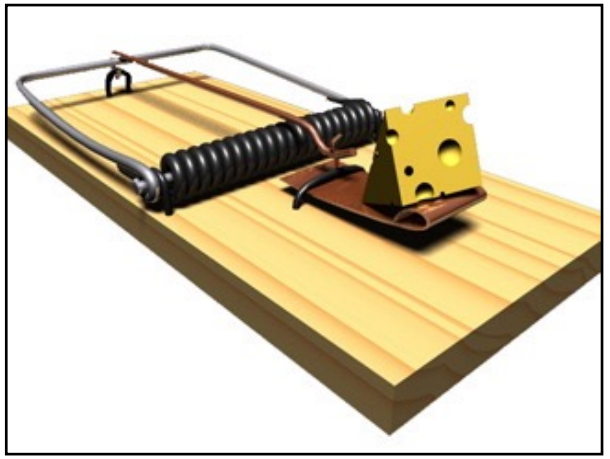

4 states for 8-queens problem

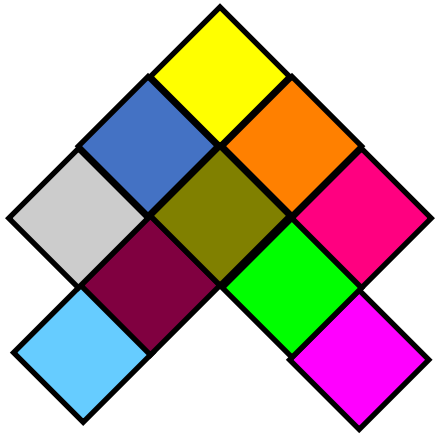
2 pairs of 2 states randomly selected based on fitness. Random crossover points selected

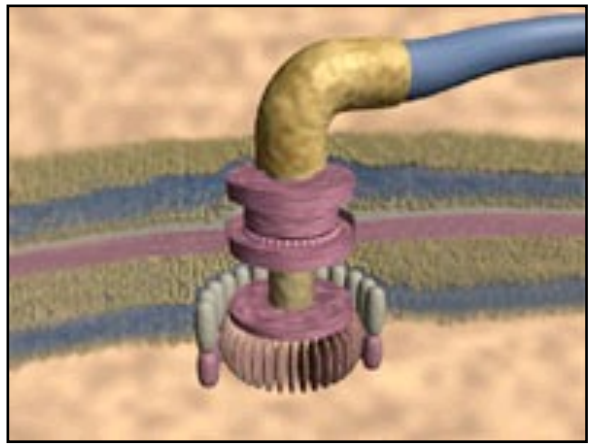
New states after crossover

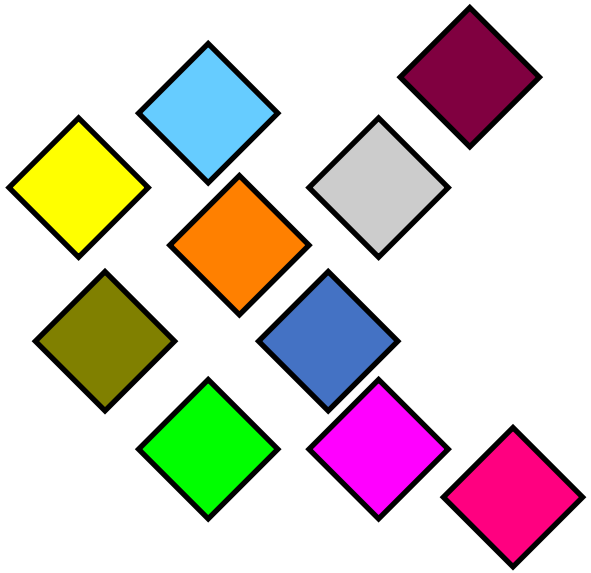
Random mutation applied

- Fitness function: number of non-attacking pairs of queens (min = 0, max = 8 × 7/2 = 28)
- 24/(24+23+20+11) = 31%
- 23/(24+23+20+11) = 29% etc

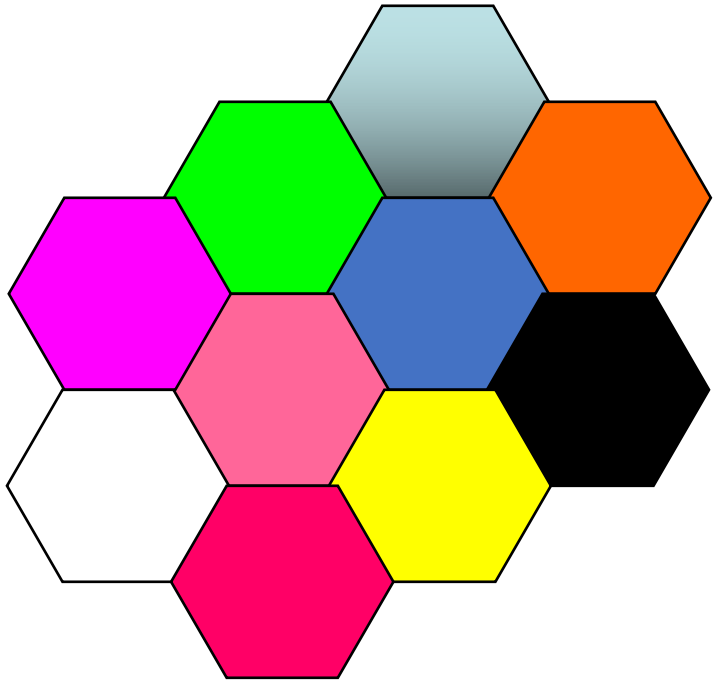


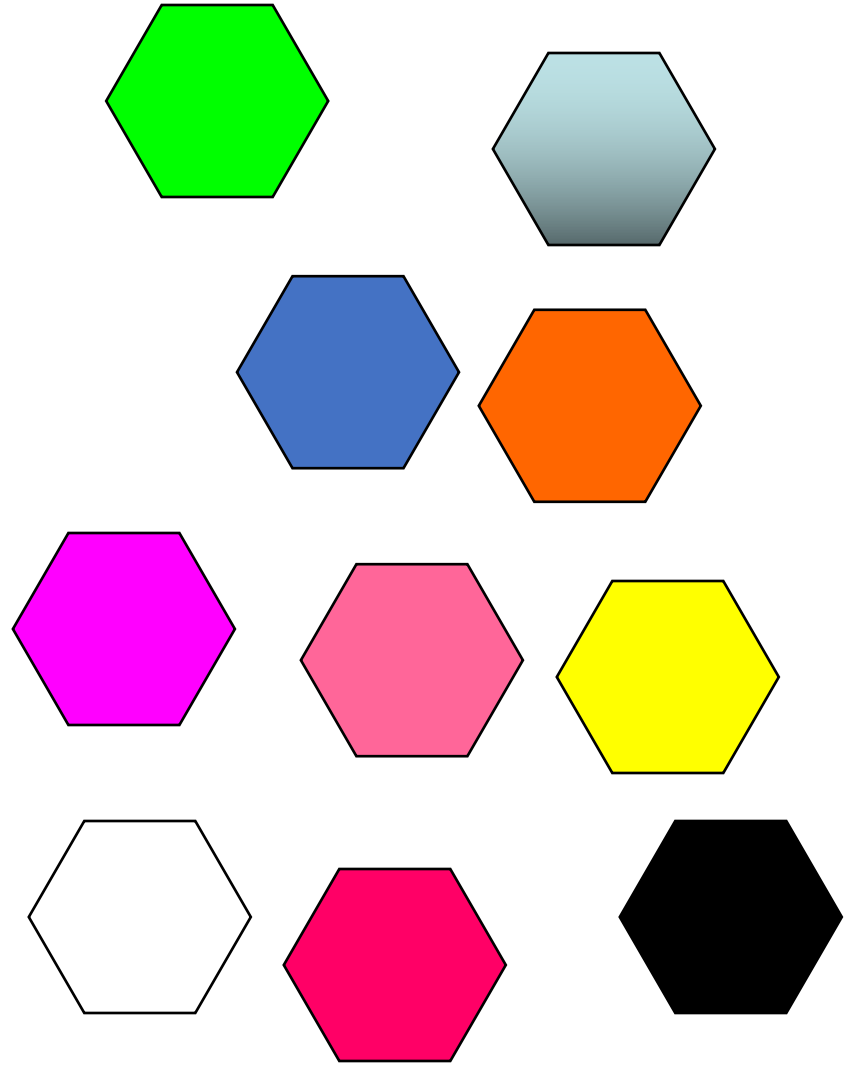


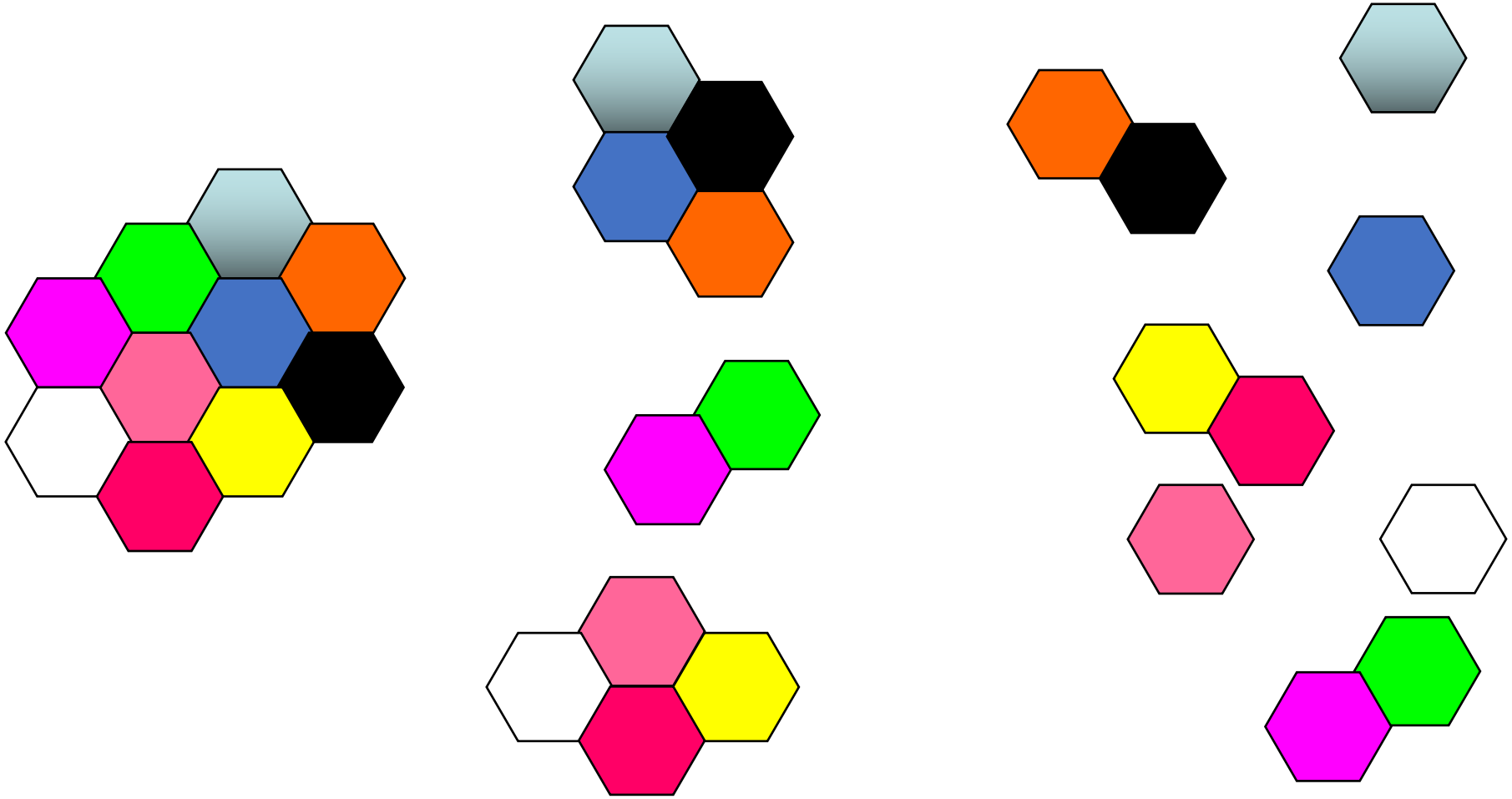




Ref:



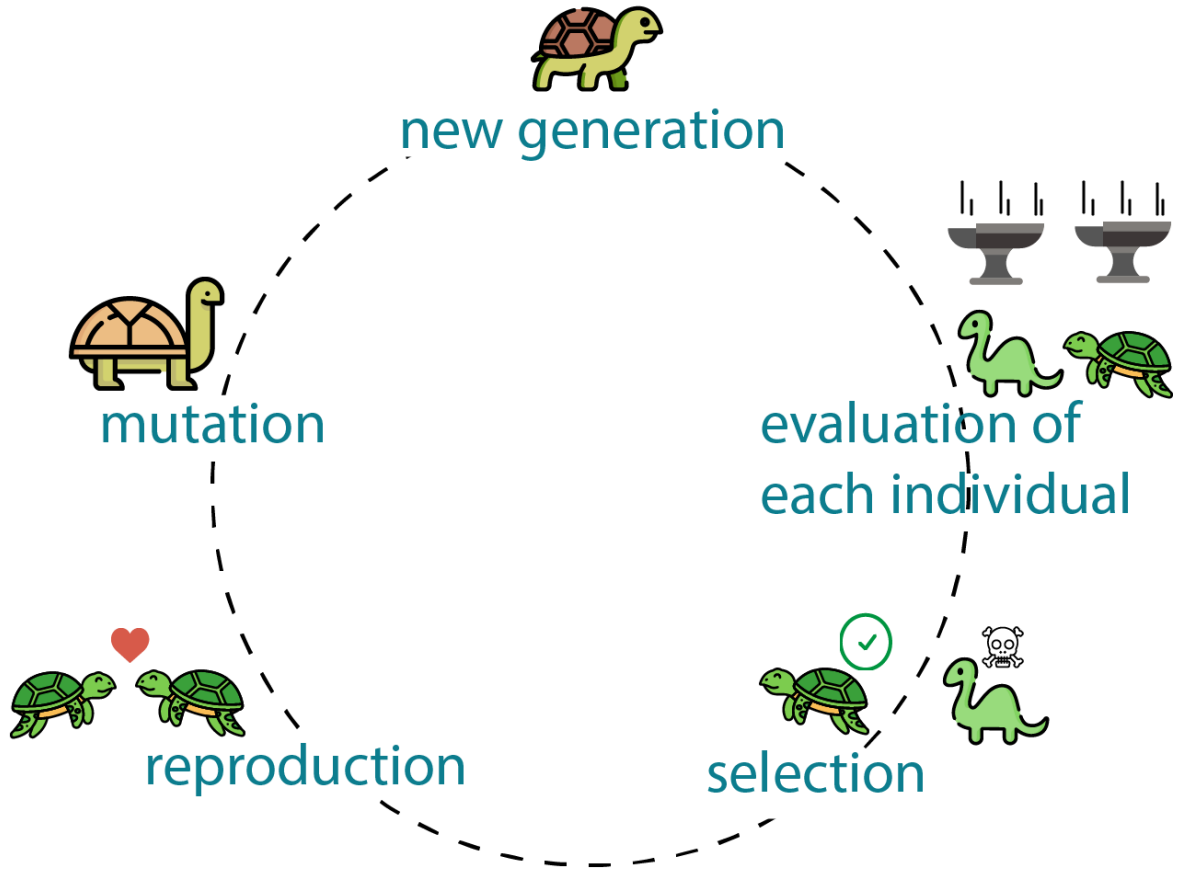




# A typical genetic algorithm has

five phases:

1. Initialization
2. Evaluation
3. Selection
4. Crossover / Reproduction
5. Mutation

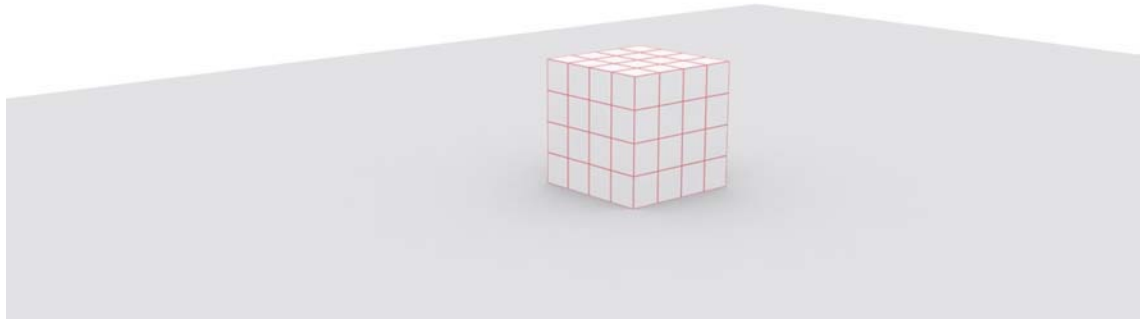


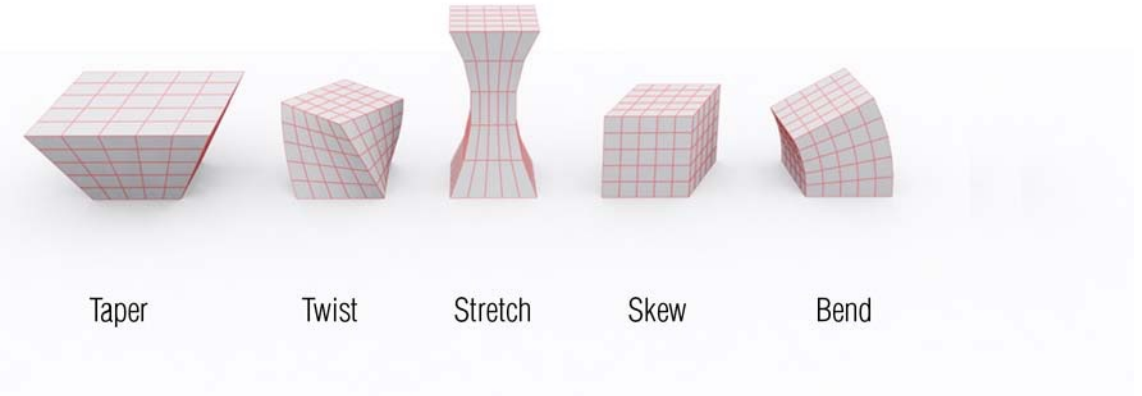



Ref:

# Evolutionary Computation History

- Different areas of evolutionary computation:
  - Genetic algorithms
  - Evolutionary programming
  - Evolution strategies
  - Genetic programming
  - Particle swarm optimization

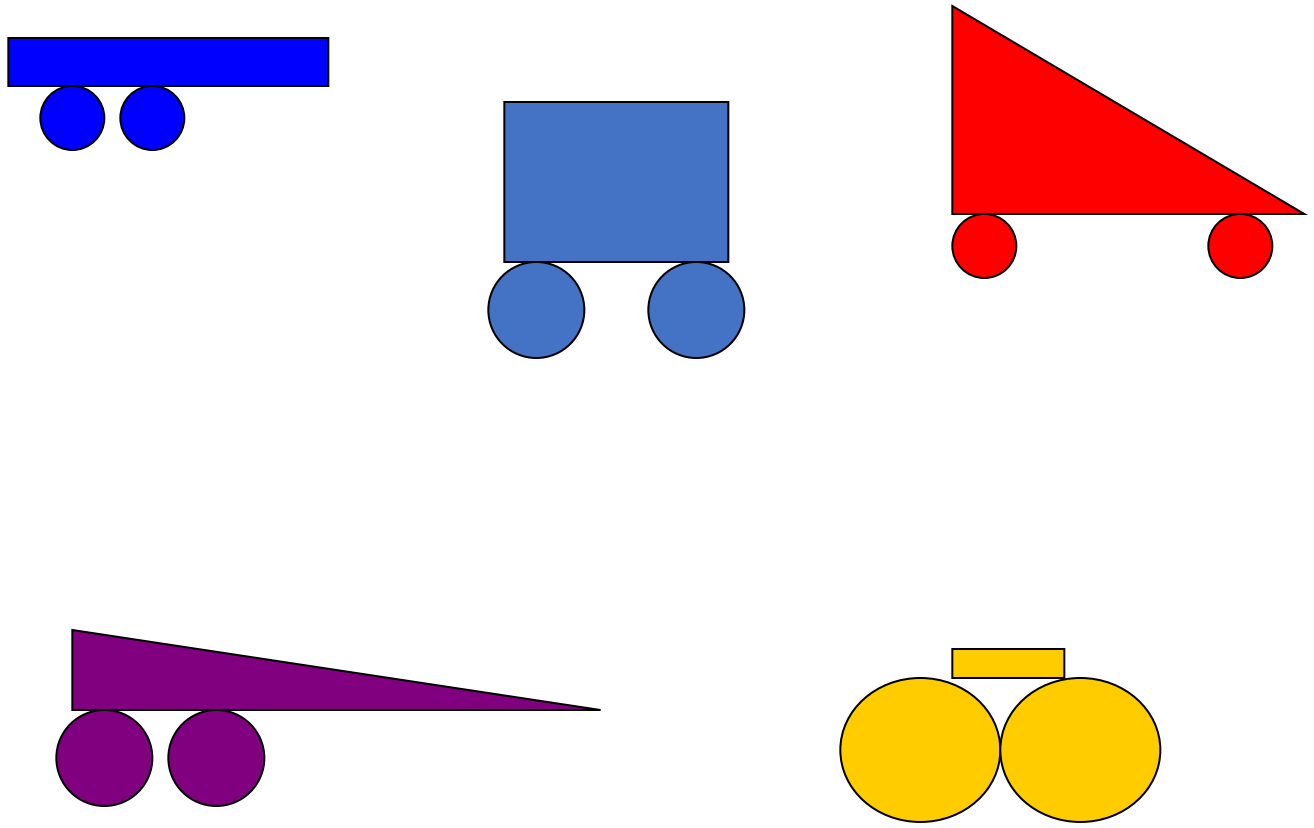







Ref: Paper – marin, bignon

# Initial population



Ref: Biologically inspired computing

# Genetic Evolution

- Fitness function =

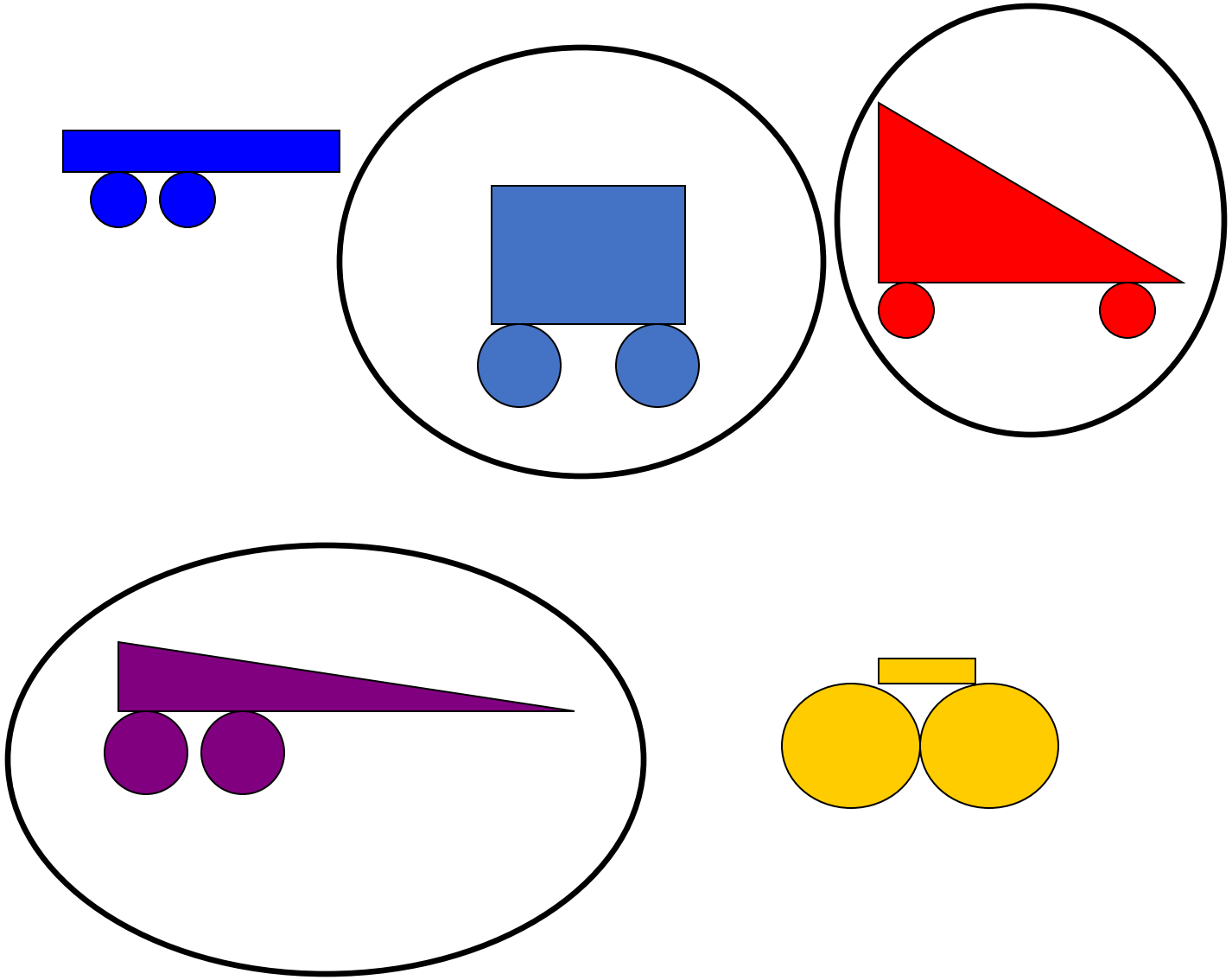
  **Top area /wheel area +top length /wheel distance**

  **Rare height /front height**

  - What could be other fitness functions?
- Genetic operators
  1. Wheel of one + body of other (cross-over)
  2. Abrupt wheel change (mutation)
  3. Abrupt body shape change (mutation)
  4. Partial color change (mutation)
  5. One wheel of one + other part of other
  6. Shape of one and rest of the things of other

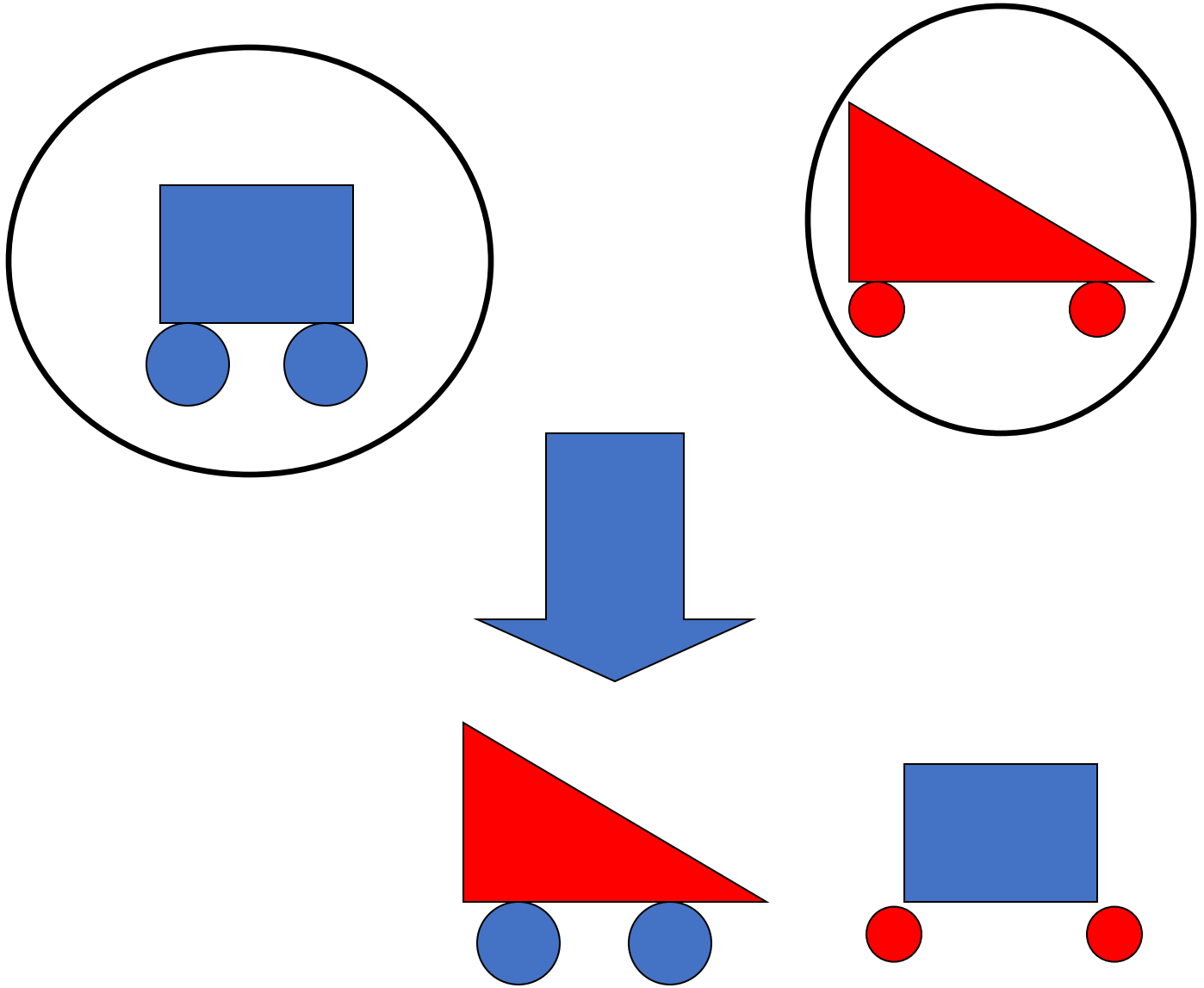
  What could be other genetic operators?

# Select



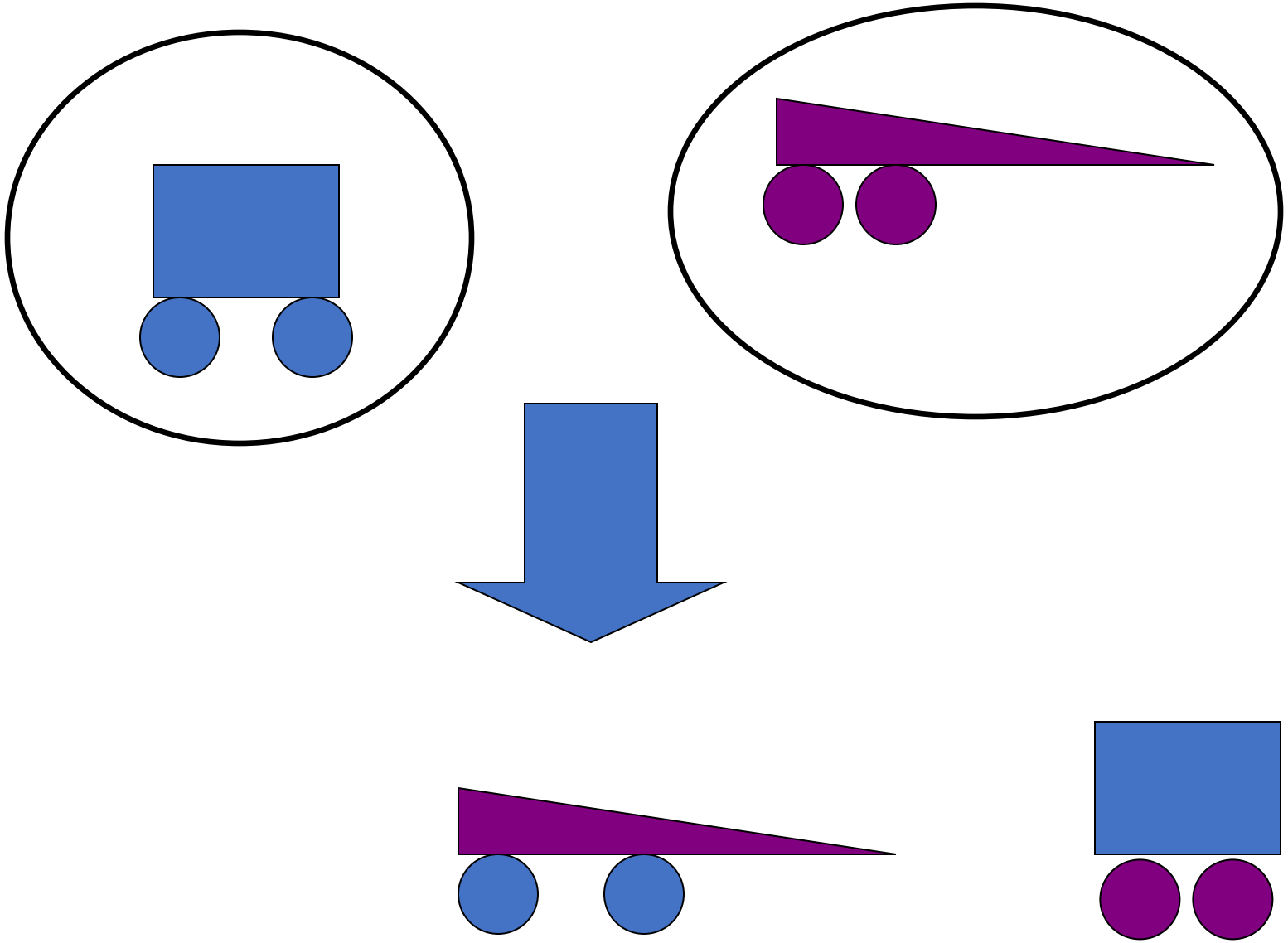
Ref: Biologically inspired computing

# Crossover



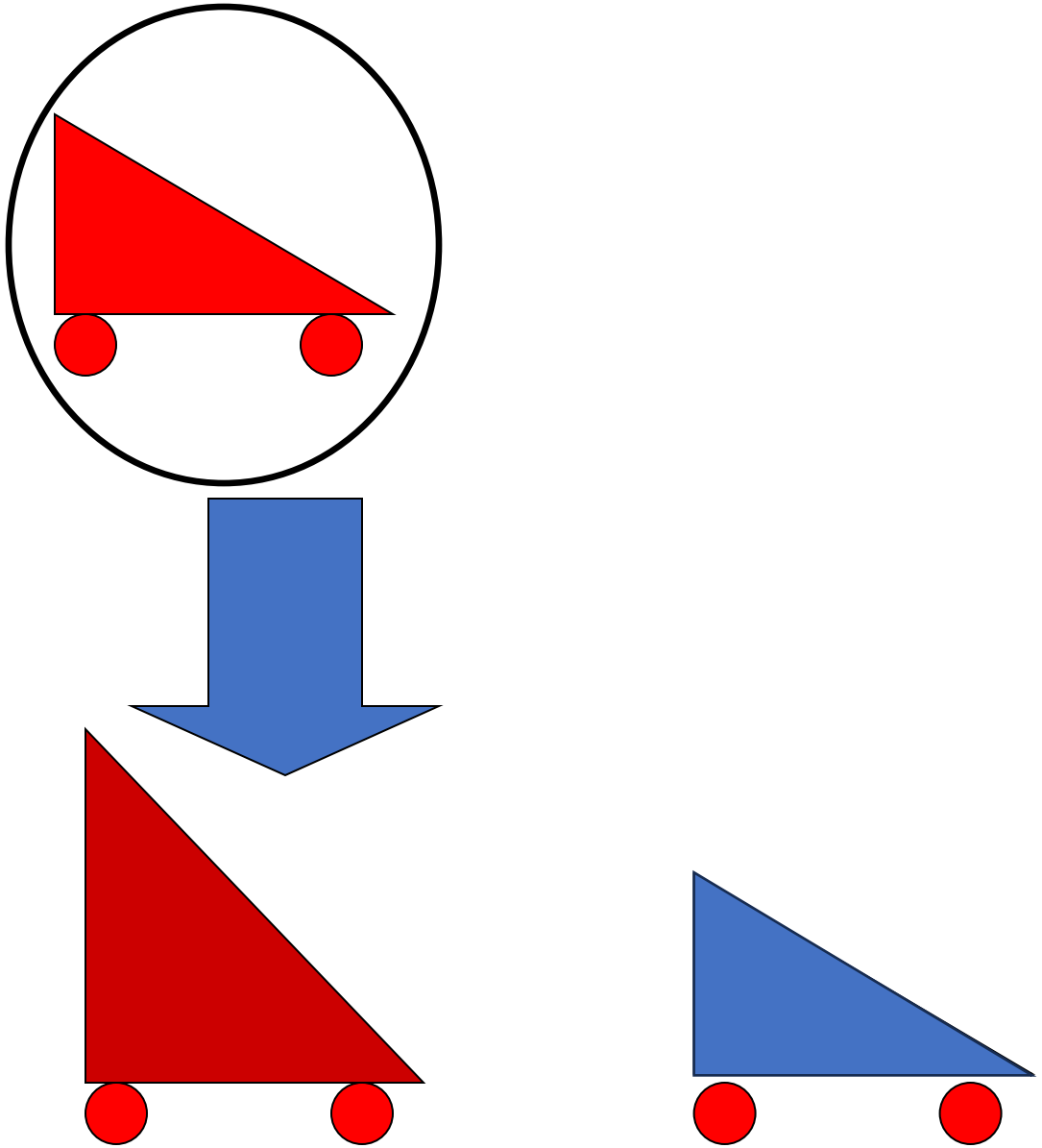
Ref: Biologically inspired computing

# Another Crossover



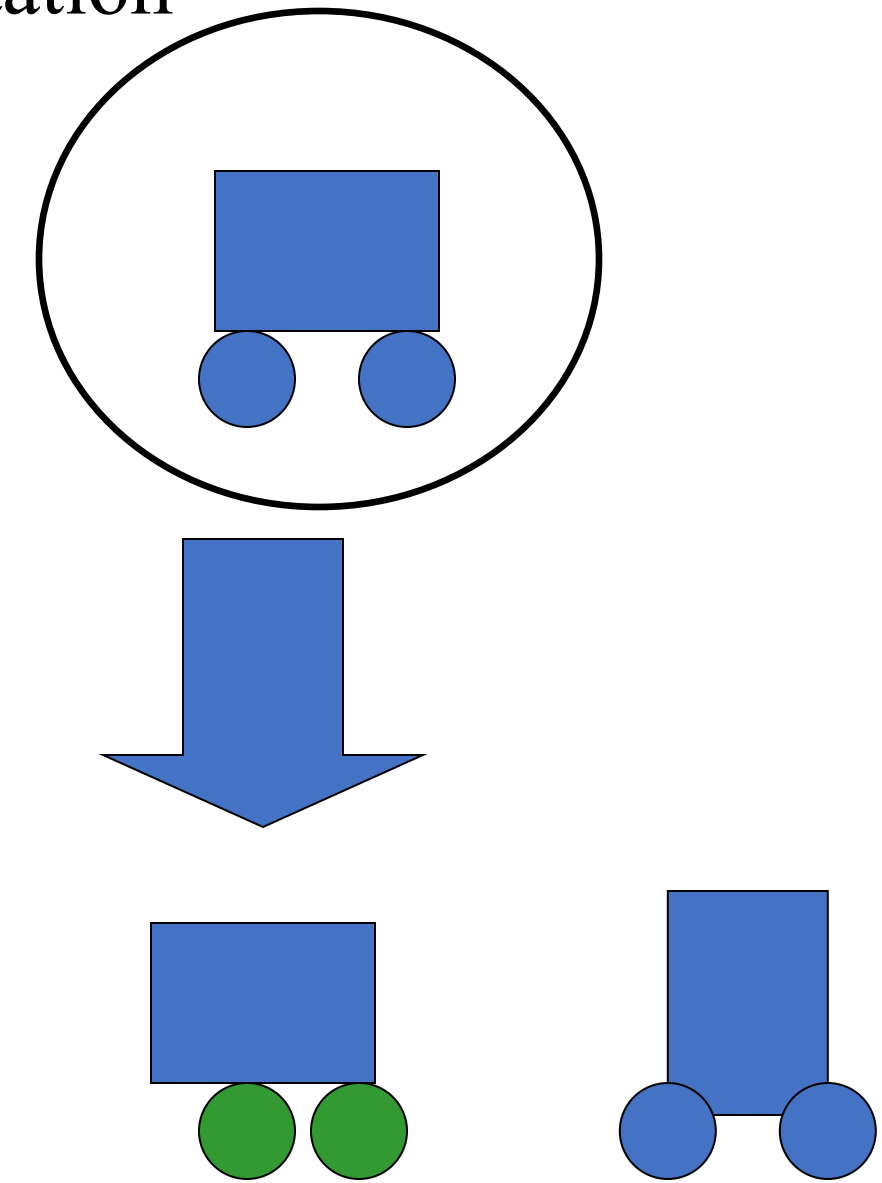
Ref: Biologically inspired computing

# A mutation



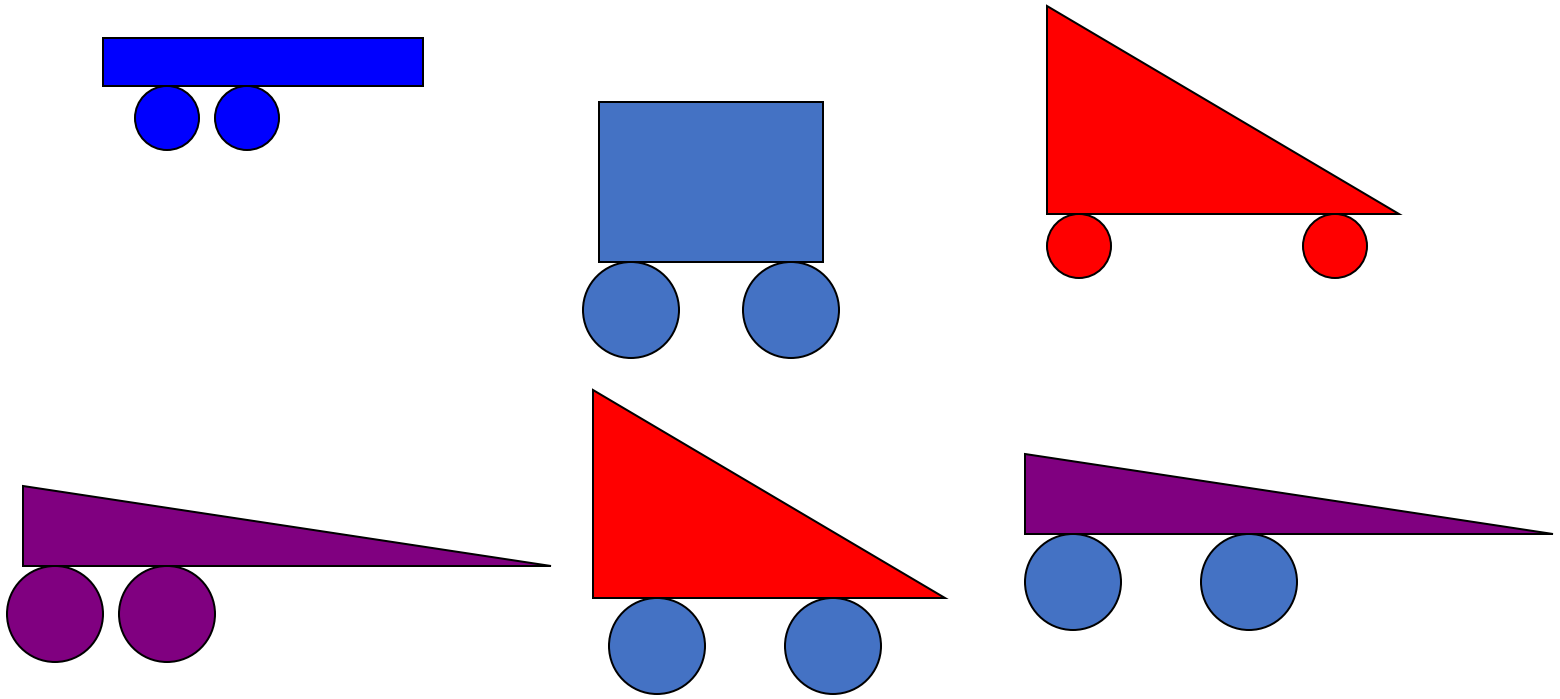
Ref: Biologically inspired computing

# Another Mutation



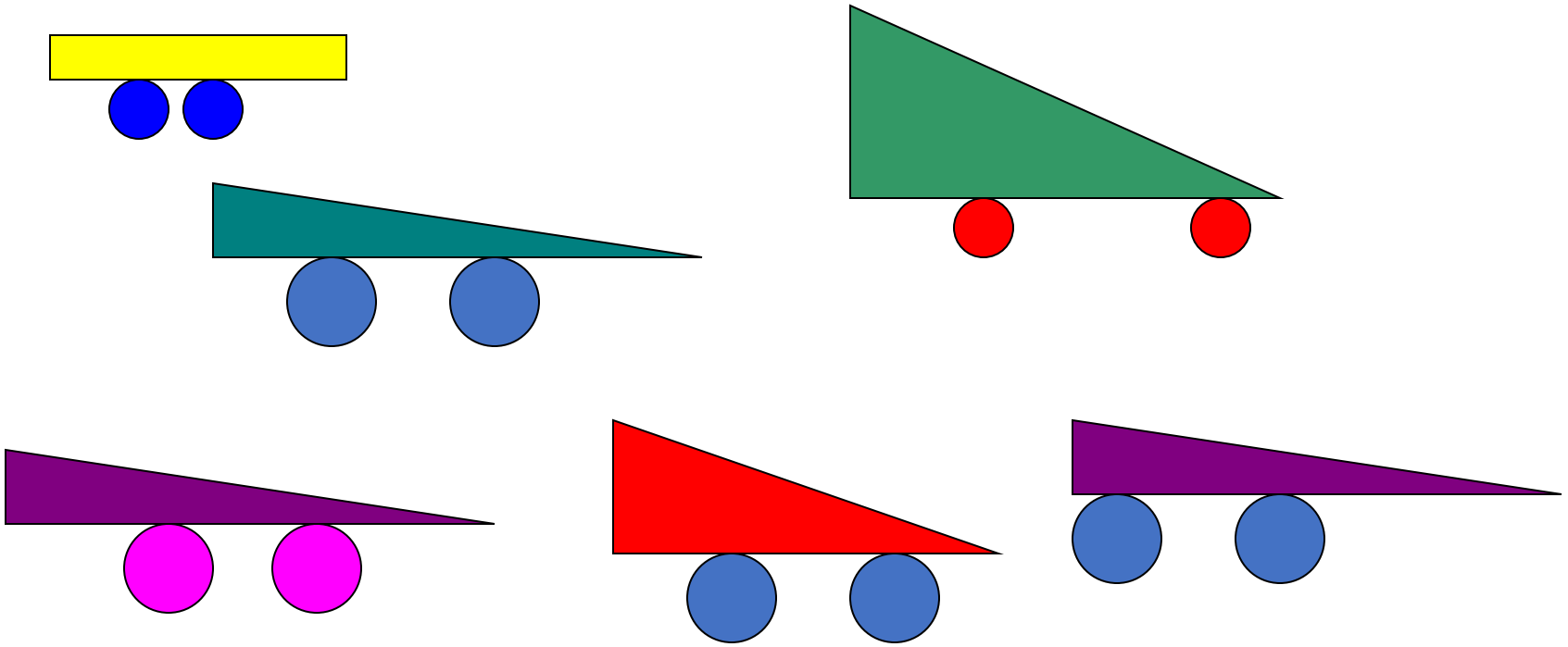
Ref: Biologically inspired computing

# New Population: Generation 2



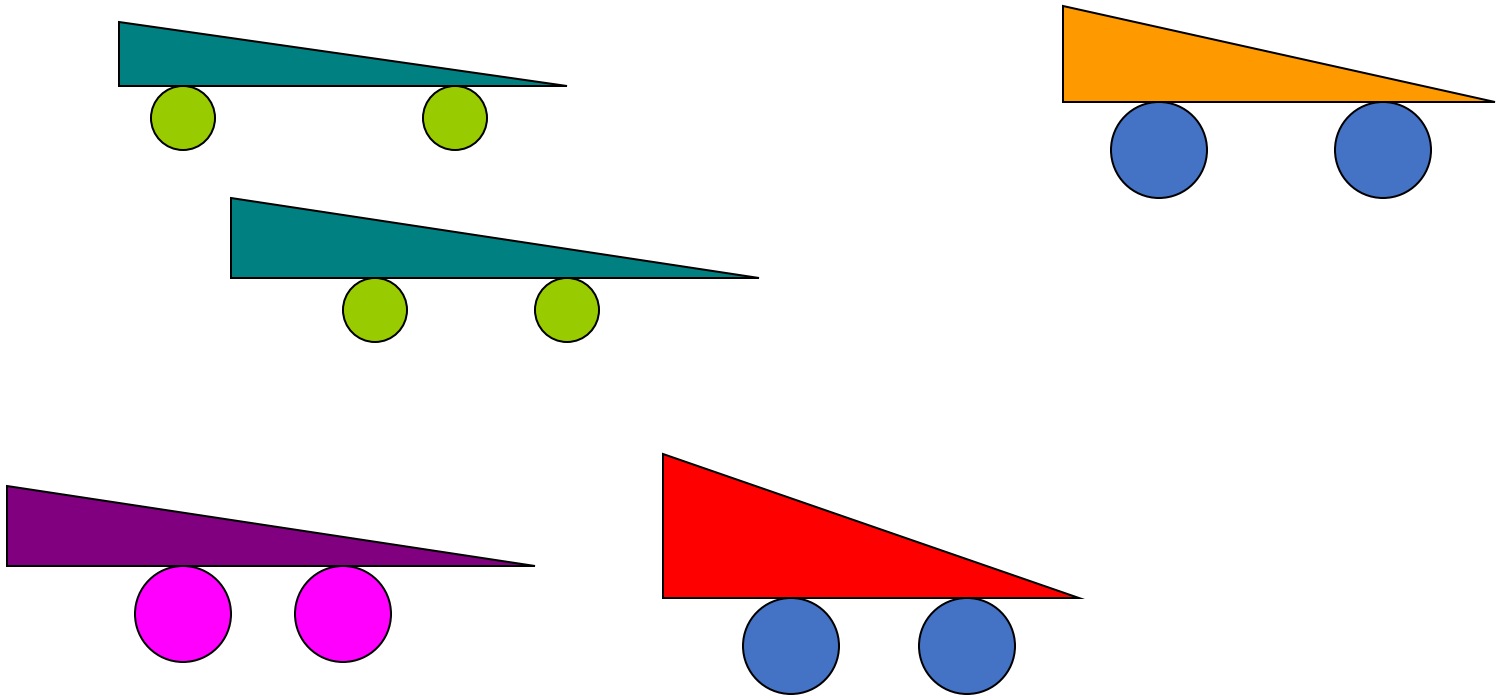
Ref: Biologically inspired computing

# Generation 3



Ref: Biologically inspired computing

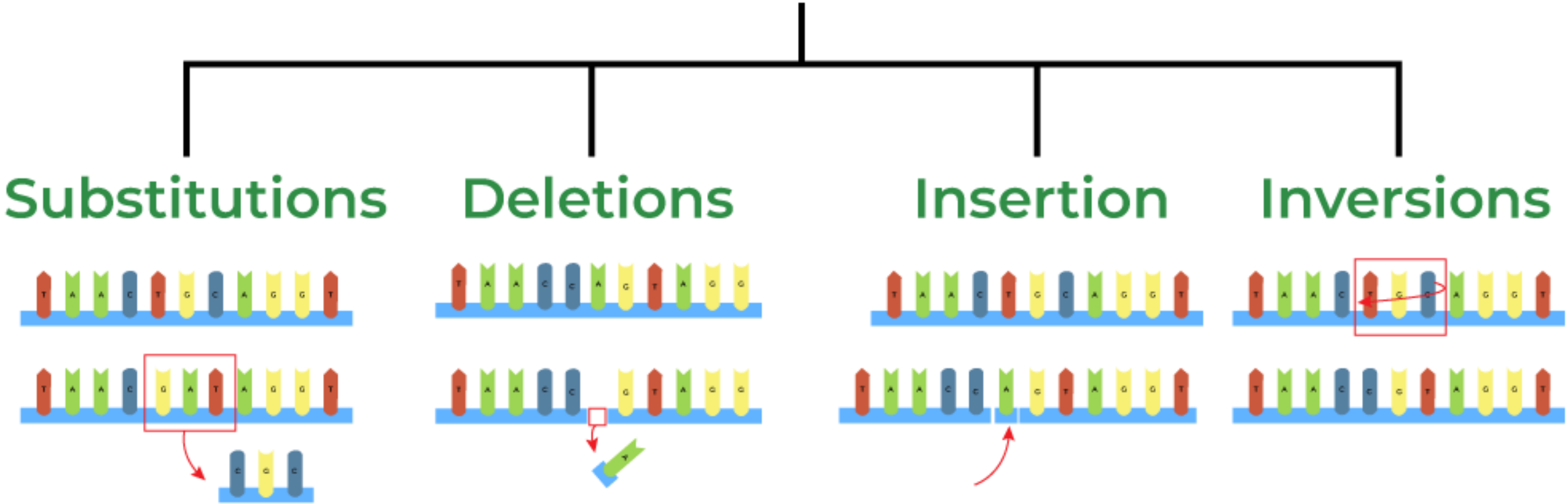
# Generation 4, etc …



Ref: Biologically inspired computing

# Types of Mutations

## (At the DNA level)

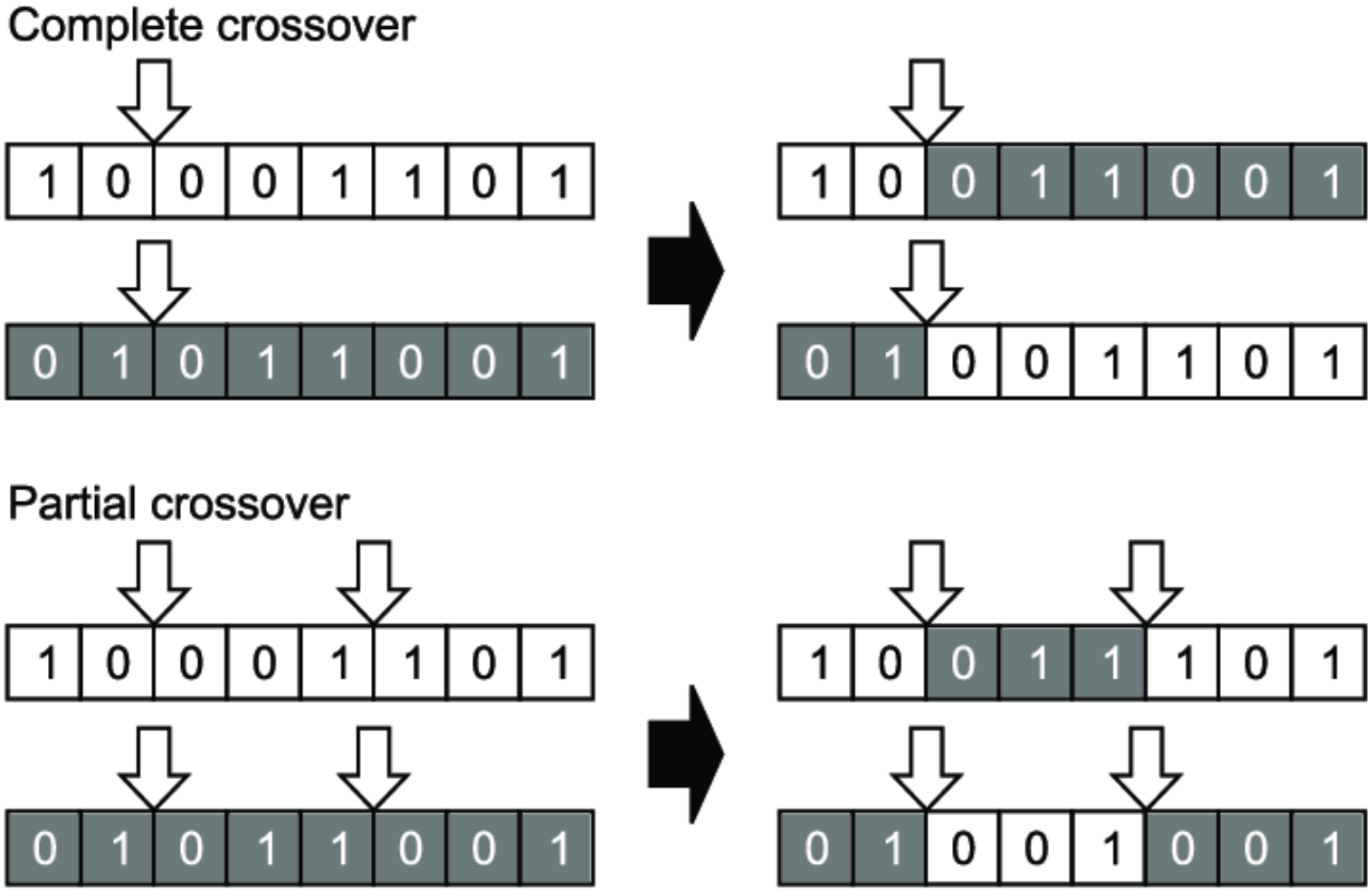



By Parag Kulkarni

# Genetic Algorithm:
# Two-Point Crossover

- Example:
  - Parent 1: **X X** | X X X | **X X**
  - Parent 2: Y Y | **Y Y Y** | Y Y
  - Offspring 1: **X X Y Y Y X X**
  - Offspring 2: Y Y X X X Y Y

# Crossover (Single point, multi-point)



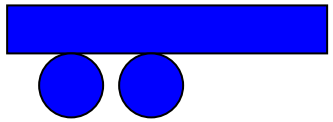
# Evolving Neural Networks

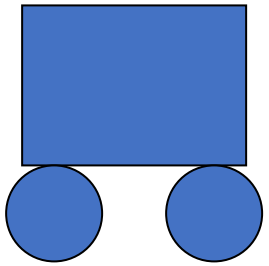
- Evolving the architecture of neural network is slightly more complicated, and there have been several ways of doing it. For small nets, a simple matrix represents which neuron connects which, and then this matrix is, in turn, converted into the necessary 'genes', and various combinations of these are evolved.

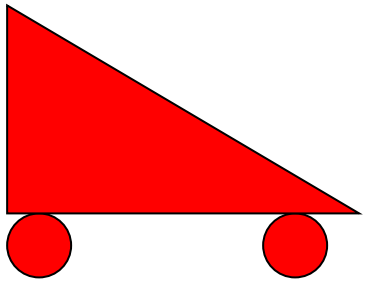
# Evolving Neural Networks

- Many would think that a learning function could be evolved via genetic programming. Unfortunately, genetic programming combined with neural networks could be *incredibly* slow, thus impractical.
- As with many problems, you have to constrain what you are attempting to create.
- For example, in 1990, David Chalmers attempted to evolve a function as good as the delta rule.
- He did this by creating a general equation based upon the delta rule with 8 unknowns, which the genetic algorithm then evolved.

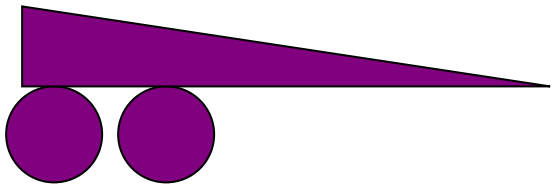
# GENERATE

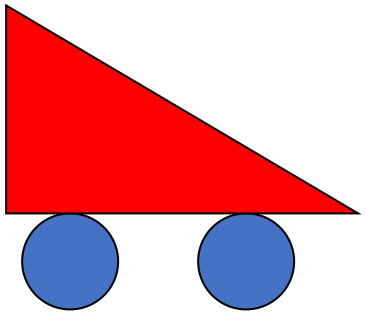


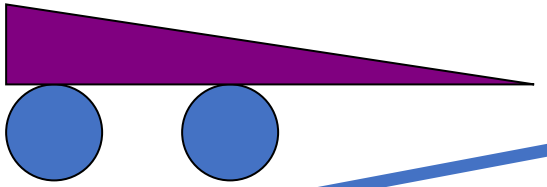




Generalization of properties using
Statistical likelihood

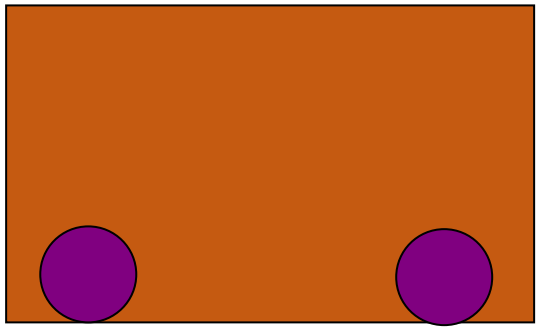


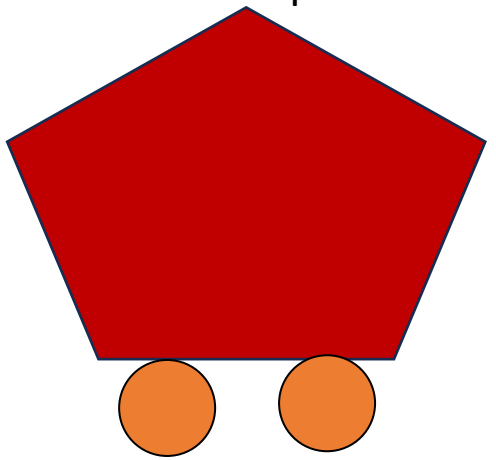




Some geometric shape
On top of two circles

A Rectangle tringle
Or rectangle with
Two same size wheels





# Surprise Test

- Model Product Recommendation as a heuristic search algorithm
  - Define heuristic function
  - Define Start State
  - Define Goal State
  - Traverse search path for a specific example as a hill climbing search
  - Discuss local Maxima and global maxima in your case
- How you can handle multiple goal scenario in this case